A Programmer's Guide to Lambda Calculus

From First Principles to a Working Interpreter in Lean 4

Abstract

This document is a self-contained introduction to the lambda calculus aimed at working programmers who are comfortable reading code but have minimal background in formal logic. We cover the untyped and simply typed lambda calculus, the three core conversions (α, β, η), and the formal notation used in type theory—including *judgments*, *typing contexts*, and the *turnstile* ($\vdash$). Every concept is paired with worked reduction examples and executable Lean 4 code.

Contents

Part I

Foundations

No prior knowledge of formal logic is assumed. If you can write a function in any programming language, you have everything you need.

1 What Is Lambda Calculus?

1.1 The Big Picture

The story begins with a crisis. In 1928, the German mathematician David Hilbert posed the *Entscheidungsproblem* ("decision problem"): is there a mechanical procedure that can determine, given any mathematical statement, whether it is true or false? Hilbert believed the answer was yes—that mathematics could, in principle, be fully automated.

He was wrong, and proving him wrong required answering a more basic question first: *what exactly is a "mechanical procedure"?* Before you can show that no procedure exists for a task, you need a rigorous definition of what a procedure is. In the mid-1930s, several mathematicians independently proposed answers.

Alonzo Church (1903–1995), a logician at Princeton, proposed the lambda calculus in 1936. His idea was radical: computation is nothing more than function application. Define functions, apply them to arguments, simplify the results—that's it. No machine, no tape, no steps through a state table. Just the algebra of functions.

Alan Turing (1912–1954), then a graduate student at Princeton (Church was his PhD advisor!), independently proposed the Turing machine in the same year—an imaginary device with a tape, a read/write head, and a table of rules. His approach was more "mechanical": it modeled computation as a physical process with states and transitions.

Church and Turing proved their models equivalent: anything computable by a Turing machine can be expressed in λ-calculus, and vice versa. This remarkable fact is called the **Church–Turing thesis**. Both men also used their formalisms to prove Hilbert wrong: the Entscheidungsproblem has no solution. There are mathematical statements that no mechanical procedure can decide.

Kurt Gödel, who had already shaken the foundations of mathematics with his incompleteness theorems (1931), showed his own system of recursive functions was equivalent to both. So three independent approaches—functions, machines, and recursion—all captured the same notion of computability. This convergence is strong evidence that the definition is "right."

The word "calculus" here does not mean derivatives and integrals. In mathematics, a **calculus** is just a formal system of rules for manipulating symbols. The lambda calculus is a calculus of functions: its rules tell you how to create functions, apply them, and simplify the results.

1.2 Why Should a Programmer Care?

Every functional language—Haskell, OCaml, Lean, even the functional parts of JavaScript—is essentially λ-calculus with conveniences bolted on. Understanding it tells you what closures really are, why currying works, and what happens under the hood when you pass a function to `map`.

Here is a concrete example. In JavaScript:

```javascript
const add = x => y => x + y;
const increment = add(1);   // returns y => 1 + y
increment(5);               // returns 6
```

This is directly analogous to the lambda calculus expression $\lambda x.\, \lambda y.\, x + y$. The `=>` in JavaScript is literally the λ. The pattern of a function returning a function (**currying**) is how λ-calculus handles multiple arguments. And the mechanism by which `increment` "remembers" that $x = 1$—a **closure**—corresponds to the β-reduction operation we will study in depth.

Programmer's Mental Model

Think of the λ-calculus as an absurdly minimal programming language with exactly *three* constructs: variables, function definitions, and function calls. No `if`, no `while`, no numbers, no strings. Despite this, it is Turing-complete: it can compute anything that any programming language can. We will see how to encode all of those "missing" features using nothing but functions.

2 Reading the Notation

Before diving in, here is a quick guide to the mathematical notation used throughout this document. If you are comfortable with set notation and logic symbols, feel free to skip ahead to Section 1. Otherwise, this page is your cheat sheet—refer back to it whenever a symbol looks unfamiliar.

2.1 Set Notation

Sets are collections of distinct elements, written with curly braces. In this document, we mainly use sets to talk about which variables appear in a term.

Symbol	Name	Meaning and example
$\{x, y\}$	set literal	The set containing x and y. Like a Python `set`: `{x, y}`.
$\in$	element of	$x \in S$ means "x is a member of S." Like Python's `x in S`.
$\notin$	not element of	$x \notin S$ means "x is *not* in S." Like `x not in S`.
$\cup$	union	$A \cup B$ is the set of elements in A *or* B (or both). $\{x\} \cup \{y, z\} = \{x, y, z\}$.
$\setminus$	set difference	$A \setminus B$ is A with all elements of B removed. $\{x, y, z\} \setminus \{y\} = \{x, z\}$. Think: Python's `A - B` for sets.
$\varnothing$	empty set	The set with no elements. Like `set()` in Python. $\mathrm{FV}(e) = \varnothing$ means "e has no free variables."
$\subseteq$	subset	$A \subseteq B$ means every element of A is also in B.

2.2 Arrows and Relations

Arrows describe how terms transform during computation:

Symbol	Name	Meaning
$\longrightarrow_\beta$	beta reduces to	One step of computation. $e \longrightarrow_\beta e'$ means "e reduces to e' in exactly one step."
$\twoheadrightarrow_\beta$	multi-step reduces	Zero or more steps. $e \twoheadrightarrow_\beta e'$ means "e eventually becomes e' after some number of reductions."
$\longrightarrow_\alpha$	alpha converts to	A renaming of bound variables. $\lambda x.\, x \longrightarrow_\alpha \lambda y.\, y$.
$\equiv_\alpha$	alpha-equivalent	Two terms are "the same up to renaming." $\lambda x.\, x \equiv_\alpha \lambda y.\, y$.
$\longrightarrow_\eta$	eta converts to	Wrapping/unwrapping a redundant lambda. $\lambda x.\, f\ x \longrightarrow_\eta f$.
$\triangleq$	defined as	Introduces a new name for an expression. $\textsc{True} \triangleq \lambda t\, f.\, t$ means "we *define* True to be $\lambda t\, f.\, t$."

2.3 Functions and Substitution

Symbol	Name	Meaning
$\mathrm{FV}(e)$	free variables	The set of variables in e that are not bound by any λ. It is a *function* from terms to sets of variables.
$e[x := a]$	substitution	"In term e, replace every free occurrence of variable x with the term a." This is the formal version of "passing an argument."
$\lambda x.\, e$	lambda abstraction	A function with parameter x and body e. Read as "the function that takes x and returns e."
$e_1\ e_2$	application	Apply function e_1 to argument e_2. Juxtaposition (writing terms next to each other) means function call.

2.4 Logic and Typing (used from Section 12 onward)

These symbols appear when we add types to the calculus:

Symbol	Name	Meaning
Γ	context	A list of variable–type assumptions, like a symbol table. $\Gamma = x{:}\mathsf{Nat},\ f{:}\mathsf{Nat}\to\mathsf{Bool}$.
$\vdash$	turnstile	Separates assumptions from conclusions. $\Gamma \vdash e : \tau$ means "given Γ, we can derive that e has type τ." Detailed explanation in Section 12.2.
$\nvdash$	negated turnstile	"We *cannot* derive…" Used to indicate a type error.
$\tau_1 \to \tau_2$	function type	The type of functions taking τ_1 and returning τ_2. Right-associative: $A \to B \to C = A \to (B \to C)$.
$\forall$	for all	Universal quantification. $\forall\alpha.\,\alpha \to \alpha$ means "for every type α, a function from α to α."
$\exists$	there exists	$\exists\, e'$ means "there is some e' such that…"

2.5 Reading Inference Rules

Typing rules are written as **inference rules**: premises above a horizontal line, conclusion below, with an optional rule name on the right.

$$\frac{\text{premise}_1 \qquad \text{premise}_2}{\text{conclusion}}\ \textsc{RuleName}$$

Read this as: "*if* premise_1 and premise_2 are both established, *then* the conclusion follows (by the rule named RuleName)."

Multiple rules can be stacked into a **proof tree** that builds a complete derivation from axioms at the top down to the final conclusion at the bottom. We will see many examples in Section 12.3.

2.6 Grammar Notation

Formal definitions use a grammar notation called BNF:

Symbol	Name	Meaning
$::=$	"is defined as"	Introduces the possible forms of something. $e ::= \ldots$ means "an expression e can be any of the following shapes."
$\mid$	"or"	Separates alternatives. $e ::= x \mid \lambda x.\, e \mid e_1\ e_2$ means "e is either a variable, or a lambda, or an application."
$f^n\ x$	iterated application	Apply f a total of n times: $f^0\ x = x$, $f^1\ x = f\ x$, $f^3\ x = f\ (f\ (f\ x))$.
$\overline{n}$	Church numeral	The overline notation $\overline{n}$ denotes the Church encoding of the natural number n (see Section 10).
$\uparrow_c^d (e)$	shift	De Bruijn index adjustment: add d to all free variable indices $\geq c$ in term e (see Section 11).

2.7 Reading an Equation Step by Step

Let's take one of the scariest-looking equations in this document and break it down piece by piece:

$$\mathrm{FV}(\lambda x.\, e) \;=\; \mathrm{FV}(e) \setminus \{x\}$$

Piece	What it means
$\mathrm{FV}(\ldots)$	FV is a *function* that takes a lambda term and returns the *set of variables* that appear free (unbound) in it.
$\lambda x.\, e$	The input to FV is a lambda abstraction: a function with parameter x and body e.
$=$	"The result is…"
$\mathrm{FV}(e)$	First, recursively compute the free variables of the body e.
$\setminus$	Set difference (remove elements). In Python: `A - B`.
$\{x\}$	The singleton set containing just the variable x.

Putting it all together in English: "The free variables of $\lambda x.\, e$ are the free variables of the body e, *minus* x (because the λ binds x, so it is no longer free)."

Here is another example—the de Bruijn β-reduction rule from Section 11:

$$(\lambda.\, e)\; s \;\longrightarrow_\beta\; \uparrow_0^{-1} \left(e[0 := \uparrow_0^1 (s)]\right)$$

Piece	What it means
$(\lambda.\, e)\; s$	A function (with body e) applied to argument s. This is the "before" side.
$\longrightarrow_\beta$	"Beta-reduces to" (one computation step).
$\uparrow_0^1 (s)$	Shift s up by 1: increase all free variable indices in s by 1, because s is about to move under where a binder used to be.
$e[0 := \ldots]$	In body e, replace every occurrence of variable 0 (the parameter) with the shifted argument.
$\uparrow_0^{-1} (\ldots)$	Shift the whole result down by 1: we removed one λ binder, so all free variable indices decrease by 1 to compensate.

In English: "To apply a function, plug the argument into the body (adjusting index numbering on the way in and out)."

3 Lean 4 in 60 Seconds

If you haven't used Lean, here is the minimum you need:

```
-- Inductive types (algebraic data types)
inductive MyNat where
  | zero : MyNat
  | succ : MyNat → MyNat
```

```
-- Pattern matching + recursion
def add : MyNat → MyNat → MyNat
  | .zero,   m => m
  | .succ n, m => .succ (add n m)

-- Option for partiality, do notation for chaining
def safeDivide (a b : Nat) : Option Nat := do
  if b == 0 then none else some (a / b)
```

Key points: Lean requires termination by default. For β-reduction (which may loop), we'll use a *fuel* parameter.

That is enough to start reading. As the code grows we lean on a handful of Lean conveniences—`do`/`←` notation, `<|>`, `guard`, `Id.run do` with `let mut`, `deriving`, `partial`/`termination_by`, and the odd `by omega`. Each is introduced in passing, but if any feels like magic, Appendix A ("A Lean 4 Primer") collects them in one place and shows the plain `match`/recursor code each one desugars to.

4 The Syntax of Lambda Terms

4.1 The Three Building Blocks

Definition: Lambda Terms

Given a countably infinite set of variables $\mathcal{V} = \{x, y, z, \ldots\}$, the set Λ of **lambda terms** is defined by the grammar:

$$e \;::=\; x \quad \text{(variable)} \;\mid\; \lambda x.\, e \quad \text{(abstraction)} \;\mid\; e_1\; e_2 \quad \text{(application)}$$

where $x \in \mathcal{V}$.

Reading the grammar

The notation $e ::= \ldots$ defines what shapes an expression e can take. The vertical bar $\mid$ means "or." So this says: "an expression is *either* a variable x, *or* an abstraction $\lambda x.\, e$, *or* an application $e_1\; e_2$—and nothing else." This is recursive: the body of a λ is itself an expression e, so terms nest to arbitrary depth. If you have used algebraic data types (`enum` in Rust, `data` in Haskell, `inductive` in Lean), this is the same idea.

Let's unpack each construct in detail.

Variables. A variable is just a name—x, y, z, f, `foo`, any identifier you like. It stands for a value that hasn't been supplied yet, exactly like a function parameter in a programming language. By itself, a variable is the simplest possible lambda term.

Abstraction (function definition). This is how you create a function. The notation $\lambda x.\, e$ is read "lambda x dot e," and it means "the function that takes a parameter called x and returns the expression e." The λ (Greek letter lambda—hence the name of the whole system) announces that a function is being defined. The variable x after the λ is the **parameter name**. The dot separates the parameter from the **body**—the expression e that gets computed when the function is called.

In different languages, this same concept is written differently:

Language	**Syntax for "$\lambda x.\, x + 1$"**
Lambda calculus	$\lambda x.\, x + 1$
JavaScript	`x => x + 1`
Python	`lambda x: x + 1`
Haskell	`\x -> x + 1`
Lean 4	`fun x => x + 1`

They all mean the same thing. In each case, there is a marker ("λ", "`=>`", "`lambda`", etc.) that says "a function is being defined here."

The word **abstraction** is used because the function "abstracts over" its parameter: $\lambda x.\, x + 1$ is a general description of "add one to something," without specifying what that something is.

Application (function call). Written $e_1\ e_2$, this means "apply the function e_1 to the argument e_2." In most programming languages you would write $f(a)$ with parentheses; in lambda calculus, you simply write $f\ a$ (juxtaposition—putting things next to each other). The function comes first, the argument comes second.

Note that e_1 and e_2 are both arbitrary lambda terms. The function can be a variable (like $f\ x$), a lambda (like $(\lambda x.\, x)\ 5$), or even a complex expression. The argument can also be anything. This is what makes lambda calculus so flexible: functions are values just like anything else, so you can pass functions to functions, return functions from functions, and so on.

Lambda term	**JavaScript analog**	**Meaning**
x	`x`	a variable reference
$\lambda x.\, x$	`x => x`	the identity function
$(\lambda x.\, x)\ y$	`(x => x)(y)`	apply identity to y
$\lambda f.\, \lambda x.\, f\ (f\ x)$	`f => x => f(f(x))`	apply f twice

4.2 Parenthesization Conventions

Without conventions, expressions get buried in parentheses. For instance, $f\ g\ h$ is ambiguous: do we mean "apply f to g, then apply the result to h" (i.e., $(f\ g)\ h$), or "apply f to the result of g applied to h" (i.e., $f\ (g\ h)$)? Two standard conventions resolve all ambiguity:

1. **Application is left-associative.** This means $f\ g\ h$ is parsed as $(f\ g)\ h$. Read left to right: first apply f to g, then apply the result to h. This is analogous to how `f(g)(h)` works in JavaScript (call f with g, then call the result with h).
2. **The body of a lambda extends as far right as possible.** This means $\lambda x.\, x\ y$ is parsed as $\lambda x.\, (x\ y)$—"the function that takes x and returns x applied to y." It is *not* $(\lambda x.\, x)\ y$ ("apply the identity to y"). The lambda "gobbles up" everything to its right, until it hits a closing parenthesis or the end of the expression.

Parsing practice

How does $\lambda f.\, \lambda x.\, f\ x\ x$ parse?

1. The outer λf grabs everything to its right: $\lambda f.\, (\lambda x.\, f\ x\ x)$.
2. The inner λx grabs everything to *its* right: $\lambda f.\, (\lambda x.\, (f\ x\ x))$.
3. Application is left-associative: $f\ x\ x = (f\ x)\ x$.

4. Fully parenthesized: $\lambda f.\,(\lambda x.\,((f\ x)\ x))$.

In English: "the function that takes f and returns a function that takes x and returns f applied to x, with the result applied to x again."

4.3 Multi-argument Functions and Currying

Lambda calculus functions take **exactly one argument**. There is no way to write $\lambda(x, y).\,e$—the grammar simply does not allow it. So how do we handle functions that need more than one input?

The trick is called **currying** (named after logician Haskell Curry, though the technique was first described by Moses Schönfinkel in 1924 and used by Gottlob Frege even earlier in the 1890s—Curry himself acknowledged this, but the name stuck). Instead of a function of two arguments, you write a function that takes the first argument and *returns another function* that takes the second argument.

Let's see this concretely. Suppose we want an "add" function that takes two numbers. We cannot write $\lambda(x, y).\,x + y$. Instead, we write:

$$\lambda x.\,\lambda y.\,x + y$$

This is a function that takes x and returns $\lambda y.\,x + y$—a new function that "remembers" x and waits for y. Let's trace through using it to add 3 and 4:

Currying in action

$$\underbrace{(\lambda x.\,\lambda y.\,x + y)}_{\text{"add"}}\ 3\ \ 4$$

$$\longrightarrow_\beta\ \underbrace{(\lambda y.\,3 + y)}_{\text{"add 3 to…"}}\ \ 4 \quad \text{(substituted 3 for } x\text{; got a function waiting for } y\text{)}$$

$$\longrightarrow_\beta\ 3 + 4 = 7 \quad \text{(substituted 4 for } y\text{)}$$

The intermediate result $\lambda y.\,3 + y$ is a **partially applied** function—it has received one argument but is still waiting for another. In JavaScript, this pattern looks like `add(3)(4)` or equivalently:

```
const add = x => y => x + y;
const add3 = add(3);    // y => 3 + y  (partial application!)
add3(4);                // 7
```

This extends to any number of arguments: a function of n arguments is a chain of n nested lambdas. As shorthand, we write $\lambda x\,y\,z.\,e$ for $\lambda x.\,\lambda y.\,\lambda z.\,e$.

4.4 Let Bindings Are Just Application

Most programming languages have `let` for naming intermediate values:

```
let x = 2 + 3 in x * x    // evaluates to 25
```

Lambda calculus has no built-in `let`, but it doesn't need one. A `let` expression is just syntactic

sugar for a function application:

$$\underbrace{\texttt{let } x = e_1 \texttt{ in } e_2}_{\text{let binding}} \quad \equiv \quad \underbrace{(\lambda x.\, e_2)\ e_1}_{\text{application}}$$

This works because $(\lambda x.\, e_2)\ e_1$ does exactly what `let` does: it evaluates e_1, binds the result to x, then evaluates e_2 with x available. Let's verify:

$$(\lambda x.\, x \cdot x)\ (2+3) \longrightarrow_\beta (2+3) \cdot (2+3) = 5 \cdot 5 = 25$$

Why this matters

Whenever you see `let` in a functional language, remember: it's not a new primitive. It's just a λ being applied immediately. This is one of the beauties of lambda calculus—seemingly essential features dissolve into the three basic constructs. We add `let` support to our parser in Section 21 as a desugaring step.

We have the theory of syntax in hand. Over the rest of this book we will turn it into running Lean code, building up step by step to De Bruijn indices, type checking, and ultimately a dependent type checker that verifies proofs. The first step is the humblest one: choosing a data structure for a λ-term.

4.5 Implementing It: Representing Terms

Your first design decision: how do you represent a λ-term as a data structure?

4.5.1 Think About It

Go back to Section 4. A λ-term has exactly three possible shapes: it's a variable, a lambda abstraction, or an application. That's it— there are no other cases.

Design challenge

4.1. Before reading further, write down (on paper or in your head) a data type that can represent any λ-term. What fields does each case need? How would you represent $\lambda x.\, x\ y$?

4.5.2 The Answer: An Algebraic Data Type

Each shape becomes a constructor:

```
inductive Term where
  | var : String → Term            -- a variable carries its name
  | lam : String → Term → Term     -- a lambda carries parameter name + body
  | app : Term → Term → Term       -- an application carries two sub-terms
  deriving Repr, DecidableEq, Inhabited
```

The `deriving` clause asks Lean to auto-generate three typeclass instances for `Term`:

- `Repr`: Produces string representations so `#eval` can display terms in the console (e.g., `Term.lam "x" (Term.var "x")`).
- `DecidableEq`: Lets us use `==` to compare terms for structural equality, and also unlocks `if x == y` and list membership tests like $x \in xs$—both of which we'll need in substitution.

- `Inhabited`: Provides a default value for the type (here, `Term.var ""`). Lean sometimes requires this when using certain combinators or partial functions.

The term $\lambda x.\, x\ y$ becomes `lam "x" (app (var "x") (var "y"))`. This is a tree—the same tree structure we visualized in Section 7. Variables are leaves, applications are internal nodes with two children, and lambdas are internal nodes with one child plus a name annotation.

Notice how the data type mirrors the grammar almost exactly:

Grammar		**Lean**
$e ::= x$	$\longleftrightarrow$	`var : String → Term`
$e ::= \lambda x.\, e$	$\longleftrightarrow$	`lam : String → Term → Term`
$e ::= e_1\ e_2$	$\longleftrightarrow$	`app : Term → Term → Term`

This is a recurring pattern: *grammars translate directly into algebraic data types.* If you can write the grammar, you can write the type.

5 Free and Bound Variables

5.1 Scope in Lambda Calculus

Every programmer understands **scope**: a variable declared inside a function is only visible within that function. Lambda calculus has the same concept.

When you write $\lambda x.\, e$, the λ creates a **binding** for the variable x, and the **scope** of that binding is the body e. Within e, the variable x is said to be **bound**—it has a definition (it's the function's parameter). A variable that is *not* bound by any enclosing λ is **free**—it refers to something defined elsewhere, like a global variable or a variable from an outer scope.

Identifying free and bound variables

Consider $\lambda x.\, x\ y$. This is a function that takes x and applies it to y. Here:

1. x is **bound**—it's the parameter of the λ.
2. y is **free**—no λ binds it, so its meaning depends on the surrounding context.

As a Python analogy:

```python
y = 10              # y is defined outside
f = lambda x: x + y  # x is bound (parameter), y is free (from outer scope)
```

If y is not defined anywhere, the expression is incomplete—it has an undefined variable. A term with no free variables is self-contained.

5.2 Formal Definition

Definition: Free Variables

$$\begin{aligned} \mathrm{FV}(x) &= \{x\} \\ \mathrm{FV}(\lambda x.\, e) &= \mathrm{FV}(e) \setminus \{x\} \\ \mathrm{FV}(e_1\ e_2) &= \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2) \end{aligned}$$

Reading the definition line by line

This is a recursive function from terms to sets of variable names. Reading each line (see Section 2 for the symbols):

1. $\mathrm{FV}(x) = \{x\}$: A bare variable x is free—it's not under any λ, so the set of free variables is just $\{x\}$.
2. $\mathrm{FV}(\lambda x.\, e) = \mathrm{FV}(e) \setminus \{x\}$: A lambda *binds* x, so take the free variables of the body e and *remove* x from them (the "$\setminus$" is set difference, like subtracting $\{x\}$ from the set).
3. $\mathrm{FV}(e_1\ e_2) = \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2)$: An application's free variables are the *union* ($\cup$) of both subterms' free variables—anything free on either side is free in the whole expression.

Computing FV step by step

Let's compute $\mathrm{FV}(\lambda x.\, x\ y)$ from scratch:

$$\begin{aligned}
\mathrm{FV}(\lambda x.\, x\ y) &= \mathrm{FV}(x\ y) \setminus \{x\} && \text{(rule 2: peel off the } \lambda x \text{ and remove } x\text{)} \\
&= \big(\mathrm{FV}(x) \cup \mathrm{FV}(y)\big) \setminus \{x\} && \text{(rule 3: union both sides of the application)} \\
&= \big(\{x\} \cup \{y\}\big) \setminus \{x\} && \text{(rule 1: each variable is its own singleton)} \\
&= \{x, y\} \setminus \{x\} && \text{(perform the union)} \\
&= \{y\} && \text{(remove } x \text{ from the set)}
\end{aligned}$$

Only y is free. This matches our earlier analysis: x is the parameter (bound), y comes from outside (free).

5.3 Closed Terms (Combinators)

A term with $\mathrm{FV}(e) = \varnothing$ (no free variables at all) is **closed**—it's completely self-contained, like a standalone program with no undefined variables. Closed terms are also called **combinators**. The three most famous are:

$$\mathbf{I} = \lambda x.\, x \qquad \mathbf{K} = \lambda x\, y.\, x \qquad \mathbf{S} = \lambda f\, g\, x.\, f\ x\ (g\ x)$$

I is the identity (returns its argument unchanged). **K** takes two arguments and returns the first, ignoring the second. **S** takes three arguments and "distributes" the third to the first two.

These three combinators form a basis: every λ-term can be expressed using just **S** and **K** (since $\mathbf{I} = \mathbf{S}\ \mathbf{K}\ \mathbf{K}$, as we will verify later).

Historical note: Combinatory logic

The **S**, **K**, **I** combinators actually predate lambda calculus. They were introduced by Moses Schönfinkel in a 1924 paper, "On the building blocks of mathematical logic," in which he showed that all of logic could be expressed using just two operations (**S** and **K**). Haskell Curry independently rediscovered and greatly expanded these ideas starting in 1927, developing them into a full theory called **combinatory logic**. Church's lambda calculus (1936) took a different but equivalent path—using variable binding rather than combinators as the primitive. The two systems are interconvertible and equally expressive.

5.4 The Danger: Variable Capture

Variable Capture

When we perform substitution (which we will define formally in Section 7), we must be careful not to accidentally change the meaning of a term by allowing a free variable to become bound. This problem is called **variable capture**.

Consider: what happens if we substitute y for x in $\lambda y.\, x$? The original term is a function that ignores its argument and returns whatever x is (a free variable). Naively replacing x with y gives $\lambda y.\, y$—but that's the *identity* function, a completely different thing! The free y got "captured" by the λy binder.

The fix: **rename the binder first**. Change $\lambda y.\, x$ to $\lambda z.\, x$ (which is the same function—the parameter name doesn't matter), and *then* substitute to get $\lambda z.\, y$. This correctly preserves the meaning: a function that ignores its argument and returns y.

This renaming operation is called **alpha conversion**, and it's the subject of the next section.

5.5 Implementing It: Free Variables in Lean

Let's turn the formal definition of free variables into code.

5.5.1 Free Variables

We need $\mathrm{FV}(e)$ in code. Go back to the formal definition from Section 5: $\mathrm{FV}(x) = \{x\}$, $\mathrm{FV}(\lambda x.\, e) = \mathrm{FV}(e) \setminus \{x\}$, $\mathrm{FV}(e_1\ e_2) = \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2)$.

Implementation challenge

5.1. Translate the free variables definition into a function `freeVars : Term → List String`. Each case of the formal definition becomes one pattern-match branch. Try it yourself before reading on.

Look at the formal definition again. It has three cases, one for each shape of term. Your data type has three constructors. So the function will have three branches. Try writing them as comments first:

```
def freeVars : Term → List String
  | .var x     => ???   -- FV(x) = {x}
  | .lam x e   => ???   -- FV(λx.e) = FV(e) \ {x}
  | .app e₁ e₂ => ???   -- FV(e₁ e₂) = FV(e₁) ∪ FV(e₂)
```

Now fill in each `???`. For the variable case, the free variables are just the variable itself: a one-element list. For the lambda case, you need "all free variables of the body, except x"—that's a filter. For the application case, you need the union of both sides—concatenation with duplicates removed.

The translation is almost mechanical:

```
def freeVars : Term → List String
  | .var x     => [x]                                  -- FV(x) = {x}
  | .lam x e   => (freeVars e).filter (· != x)         -- FV(λx.e) = FV(e)\{x}
  | .app e₁ e₂ => (freeVars e₁ ++ freeVars e₂).eraseDups  -- union
```

This is a pattern you'll see repeatedly: **formal definitions translate almost line-for-line into code**. If you understand the math, you can write the program.

6 Alpha Conversion (α)

6.1 The Core Idea: Names Don't Matter

Here is a question: are these two JavaScript functions the same?

```
const f = x => x + 1;
const g = y => y + 1;
```

Obviously yes. They both add 1 to their argument. The fact that one calls its parameter `x` and the other calls it `y` is irrelevant—parameter names are arbitrary labels.

Alpha conversion (α-conversion) is the formal version of this observation. It says that you can rename a bound variable at any time, as long as you do it consistently and don't create a name clash.

The α-conversion Rule

$$\lambda x.\, e \quad \longrightarrow_\alpha \quad \lambda y.\, e[x := y] \qquad \text{provided } y \notin \mathrm{FV}(e) \text{ and } y \notin \mathrm{BV}(e)$$

Two terms differing only in bound variable names are α-equivalent: $\lambda x.\, x \;\equiv_\alpha\; \lambda y.\, y \;\equiv_\alpha\; \lambda z.\, z$.

The notation $e[x := y]$ here means "replace every free x in e with y" (the same substitution operation from the next section). The two conditions ensure safety: y must not already appear in e either free or bound, so the renaming doesn't clash with any existing names.

6.2 Why Alpha Conversion Matters

Alpha conversion is not just a theoretical nicety. As we saw in Section 5, without it, substitution can go wrong due to variable capture. Every lambda calculus implementation must handle α-equivalence. In practice, there are three common approaches:

1. Rename variables on the fly during substitution (the classical approach, which we implement in our named Lean 4 interpreter).
2. Use **de Bruijn indices**, where variables are numbers rather than names, so α-equivalence becomes structural equality (see Section 11).
3. Use a locally nameless representation, combining the best of both.

Alpha-converting $\lambda x.\, \lambda y.\, x\ y$

Rename the outer x to z:

1. Check that z is not free in the body $(\lambda y.\, x\ y)$. We have $\mathrm{FV}(\lambda y.\, x\ y) = \{x\}$, and $z \notin \{x\}$. ✓
2. Replace every free x in the body with z: $\lambda y.\, z\ y$.
3. Result: $\lambda z.\, \lambda y.\, z\ y$.

Now rename the inner y to w:

1. Check $w \notin \mathrm{FV}(z\ y) = \{z, y\}$. Since w is neither z nor y: ✓
2. Replace: $z\ w$.
3. Result: $\lambda z.\, \lambda w.\, z\ w$.

The original $\lambda x.\, \lambda y.\, x\ y$ and the final $\lambda z.\, \lambda w.\, z\ w$ are α-equivalent ($\equiv_\alpha$). They are the same function—just with different parameter names.

Exercises: Variables and Renaming

6.1. Compute $\mathrm{FV}(\lambda x.\, \lambda y.\, x\ z\ y)$. Which variables are free? Which are bound?
6.2. Is the term $\lambda f.\, \lambda x.\, f\ (f\ x)$ closed? Why or why not?
6.3. Alpha-convert $\lambda x.\, \lambda x.\, x$ so that no variable name is used twice. What does this function do?
6.4. Can you alpha-convert $\lambda x.\, x\ y$ to $\lambda y.\, y\ y$? Why or why not?

7 Beta Reduction (β)

7.1 The Heart of Computation

Alpha conversion renames things. **Beta reduction** is where actual *computation* happens. It is the single rule that makes lambda calculus a model of computation, and it corresponds to something every programmer does constantly: **calling a function**.

When you write $(\lambda x.\, x + 1)\ 5$ and evaluate it, you get $5 + 1 = 6$. What happened? You took the function $\lambda x.\, x + 1$ ("add 1 to the argument"), and *plugged in* 5 for x in the body. That "plugging in" is beta reduction.

The β-reduction Rule

$$\boxed{(\lambda x.\, e)\ a \quad \longrightarrow_\beta \quad e[x := a]}$$

Read this as: "a function $\lambda x.\, e$ applied to an argument a reduces to the body e with every free x replaced by a."

The expression $(\lambda x.\, e)\ a$ is called a **redex** (short for "reducible expression"—a spot where a reduction can happen). The result $e[x := a]$ is called the **reduct**.

What is happening mechanically?

Beta reduction is function calling, broken into two parts:

1. **Pattern match.** Look at the term. If it has the shape "something applied to something," *and* the left side is a λ-abstraction, then you have a redex.
2. **Substitute.** Take the body of the λ, find every place the parameter appears, and replace it with the argument. Erase the λ wrapper.

That's it. All of computation in the λ-calculus is just pattern-matching on this shape and performing substitution.

7.2 Why Is This "Computation"?

At first, beta reduction might seem too simple to deserve the name "computation." All we're doing is substituting one thing for another—how can that express loops, conditionals, data structures, and everything else a real program needs?

This question is worth sitting with. In 1933, when Church first defined the λ-calculus, it was not at all obvious that it had anything to do with computation. Church was working on the foundations of logic, trying to build a system where mathematical functions could be treated as first-class objects. The substitution rule was just the natural way functions should behave:

if you define $f(x) = x + 1$ and then ask for $f(5)$, you substitute 5 for x and get 6. Nothing controversial there.

The surprise came when Church realized that this simple mechanism was *universal*. The key insight unfolds in three steps.

Step 1: You can encode data as behavior. Lambda calculus has no built-in data types. But you can represent any piece of data as a function that describes *how it behaves*. Define $\textsc{True} \triangleq \lambda t.\, \lambda f.\, t$ and $\textsc{False} \triangleq \lambda t.\, \lambda f.\, f$. These are just functions, but they behave like booleans: True selects the first of two arguments, False selects the second. Now "if b then x else y" is just $b\ x\ y$—a beta reduction:

$$\textsc{True}\ \texttt{"yes"}\ \texttt{"no"} \longrightarrow_\beta (\lambda f.\, \texttt{"yes"})\ \texttt{"no"} \longrightarrow_\beta \texttt{"yes"}$$

We just evaluated a conditional using nothing but substitution. Similarly, define the number n as a function that applies f exactly n times: $\overline{3} = \lambda f.\, \lambda x.\, f\ (f\ (f\ x))$. Now "add 1" is just "apply f one more time"—again, a beta reduction. Arithmetic from substitution.

Step 2: You can express recursion. The term $(\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$ reduces to itself. This *is* an infinite loop, expressed as substitution. And by modifying this pattern slightly (the **Y** combinator, Section 10.7), we get controlled recursion: a function that calls itself as many times as needed and then stops.

This is where the magic happens. With data encoding and recursion, you can express every computable function as a chain of beta reductions.

Step 3: This is *all* of computation. Church's 1936 paper made this precise. He defined a class of functions called "λ-definable functions"—functions on natural numbers that could be expressed as λ-terms using Church numerals. He then showed that this class included all functions that mathematicians intuitively considered "effectively calculable": addition, multiplication, exponentiation, primality testing, and crucially, the recursive functions defined earlier by Gödel and Herbrand.

This was a bold claim. Church was saying: *if a function can be computed at all, by any method whatsoever, then it can be computed by beta reduction.*

Turing, independently working on the same problem, proved the same thing for his Turing machines using a different approach. When Church (1936) and Turing (1937) proved that λ-definable functions and Turing-computable functions were exactly the same class, the result was electrifying. Two completely different formalizations of "computation"—one based on abstract symbol manipulation, the other on an imaginary physical machine—captured the same set of functions. This convergence was strong evidence that both had found something fundamental.

This is the **Church–Turing thesis**: any effectively calculable function is computable by a Turing machine / expressible in λ-calculus. It is not a theorem (you cannot prove it, because "effectively calculable" is an informal notion), but no counterexample has ever been found. Every alternative model of computation—register machines, cellular automata, quantum computers, programming languages—has turned out to be equivalent in power.

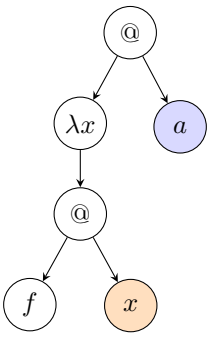
The philosophical punchline. A system with nothing but function definitions and function calls, where the only operation is substituting arguments into function bodies, is powerful enough to express *all of computation*. Beta reduction is not just "a rule"—it is *the* rule. Every program you have ever written, in any language, could in principle be expressed as a lambda

term and evaluated by iterated substitution. The elaborate machinery of modern computers—registers, memory, instruction sets—is, in a precise mathematical sense, no more powerful than pen-and-paper substitution.

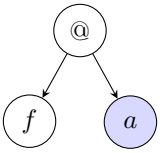
7.3 Visualizing Reduction as Tree Surgery

Lambda terms have a natural tree structure. Seeing them as trees makes β-reduction much more concrete: it is literally a tree transformation where you splice the argument subtree into the body.

Consider $(\lambda x.\, f\ x)\ a$. As a tree, the top node is an application ("@"), whose left child is $\lambda x.\, f\ x$ and whose right child is a:

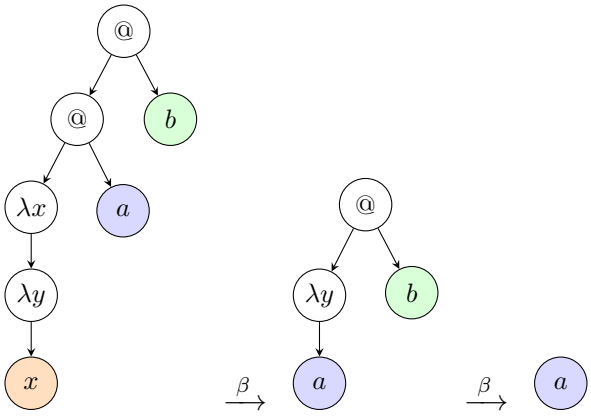


β-reduction works in three steps: (1) identify the redex (the "@" whose left child is a λ), (2) take the right child (a, shaded blue) and substitute it for every leaf labeled x (shaded orange), (3) remove the λ-node and the "@" above it. The result:



The variable leaf x has been replaced by the argument a. This is β-reduction visualized as tree surgery.

Here is a more complex example. The **K** combinator applied to two arguments: $(\lambda x.\, \lambda y.\, x)\ a\ b$. Because application is left-associative, this is $((\lambda x.\, \lambda y.\, x)\ a)\ b$:



First reduction: the inner "@" fires, substituting a for x inside $\lambda y.\, x$, giving $\lambda y.\, a$. Second reduction: the outer "@" fires, substituting b for y in a—but a has no y, so b is discarded. Final result: a.

7.4 Substitution: The Engine Behind Beta Reduction

Beta reduction says $(\lambda x.\, e)\ a \longrightarrow_\beta e[x := a]$. The hard part is not the pattern-matching (finding a λ applied to something). The hard part is the operation $e[x := a]$: "replace every free x in e with a." This is called **substitution**, and getting it right is the most subtle aspect of lambda calculus.

7.4.1 Why Not Just Find-and-Replace?

Your first instinct might be: "substitution is just find-and-replace—search for every x in e and swap in a." And in many cases, that works perfectly:

$$(x + 1)[x := 5] \;=\; 5 + 1 \quad \checkmark$$

But lambda calculus has **variable binding**—the λ operator creates a new scope. This makes naive find-and-replace dangerous. Consider three scenarios that show why:

Problem 1: Don't replace bound variables. In $(\lambda x.\, x)[x := 5]$, should we replace x with 5? No! The x inside $\lambda x.\, x$ is *bound* by that λ—it's a fresh parameter, a local variable with its own scope. The outer substitution should not touch it. Just as in this Python code:

```python
x = 5
f = lambda x: x    # this x is a NEW x, shadowing the outer one
f(10)               # returns 10, not 5
```

The correct result is: $(\lambda x.\, x)[x := 5] = \lambda x.\, x$. The substitution stops at the λx because x is **shadowed**.

Problem 2: Do substitute into other binders. What about $(\lambda y.\, x)[x := 5]$? Here the λ binds y, not x. The x in the body is still free—it refers to the outer x, which we're substituting. So we should go inside:

$$(\lambda y.\, x)[x := 5] \;=\; \lambda y.\, 5 \quad \checkmark$$

Problem 3: Variable capture—the real danger. Now the tricky case. What about $(\lambda y.\, x)[x := y]$? We want to replace the free x with y. Naively:

$$(\lambda y.\, x)[x := y] \overset{\text{WRONG}}{=} \lambda y.\, y$$

We got $\lambda y.\, y$—the identity function! But the original term $\lambda y.\, x$ is *not* the identity. It's a function that ignores its argument and returns the (free) variable x. By blindly substituting y into a scope where y is already bound, we accidentally **captured** the free y, turning it into a bound variable. The y we substituted in now refers to the lambda's parameter instead of the outer variable.

This is called **variable capture**, and it produces wrong results. The fix is to rename the conflicting binder before substituting:

$$(\lambda y.\, x)[x := y] \;=\; (\lambda z.\, x)[x := y] \quad (\text{rename } y \to z \text{ first}) \;=\; \lambda z.\, y \quad \checkmark$$

Now the free y stays free, as intended.

7.4.2 The Complete Rules

With the motivation clear, here is the formal definition. Each rule handles one shape of expression:

Capture-Avoiding Substitution

$$
\begin{aligned}
x[x := a] &= a && \text{(1) target variable: replace it}\\
y[x := a] &= y && \text{(2) different variable: leave it}\\
(\lambda x.\, e)[x := a] &= \lambda x.\, e && \text{(3) shadowed: stop}\\
(\lambda y.\, e)[x := a] &= \lambda y.\, (e[x := a]) && \text{(4) safe: go inside}\\
&\qquad \text{if } y \neq x,\ y \notin \mathrm{FV}(a) &&\\
(\lambda y.\, e)[x := a] &= \lambda z.\, (e[y := z][x := a]) && \text{(5) capture: rename first}\\
&\qquad \text{if } y \neq x,\ y \in \mathrm{FV}(a),\ z \text{ fresh} &&\\
(e_1\ e_2)[x := a] &= (e_1[x := a])\ (e_2[x := a]) && \text{(6) application: both sides}
\end{aligned}
$$

7.4.3 Rule by Rule, in Depth

Let's go through each rule carefully, with the reasoning behind it and a concrete example.

Rule 1: $x[x := a] = a$**.** The simplest case. We found the variable we're substituting for, so we replace it. Example: $x[x := 5] = 5$.

Rule 2: $y[x := a] = y$ **when** $y \neq x$**.** We found a variable, but it's not the one we're looking for. Leave it alone. Example: $y[x := 5] = y$.

Rule 3: $(\lambda x.\, e)[x := a] = \lambda x.\, e$ **(shadowing).** The λ creates a *new* x that shadows the one we're substituting for. Inside this λ, every x refers to the parameter, not the outer variable. So we stop—the substitution has no effect.

Think of it like nested function definitions:

```
x = 10
def f(x):       # this x shadows the outer x
    return x    # refers to the parameter, not 10
```

Example: $(\lambda x.\, x\ x)[x := \mathbf{I}] = \lambda x.\, x\ x$. The x's inside are bound, not free.

Rule 4: $(\lambda y.\, e)[x := a] = \lambda y.\, (e[x := a])$ **(safe descent).** The λ binds a *different* variable y, so it doesn't shadow x. We can safely substitute inside the body. But there is a condition: $y \notin \mathrm{FV}(a)$. This means the term a (what we're plugging in) does not mention y freely. Since a doesn't mention y, going under λy can't accidentally capture anything.

Example: $(\lambda y.\, x\ y)[x := \lambda z.\, z] = \lambda y.\, (\lambda z.\, z)\ y$. Here $a = \lambda z.\, z$, and $y \notin \mathrm{FV}(\lambda z.\, z) = \varnothing$, so it's safe.

Rule 5: $(\lambda y.\, e)[x := a] = \lambda z.\, (e[y := z][x := a])$ **(capture avoidance).** This is the critical rule—the one that makes substitution "capture-avoiding." We need to substitute inside λy, but a contains y as a free variable. If we went ahead and substituted, the λy would capture the y in a, changing its meaning.

The fix is a two-step process:

1. α-convert: rename the bound y to a fresh name z that appears nowhere in the expression (specifically, $z \notin \mathrm{FV}(e) \cup \mathrm{FV}(a) \cup \{x\}$). This gives $\lambda z.\, e[y := z]$.
2. Now it's safe to substitute a into the body, because the binder is z, not y, so no capture can occur.

Rule 6: $(e_1\ e_2)[x := a] = (e_1[x := a])\ (e_2[x := a])$ **(application).** An application has no binders—it's just two expressions side by side. So we substitute into both independently. Example: $(f\ x)[x := 5] = f\ 5$.

7.4.4 Worked Example: Capture Avoidance in Action

Let's trace through the dangerous case step by step. We want to compute:

$$(\lambda y.\, x\ y)[x := y]$$

In English: we're substituting y for x inside the term $\lambda y.\, x\ y$.

Step 1: Identify which rule applies. The outermost structure is $\lambda y.\,(\ldots)$, and we're substituting for x (not y), so rules 1–3 don't apply. We need either rule 4 or 5. Check: is $y \in \mathrm{FV}(a)$? Here $a = y$, so $\mathrm{FV}(a) = \{y\}$. Yes, $y \in \mathrm{FV}(a)$. This means rule 5 applies—we're in the dangerous case.

Step 2: Pick a fresh name. We need z that doesn't appear free in e, a, or equal x. We have $\mathrm{FV}(e) = \mathrm{FV}(x\ y) = \{x, y\}$, $\mathrm{FV}(a) = \{y\}$, and x is the substitution variable. So we need $z \notin \{x, y\}$. Pick z.

Step 3: Rename the binder. α-convert $\lambda y.\, x\ y$ to $\lambda z.\, x\ z$ (replacing the bound y with z in the body):

$$(\lambda y.\, x\ y)\ \equiv_\alpha\ (\lambda z.\, x\ z)$$

Step 4: Now substitute safely.

$$(\lambda z.\, x\ z)[x := y]\ =\ \lambda z.\, (x\ z)[x := y]$$

Now apply rule 6 (application): $(x\ z)[x := y] = x[x := y]\ z[x := y] = y \cdot z$ (by rules 1 and 2). So:

$$(\lambda y.\, x\ y)[x := y]\ =\ \lambda z.\, y\ z$$

The free y in the result is still free—it refers to whatever y was in the surrounding context. If we had skipped the renaming, we would have gotten $\lambda y.\, y\ y$ (the self-application function), which is completely wrong.

Why these rules exist

The six rules encode a simple principle: substitution should respect scope. Rules 1–2 handle variables. Rule 6 handles applications (which have no scope). The interesting rules are 3–5, which handle binders:
Rule 3 says: if the binder re-introduces the same variable we're substituting for, the outer variable is no longer visible here. This is **shadowing**, exactly like local variables in any programming language.
Rules 4–5 say: if the binder introduces a *different* variable, we can go inside—but only if it's safe. The safety check ($y \notin \mathrm{FV}(a)$) prevents variable capture. If the check fails, we rename first.
The deep reason these rules are necessary is that lambda calculus uses **names** to connect

variables to their binders. Names are convenient for humans but create a bookkeeping problem: different binders can accidentally use the same name, and substitution can introduce new name collisions. De Bruijn indices (Section 11) eliminate this problem entirely by using positions instead of names—which is why most implementations prefer them.

7.4.5 A Flowchart for Substitution

When computing $e[x := a]$, follow this decision process:

1. **Is e a variable?**
   - Is it x? $\longrightarrow$ Replace with a. (Rule 1)
   - Is it something else? $\longrightarrow$ Leave it alone. (Rule 2)
2. **Is e an application $e_1\ e_2$?** $\longrightarrow$ Substitute into both sides: $(e_1[x := a])\ (e_2[x := a])$. (Rule 6)
3. **Is e a λ-abstraction $\lambda y.\, body$?**
   - Is $y = x$? $\longrightarrow$ Stop. The λ shadows x. (Rule 3)
   - Is $y \neq x$ and $y \notin \mathrm{FV}(a)$? $\longrightarrow$ Go inside: $\lambda y.\, (body[x := a])$. (Rule 4)
   - Is $y \neq x$ and $y \in \mathrm{FV}(a)$? $\longrightarrow$ **Danger!** Rename y to a fresh z first, *then* go inside. (Rule 5)

7.5 Worked Reductions

Let's work through several beta reductions step by step, starting from the simplest and building up.

The simplest reduction

$$(\lambda x.\, x)\ 5 \quad \longrightarrow_\beta \quad 5$$

The identity function $\lambda x.\, x$ applied to 5: replace x with 5 in the body (x), giving 5. The function returns its argument unchanged.

Identity applied to identity

$$(\lambda x.\, x)\ (\lambda y.\, y) \quad \longrightarrow_\beta \quad \lambda y.\, y \quad \text{(substitute } (\lambda y.\, y) \text{ for } x \text{ in body } x)$$

The argument is itself a function—that's fine! In lambda calculus, functions are values like any other. The result is just $\lambda y.\, y$ (the identity function), returned unchanged.

The K combinator discards its second argument

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\ a\ b &\longrightarrow_\beta (\lambda y.\, a)\ b \quad \text{(substitute } a \text{ for } x\text{; the inner } \lambda y \text{ remains)} \\ &\longrightarrow_\beta a \quad \text{(substitute } b \text{ for } y \text{ in body } a\text{; but } y \notin \mathrm{FV}(a)\text{, so nothing changes)}\end{aligned}$$

The **K** combinator takes two arguments and returns the first, discarding the second. Notice that the second reduction "does nothing" because a doesn't mention y—the argument b is simply thrown away.

A multi-step reduction: S K K = I

$\mathbf{S} = \lambda f\, g\, x.\, f\ x\ (g\ x)$. Let's apply it to $\mathbf{K}$ twice:

$$\begin{aligned}\mathbf{S}\ \mathbf{K}\ \mathbf{K} &= (\lambda f\, g\, x.\, f\ x\ (g\ x))\ \mathbf{K}\ \mathbf{K} \quad \text{(expand } \mathbf{S}\text{)}\\ &\longrightarrow_\beta (\lambda g\, x.\, \mathbf{K}\ x\ (g\ x))\ \mathbf{K} \quad \text{(plug in first } \mathbf{K} \text{ for } f\text{)}\\ &\longrightarrow_\beta \lambda x.\, \mathbf{K}\ x\ (\mathbf{K}\ x) \quad \text{(plug in second } \mathbf{K} \text{ for } g\text{)}\end{aligned}$$

Now simplify the body. Recall $\mathbf{K}\ x\ (\mathbf{K}\ x)$: $\mathbf{K}$ takes two arguments and returns the first. So $\mathbf{K}\ x$ (anything) $\twoheadrightarrow_\beta x$:

$$\lambda x.\, \mathbf{K}\ x\ (\mathbf{K}\ x)\ \twoheadrightarrow_\beta\ \lambda x.\, x\ =\ \mathbf{I}$$

We've derived the identity function from $\mathbf{S}$ and $\mathbf{K}$ alone!

Non-termination: Ω

Now something surprising. Consider the term $\lambda x.\, x\ x$: a function that applies its argument *to itself*. What happens when we apply this to a copy of itself?

$$\begin{aligned}\Omega &= (\lambda x.\, x\ x)\ (\lambda x.\, x\ x)\\ &\longrightarrow_\beta (\lambda x.\, x\ x)\ (\lambda x.\, x\ x) \quad \text{(substitute } (\lambda x.\, x\ x) \text{ for } x \text{ in } x\ x\text{)}\\ &= \Omega\end{aligned}$$

We got right back where we started! Ω reduces to itself, forever. This is the λ-calculus equivalent of an **infinite loop**. It shows that not all lambda terms terminate—which makes sense, since any system powerful enough to express all computable functions must also be able to express non-terminating ones.

7.6 Normal Forms: When Computation Is "Done"

When you evaluate $(\lambda x.\, x)\ 5$ and get 5, you're done—there's nothing left to reduce. But when you try Ω, the reduction never ends. How do we formalize the difference?

A **redex** is any sub-expression with the shape $(\lambda x.\, e)\ a$—a function directly applied to an argument. A term in **normal form** contains *no* redexes anywhere inside it. There is nowhere left to apply the β-rule, so computation has finished.

Some examples:

1. x is in normal form (just a variable, no redex).
2. $\lambda x.\, x$ is in normal form (a function, but not applied to anything).
3. $x\ y$ is in normal form (an application, but x is not a λ).
4. $(\lambda x.\, x)\ y$ is *not* in normal form—it contains a redex.
5. Ω is *not* in normal form, and it never reaches one.

There is also a weaker notion called **weak head normal form** (WHNF): a term is in WHNF if its outermost structure is either a λ-abstraction or an application whose leftmost part is not a redex. Most practical implementations (including Haskell's runtime) stop at WHNF because fully normalizing under lambdas is expensive and usually unnecessary.

Theorem 7.1 (Church–Rosser / Confluence)**.** If $e \twoheadrightarrow_\beta e_1$ and $e \twoheadrightarrow_\beta e_2$ (possibly via different reduction paths), then there exists some e' such that $e_1 \twoheadrightarrow_\beta e'$ and $e_2 \twoheadrightarrow_\beta e'$. In particular, **if a normal form exists, it is unique** up to $\equiv_\alpha$.

What Church–Rosser means

This theorem says: it doesn't matter what *order* you reduce things in. You might reduce the left sub-expression first, or the right one first, or something deep inside the term. As long as you *eventually* reach a normal form, it will be **the same** normal form regardless of the path you took. Different paths may take different numbers of steps (and some paths may diverge entirely), but they can never reach *different* normal forms. This is a very reassuring property—it means the "answer" is well-defined.

Alonzo Church and J. Barkley Rosser proved this in 1936. It was a crucial result: without it, the λ-calculus would be ambiguous as a model of computation, since different evaluation orders could yield different "answers." The proof was notoriously difficult—Church himself called it one of the hardest things he had worked on—and it took decades for truly elegant proofs to emerge (notably Tait and Martin-Löf's parallel reduction technique in the 1970s).

Exercises: Beta Reduction

7.1. Reduce $(\lambda x.\, x\ x)\ (\lambda y.\, y)$ to normal form.
7.2. Reduce $(\lambda f.\, \lambda x.\, f\ (f\ x))\ (\lambda y.\, y\ y)$ step by step. Does it terminate?
7.3. How many redexes are in the term $(\lambda x.\, x)\ ((\lambda y.\, y)\ z)$? List them.
7.4. Is $\lambda x.\, (\lambda y.\, y)\ x$ in normal form? If not, reduce it. Can you η-reduce the result?
7.5. Perform the substitution $(\lambda y.\, x\ y)[x := \lambda y.\, y]$ using the capture-avoiding rules. Which rule (1–6) applies at each step?

7.7 Implementing It: Substitution and Evaluation

Now let's implement the substitution and evaluation machinery we just studied.

7.7.1 Substitution: Where It Gets Interesting

Now the hard part. We need $e[x := a]$—the six rules from Section 7. The first two rules (variables) and the last rule (application) are straightforward. The challenge is the lambda cases.

The key challenge

7.6. Try writing `subst (x : String) (s : Term) : Term → Term` yourself. Start with the skeleton—three branches for three constructors. Handle the variable and application cases first (they're direct translations of Rules 1, 2, 6). Then think carefully about the lambda case $(\lambda y.\, e)[x := s]$: there are three sub-cases (Rules 3, 4, 5). What conditions distinguish them?

Before looking at the answer, think through the lambda case. You have $(\lambda y.\, e)[x := s]$ and three questions to ask, *in order*:

1. Is $y = x$? If yes, the λ shadows the variable we're substituting for. Stop. (Rule 3.)
2. Is $y \neq x$ and $y \notin \mathrm{FV}(s)$? If yes, it's safe to go inside. (Rule 4.)
3. Otherwise $y \in \mathrm{FV}(s)$—danger! Rename y to something fresh before going inside. (Rule 5.)

These three conditions map directly to an `if/else if/else` chain. You also need a helper to generate fresh names that don't collide with anything in scope.

Here is the implementation. Read it against the substitution rules—each `if/else` branch corresponds to one rule:

```
partial def freshVar (avoid : List String) (base : String := "x") : String :=
  let rec go (n : Nat) : String :=
    let candidate := s!"{base}{n}"
    if candidate ∈ avoid then go (n + 1) else candidate
  if base ∉ avoid then base else go 0

partial def subst (x : String) (s : Term) : Term → Term
  | .var y     => if y == x then s else .var y          -- Rules 1,2
  | .app e₁ e₂ => .app (subst x s e₁) (subst x s e₂) -- Rule 6
  | .lam y e   =>
    if y == x then .lam y e                            -- Rule 3: shadowed
    else if y ∉ freeVars s then
      .lam y (subst x s e)                             -- Rule 4: safe
    else
      let z := freshVar (freeVars s ++ freeVars e ++ [x])
      .lam z (subst x s (subst y (.var z) e))          -- Rule 5: rename
```

The `freshVar` function generates a name that doesn't collide with anything in scope. This is the "pick a fresh z" step from Rule 5.

Why `partial` here?

Both functions are marked `partial`. Lean normally insists that every `def` terminate, and it proves this automatically only when each recursive call is on a *structural subterm* of an argument. Neither function qualifies: `freshVar.go` recurses on `n + 1` (which *grows*), and `subst`'s Rule 5 recurses on `subst y (.var z) e`—a freshly *built* term, not a piece of the input. Without `partial`, Lean rejects both with "fail to show termination." Marking them `partial` tells Lean "trust me, this halts" and opts out of the termination check. (Both really do halt: names eventually stop colliding, and the rename recursion shrinks the term's λ-nesting even though Lean can't see it.) The alternative—a `termination_by` clause that spells out a decreasing measure—is shown in Appendix A.

Substitution is not "calling the function"

A common early mistake is to test substitution by writing something like `subst "x" (.var "a") K`, where $\mathrm{K} = \lambda x.\, \lambda y.\, x$, and expecting the result to be $\lambda y.\, a$ ("K applied to a"). But the result is K unchanged—because x is *not free* in K. It's bound by K's own outermost λ, so the substitution hits Rule 3 (shadowing) and stops.

This isn't a bug. Substitution is a textual operation: it replaces *free* occurrences of a variable. To actually compute "K applied to a," you need **beta reduction**, which first *strips* K's outer λ and then substitutes into the *body*:

$$(\lambda x.\, \lambda y.\, x)\; a \quad \longrightarrow_\beta \quad (\lambda y.\, x)[x := a] \quad = \quad \lambda y.\, a$$

Beta reduction calls substitution as one of its internal steps, but they are different operations. We'll implement beta reduction next, in the evaluator.

7.8 Evaluation

7.8.1 Single-Step Reduction

You have terms and substitution. Now: how do you actually *evaluate*?

The core idea: scan the term for a redex—an application whose left side is a λ. When you find one, fire the β-rule.

Design challenge

7.7. A `betaStep` function should find *one* redex and reduce it, returning the updated term. If no redex exists, return `none`. Think about: what order should you search for redexes? What if the redex is nested deep inside the term?

Before coding, think about what this function must do by case analysis on the term's shape:

1. **Variable.** No redex here—a variable isn't an application. Return `none`.
2. **Application $e_1\ e_2$.** This is where the action is. Ask: is e_1 a λ? If yes, you found a redex—fire the rule! If no, try to reduce e_1 recursively. If e_1 is already stuck, try e_2.
3. **Lambda $\lambda x.\, e$.** Not a redex itself, but there might be one inside the body e. Try recursing into it.

Notice how the application case has a special check at the top ("is the left side a λ?") before falling through to the recursive cases. This is what makes the evaluation strategy **normal order**—we try the outermost redex first.

Here is normal-order evaluation (leftmost outermost first):

```
def betaStep : Term → Option Term
  | .app (.lam x body) arg => some (subst x arg body)  -- found a redex!
  | .app e₁ e₂ =>
    match betaStep e₁ with                             -- try left side
    | some e₁' => some (.app e₁' e₂)
    | none     => match betaStep e₂ with               -- then right side
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | .lam x e => match betaStep e with                   -- reduce under λ
                | some e' => some (.lam x e')
                | none    => none
  | _ => none                                          -- variable: nothing to do
```

The first branch is the redex case—the β-rule in action. The remaining branches recursively search for a redex deeper inside the term.

7.8.2 Multi-Step with Fuel

One step isn't enough—we want to keep reducing until we reach normal form (or give up). But here's a problem: lambda calculus evaluation might **not terminate** (Ω!). Lean requires all functions to terminate.

Termination challenge

7.8. How do you write a "keep reducing until done" function in a language that demands termination? Think about what parameter you could add that guarantees the loop eventually stops.

The standard trick: a **fuel** parameter that decreases every step.

```
def eval (fuel : Nat) (t : Term) : Term :=
```

```
  match fuel with
  | 0         => t                        -- out of fuel: return as-is
  | fuel' + 1 => match betaStep t with
                 | none    => t           -- normal form reached!
                 | some t' => eval fuel' t'
```

Let's test it:

```
-- (λx. x)(λy. y) →β (λy. y)
#eval eval 100 (.app (.lam "x" (.var "x")) (.lam "y" (.var "y")))
-- Result: Term.lam "y" (Term.var "y")

-- K a b = (λx. λy. x) a b →β a
#eval eval 100
  (.app (.app (.lam "x" (.lam "y" (.var "x"))) (.var "a")) (.var "b"))
-- Result: Term.var "a"
```

Because `Term` derives `Repr`, `#eval` prints the raw constructor tree (`Term.lam "y" (Term.var "y")`), not the pretty λ-notation. Once we add a `ToString` instance (Appendix E), `IO.println (toString (eval 100 t))` prints the friendly form `(λy. y)` instead. Either way the value is the same: $(\lambda y.\, y)$ and a, exactly the reductions we expected.

It works! We have a complete λ-calculus interpreter. But there's a problem lurking beneath the surface…

8 Evaluation Strategies

8.1 The Problem: Which Redex First?

A lambda term can contain more than one redex at the same time. For example, consider the expression $(\lambda x.\, x)\ ((\lambda y.\, y)\ z)$. There are two redexes here:

1. The **outer** redex: the whole expression is $(\lambda x.\, x)$ applied to $(\lambda y.\, y)\ z$.
2. The **inner** redex: inside the argument, $(\lambda y.\, y)\ z$ is itself a function applied to an argument.

Which one do we reduce first? The Church–Rosser theorem guarantees that if we eventually reach a normal form, it will be the same one regardless of order. But the *path* we take can matter enormously—some orders terminate while others don't!

An **evaluation strategy** is a rule for choosing which redex to reduce next. Different strategies correspond to different programming language designs.

8.2 The Main Strategies

Normal order (reduce the leftmost outermost redex first). "Outermost" means we reduce the biggest enclosing redex before diving into its parts. In programming terms, we pass arguments into functions *before* evaluating those arguments. This is the strategy behind **lazy** evaluation.

Applicative order (reduce the leftmost innermost redex first). "Innermost" means we fully evaluate arguments *before* passing them into a function. This is how most familiar languages work: in `f(g(3))`, Python evaluates `g(3)` first, then passes the result to `f`. This is **eager** (or strict) evaluation.

Call-by-need (normal order with sharing). Like normal order, but if an argument is used multiple times, it's evaluated only once and the result is shared. This gives the termination benefits of normal order without redundant computation. This is what Haskell does.

Strategy	Rule	Languages
Normal order	Leftmost outermost first	Lazy evaluation
Applicative order	Leftmost innermost first	JS, Python, C, Java, Lean
Call-by-need	Normal order + memoization	Haskell

8.3 Why It Matters: A Life-or-Death Example

Strategy matters: K I Ω

Recall $\mathbf{K} = \lambda x\, y.\, x$ (takes two arguments, returns the first). Consider $\mathbf{K}\ \mathbf{I}\ \Omega$, where Ω is the non-terminating term from Section 7.
Normal order reduces the outermost redex first—the **K** applied to **I**:

$$(\lambda x\, y.\, x)\ \mathbf{I}\ \Omega \longrightarrow_\beta (\lambda y.\, \mathbf{I})\ \Omega \longrightarrow_\beta \mathbf{I} \quad \checkmark$$

K discards its second argument, so Ω is thrown away *before it's ever evaluated.* Done in two steps.
Applicative order tries to evaluate the arguments first, including Ω:

$$(\lambda x\, y.\, x)\ \mathbf{I}\ \underbrace{\Omega}_{\text{evaluate this first!}} \longrightarrow (\lambda x\, y.\, x)\ \mathbf{I}\ \Omega \longrightarrow \cdots \quad \text{(loops forever!)}$$

It gets stuck trying to evaluate Ω and never finishes. The program *hangs*, even though the answer (**I**) is perfectly well-defined and doesn't depend on Ω at all.

This example demonstrates a fundamental theorem:

Theorem 8.1 (Normalization / Leftmost Reduction)**.** If a term has a normal form, normal-order (leftmost-outermost) reduction will find it. Applicative order may diverge even when a normal form exists.

(This is the *normalization theorem*, sometimes called the leftmost- or standard-reduction theorem. Do not confuse it with the separate *Standardization Theorem*, which says any reduction sequence can be reordered into a standard one.) The normalization theorem was proved by Curry and Feys in 1958, and the operational picture was sharpened by Plotkin in 1975, who formalized the precise relationship between different evaluation strategies and their corresponding programming language semantics. Plotkin's paper "Call-by-name, call-by-value, and the λ-calculus" is one of the most cited papers in programming language theory.

The trade-off

If normal order is "safer" (it always finds a normal form when one exists), why don't all languages use it? The trade-off is **efficiency**. Under applicative order, each argument is evaluated exactly once. Under normal order, if an argument is used three times in the body, it might be evaluated three times. Call-by-need (Haskell's approach) gets the best of both worlds: lazy semantics with no redundant work—but at the cost of implementation complexity.

Normal form vs. diverges-when-applied

It is easy to blur two different questions: "does reducing this term terminate?" and "does this term loop?" Keep them apart.

A term **has a normal form** if repeatedly firing redexes (in normal order) eventually reaches a term with *no* redex left. That is a property of the term *as written*. For example, $\lambda x.\,(x\,x)\,(x\,x)$ is *already* a normal form: its only applications are $x\,x$, and x is a plain variable, not a λ, so there is nothing to reduce. Normalizing it halts immediately.

A term **diverges when applied** if *supplying it an argument* creates an unbounded reduction. The very same $\lambda x.\,(x\,x)\,(x\,x)$ loops forever the moment you apply it to a self-duplicator like $\lambda y.\,y\,y$—that rebuilds Ω. But that divergence lives in the *application*, not in the original (already-normal) function.

So a term can sit in a perfectly good normal form and still build a machine that loops once you feed it input—exactly like a Lean function that is well-defined but would run forever on certain arguments. When an exercise asks you to "normalize the body," answer the first question; do not assume a self-application must diverge.

9 Eta Conversion (η)

We have seen two operations so far: α-conversion (renaming) and β-reduction (function calling). There is one more: η-conversion (pronounced "EH-tah"), which captures a subtler equivalence.

9.1 The Core Idea: Pointless Wrappers

Consider this JavaScript code:

```
const f = x => Math.sqrt(x);
```

This creates a function that takes `x` and passes it to `Math.sqrt`. But `Math.sqrt` is *already* a function that takes one argument! So the wrapper is redundant—you could just write:

```
const f = Math.sqrt;
```

These two definitions behave identically on all inputs. Eta conversion is the formal rule that says these are the same.

The η-conversion Rule

$$\lambda x.\,f\;x \quad\longrightarrow_{\eta}\quad f \qquad \text{provided } x \notin \mathrm{FV}(f)$$

The condition $x \notin \mathrm{FV}(f)$ is essential. If f mentions x freely, then $\lambda x.\,f\;x$ is not just a wrapper around f—the λ actually binds an x that f uses. For example, $\lambda x.\,(x{+}1)\;x$ cannot be η-reduced because the body uses x in two places.

When to use η-conversion

η-conversion expresses **extensionality**: if two functions produce the same output for every input, they are the same function. Most functional programmers encounter this as the "remove pointless wrappers" refactoring: `x => f(x)` simplifies to `f`. In Haskell, this is called **eta reduction** and is a common style recommendation (`map (\x -> f x) xs` simplifies to `map f xs`).

10 Church Encodings

10.1 The Big Idea: Everything Is a Function

Lambda calculus has no built-in data types. No booleans, no numbers, no strings, no lists. There is *nothing* except variables, function definitions, and function calls. So how could this system possibly be useful for computation?

The answer is Church's remarkable insight: you don't *need* built-in data. You can *encode* any data type as a pure function, by representing each value as a function that describes "what you do with it." A boolean is a function that *chooses*. A number is a function that *repeats*. A pair is a function that *holds two things and gives you access to them*.

These function-based representations are called **Church encodings**, and they are one of the most beautiful ideas in computer science. They show that data and behavior are two sides of the same coin.

Church developed these encodings out of necessity. To prove that his λ-calculus could serve as a foundation for mathematics—and specifically to show it could express arithmetic, which was needed for his attack on Hilbert's Entscheidungsproblem—he had to demonstrate that numbers, logical operations, and recursive definitions could all be expressed using nothing but functions. The encodings were his proof that the system was powerful enough. His student Stephen Kleene then extended the work, showing how to encode more complex operations (including the predecessor function, as we'll see).

In the definitions below, the symbol $\triangleq$ means "is defined as" (see Section 2). We use it instead of $=$ to emphasize that we are *introducing a name* for a term, not stating an equation we discovered.

10.2 Church Booleans

What is a boolean? At its core, it's something that lets you choose between two alternatives. "If true, do this; if false, do that." A Church boolean *is* that choice, expressed directly as a function.

Church Booleans

$$\textsc{True} \triangleq \lambda t\, f.\, t \quad \text{(select first)} \qquad \textsc{False} \triangleq \lambda t\, f.\, f \quad \text{(select second)}$$

Both TRUE and FALSE are functions that take two arguments, t and f (short for "true branch" and "false branch"). TRUE returns the first one; FALSE returns the second one.

Data as behavior

In most languages, `if` is a language construct and booleans are primitive values. In Church encoding, there is no `if`—the boolean itself *is* the conditional. When you "apply a boolean to two choices," it picks one. There is no need for a separate `if` statement because the boolean already knows how to choose.
This is the fundamental insight of Church encodings: **data is represented by what you do with it**, not by some internal structure.

Using these booleans, we can define the standard logical operations. Each one works by cleverly applying a boolean to the right pair of arguments:

$$\textsc{And} \triangleq \lambda p\, q.\ p\ q\ \textsc{False} \qquad \textsc{Or} \triangleq \lambda p\, q.\ p\ \textsc{True}\ q$$
$$\textsc{Not} \triangleq \lambda p.\ p\ \textsc{False}\ \textsc{True} \qquad \textsc{If} \triangleq \lambda p\, a\, b.\ p\ a\ b$$

How do these work? Take And: it applies p to two arguments, q and False. If p is True, it selects the first argument (q), so the result depends on q—exactly right for "and." If p is False, it selects the second argument (False), giving false immediately—also correct.

Not swaps the two arguments: it gives a boolean False as its "true branch" and True as its "false branch," so True becomes False and vice versa.

If is the most elegant—it's literally just function application! Applying a Church boolean to two choices *is* an if-then-else.

And True False = False

Let's verify step by step that "true and false = false":

$$\begin{aligned}
\textsc{And}\ \textsc{True}\ \textsc{False} &= (\lambda p\, q.\ p\ q\ \textsc{False})\ \textsc{True}\ \textsc{False} && \text{(expand the definition of And)}\\
&\longrightarrow_\beta (\lambda q.\ \textsc{True}\ q\ \textsc{False})\ \textsc{False} && \text{(substitute True for } p)\\
&\longrightarrow_\beta \textsc{True}\ \textsc{False}\ \textsc{False} && \text{(substitute False for } q)\\
&= (\lambda t\, f.\ t)\ \textsc{False}\ \textsc{False} && \text{(expand the definition of True)}\\
&\longrightarrow_\beta (\lambda f.\ \textsc{False})\ \textsc{False} && \text{(True selects its first argument: False)}\\
&\longrightarrow_\beta \textsc{False} \quad \checkmark && \text{(the remaining False argument is discarded)}
\end{aligned}$$

10.3 Church Numerals

How do you represent numbers as functions? Church's idea: a number n is a function that *applies another function n times*. The number 0 applies f zero times (just returns the starting value). The number 1 applies f once. The number 3 applies f three times. The number *is* the repetition count.

Church Numerals

$\overline{n}$ applies its first argument n times to its second:

$$\overline{0} = \lambda f\, x.\ x \qquad \overline{1} = \lambda f\, x.\ f\ x \qquad \overline{n} = \lambda f\, x.\ \underbrace{f\ (f\ (\cdots (f}_{n}\ x) \cdots))$$

Reading Church numeral notation

The overline $\overline{n}$ is shorthand for "the Church encoding of number n." The notation $f^n\ x$ means "apply f a total of n times to x": $f^0\ x = x$, $f^1\ x = f\ x$, $f^2\ x = f\ (f\ x)$, etc. So $\overline{n}$ is a function that takes f and x and applies f exactly n times. The number *is* the iteration.

Writing out the first few concretely:

$$\begin{aligned}
\overline{0} &= \lambda f\, x.\ x && \text{(apply } f \text{ zero times: just return } x)\\
\overline{1} &= \lambda f\, x.\ f\ x && \text{(apply } f \text{ once)}\\
\overline{2} &= \lambda f\, x.\ f\ (f\ x) && \text{(apply } f \text{ twice)}\\
\overline{3} &= \lambda f\, x.\ f\ (f\ (f\ x)) && \text{(apply } f \text{ three times)}
\end{aligned}$$

Notice something: $\overline{0} = \lambda f\, x.\, x$ has the same structure as FALSE (select the second argument). This is not a coincidence—zero and "false" are deeply connected in Church encoding.

Now we can define arithmetic. Each operation works by manipulating how many times f gets applied:

$$\text{SUCC} \triangleq \lambda n\, f\, x.\, f\ (n\ f\ x) \qquad \text{ADD} \triangleq \lambda m\, n\, f\, x.\, m\ f\ (n\ f\ x)$$
$$\text{MUL} \triangleq \lambda m\, n\, f.\, m\ (n\ f) \qquad \text{EXP} \triangleq \lambda m\, n.\, n\ m$$

How does SUCC (successor, i.e., "add 1") work? It takes a Church numeral n and returns a new function that applies f one extra time: first do $n\ f\ x$ (apply f a total of n times), then wrap one more f around the result.

ADD works similarly: apply f a total of n times (via $n\ f\ x$), then apply f another m times on top (via $m\ f\ (\ldots)$). The total is $m + n$ applications of f.

MUL is more subtle: $n\ f$ is a function that applies f exactly n times. Then $m\ (n\ f)$ applies *that combined function* m times. n applications repeated m times gives $m \times n$ applications total.

Succ $\overline{2} = \overline{3}$

$$\begin{aligned}
\text{SUCC}\ \overline{2} &= (\lambda n\, f\, x.\, f\ (n\ f\ x))\ \overline{2} && \text{(expand SUCC)}\\
&\longrightarrow_\beta \lambda f\, x.\, f\ \left(\overline{2}\ f\ x\right) && \text{(plug in } \overline{2} \text{ for } n)\\
&\twoheadrightarrow_\beta \lambda f\, x.\, f\ (f\ (f\ x)) && (\overline{2}\ f\ x = f\ (f\ x)\text{; wrap one more } f)\\
&= \overline{3} \quad \checkmark
\end{aligned}$$

10.4 Church Pairs

A pair holds two values and lets you extract either one. In Church encoding, a pair is a function that "closes over" its two elements and waits for you to tell it which one you want—by passing it either TRUE (to get the first) or FALSE (to get the second).

$$\text{PAIR} \triangleq \lambda a\, b\, f.\, f\ a\ b \qquad \text{FST} \triangleq \lambda p.\, p\ \text{TRUE} \qquad \text{SND} \triangleq \lambda p.\, p\ \text{FALSE}$$

PAIR $a\ b$ creates a function $\lambda f.\, f\ a\ b$ that remembers a and b in its closure. To extract the first element, FST passes TRUE (which selects the first of two arguments) as the extraction function. To extract the second, SND passes FALSE.

Fst (Pair $x\ y$) $= x$

$$\begin{aligned}
\text{FST}\ (\text{PAIR}\ x\ y) &\twoheadrightarrow_\beta (\lambda p.\, p\ \text{TRUE})\ (\lambda f.\, f\ x\ y) && (\text{PAIR}\ x\ y \text{ becomes } \lambda f.\, f\ x\ y)\\
&\longrightarrow_\beta (\lambda f.\, f\ x\ y)\ \text{TRUE} && \text{(pass the pair its extractor (TRUE))}\\
&\longrightarrow_\beta \text{TRUE}\ x\ y && \text{(the pair hands both elements to TRUE)}\\
&\twoheadrightarrow_\beta x \quad \checkmark && (\text{TRUE selects the first: } x)
\end{aligned}$$

10.5 IsZero and Predecessor

With booleans and numerals defined, we need a way to *test* whether a number is zero. This turns out to be surprisingly elegant:

$$\textsc{IsZero} \triangleq \lambda n.\, n\ (\lambda\_.\ \textsc{False})\ \textsc{True}$$

How does it work? Recall that a Church numeral $\overline{n}$ applies its first argument n times to its second. If $n = 0$, it applies the function zero times, just returning True. If $n \geq 1$, it applies $\lambda\_.\ \textsc{False}$ at least once—and that function ignores its argument and returns False regardless.

IsZero $\overline{0}$ = True and IsZero $\overline{2}$ = False

$\textsc{IsZero}\ \overline{0} = (\lambda n.\, n\ (\lambda\_.\ \textsc{False})\ \textsc{True})\ (\lambda f\, x.\, x) \longrightarrow_\beta (\lambda f\, x.\, x)\ (\lambda\_.\ \textsc{False})\ \textsc{True} \twoheadrightarrow_\beta \textsc{True}$ ✓

$\textsc{IsZero}\ \overline{2} \twoheadrightarrow_\beta (\lambda\_.\ \textsc{False})\ \big((\lambda\_.\ \textsc{False})\ \textsc{True}\big) \twoheadrightarrow_\beta \textsc{False}$ ✓

Predecessor is famously the hardest basic Church encoding. Successor is easy—just wrap one more f around the result. But *removing* one layer of f is not straightforward, because a Church numeral only knows how to apply f forwards; there is no built-in way to "undo" an application.

The solution was discovered by Stephen Kleene in 1932, when he was a PhD student of Church's at Princeton. Legend has it that the insight came to him while sitting in the dentist's chair. The problem had stumped Church's group for months—without predecessor, they couldn't define subtraction, comparison, or any of the recursive functions needed to show that λ-calculus could express all of arithmetic. Kleene's breakthrough was the "pair-walking trick": iterate a pair-updating function n times, starting from a carefully chosen initial pair, so that after n steps the desired value sits in the first component.

$$\textsc{Pred} \triangleq \lambda n.\, \textsc{Fst}\ \big(n\ \Phi\ (\textsc{Pair}\ \overline{0}\ \overline{0})\big) \qquad \text{where} \quad \Phi \triangleq \lambda p.\, \textsc{Pair}\ (\textsc{Snd}\ p)\ (\textsc{Succ}\ (\textsc{Snd}\ p))$$

The helper Φ takes a pair (a, b) and produces (old b, $b+1$). Starting from $(0, 0)$:

Step	Pair before	Φ	Pair after
0	—	—	$(0, 0)$
1	$(0, 0)$	$(\text{snd}, \text{succ snd})$	$(0, 1)$
2	$(0, 1)$		$(1, 2)$
3	$(1, 2)$		$(2, 3)$
n	$(n{-}2,\ n{-}1)$		$(n{-}1,\ n)$

After n applications of Φ, the first component is $n - 1$. Taking Fst of the final pair gives the predecessor. (For $n = 0$, the pair never changes from $(0, 0)$, so $\textsc{Pred}\ \overline{0} = \overline{0}$.)

Why Pred is hard

Successor adds a layer of f; predecessor must remove one. But Church numerals are "write-only"—they apply f a fixed number of times and give you no way to "un-apply" it. Kleene's trick works around this by never removing anything: instead, it rebuilds the count from scratch using a pair that always trails one step behind. This is why predecessor is $O(n)$ even though the number n is "already there" in the encoding.

10.6 Church-Encoded Lists

Lists round out the data structures story. A Church-encoded list is a function that takes two arguments—a "cons handler" and a "nil value"—and folds itself. This is exactly the same pattern as `foldr` in Haskell or `reduce` in JavaScript.

Church Lists

$$\textsc{Nil} \triangleq \lambda c\, n.\, n$$
$$\textsc{Cons} \triangleq \lambda h\, t\, c\, n.\, c\; h\; (t\; c\; n)$$

Nil is a list that ignores the cons handler and returns the nil value directly—like an empty list that has nothing to fold over. Cons h t is a list with head h and tail t: when folded, it applies the cons handler c to h and the result of folding the tail.

Lists are their own fold

In Haskell, `foldr f z [1,2,3]` computes `f 1 (f 2 (f 3 z))`. A Church list *is* this computation. The list Cons 1 (Cons 2 (Cons 3 Nil)) is a function that, when given c and n, computes $c\ 1\ (c\ 2\ (c\ 3\ n))$. The list doesn't store data passively and wait for you to traverse it—it *is* the traversal.

With this encoding, list operations become simple:

$$\textsc{Head} \triangleq \lambda l.\, l\ (\lambda h\ \_.\, h)\ \textsc{False} \qquad \text{(fold with "take first")}$$
$$\textsc{IsNil} \triangleq \lambda l.\, l\ (\lambda \_\ \_.\, \textsc{False})\ \textsc{True} \qquad \text{(any cons returns False)}$$
$$\textsc{Map} \triangleq \lambda f\, l\, c\, n.\, l\ (\lambda h\, t.\, c\ (f\ h)\ t)\ n \qquad \text{(apply } f \text{ in the cons handler)}$$
$$\textsc{Append} \triangleq \lambda l_1\, l_2\, c\, n.\, l_1\ c\ (l_2\ c\ n) \qquad \text{(fold } l_1 \text{ then } l_2\text{)}$$

Head folds the list with a handler that returns the first element it sees (ignoring the rest). IsNil works like IsZero: if no cons is ever applied, return True; otherwise False. Append is particularly elegant: fold the first list with c, but instead of stopping at n, continue folding into the second list.

Extracting the head of a list

Let $L = \textsc{Cons}\ a\ (\textsc{Cons}\ b\ \textsc{Nil})$.

$$\begin{aligned}\textsc{Head}\ L &= L\ (\lambda h\ \_.\, h)\ \textsc{False} \\ &\twoheadrightarrow_\beta (\lambda h\ \_.\, h)\ a\ \big((\lambda h\ \_.\, h)\ b\ \textsc{False}\big) \longrightarrow_\beta a \quad \checkmark\end{aligned}$$

Exercises: Church Encodings

10.1. Reduce IsZero (Succ $\overline{0}$) step by step. What is the result?
10.2. Verify that Add $\overline{2}\ \overline{3} = \overline{5}$ by reducing it to normal form.
10.3. Define Sub (subtraction) using Pred: Sub $\overline{m}\ \overline{n}$ should apply Pred a total of n times to $\overline{m}$.
10.4. Define Length for Church lists. *Hint:* fold with a handler that ignores the element and increments a counter.
10.5. Is Nil the same term as $\overline{0}$? Explain why or why not, and what this says about Church encodings.

In most programming languages, writing a recursive function is straightforward: you give the function a name and call that name inside its own body.

```
function factorial(n) {
    if (n === 0) return 1;
    return n * factorial(n - 1);  // calls itself by name
}
```

But lambda calculus has *no names*. There is no way to say "define a function called `factorial` and refer to it from within itself." Every function is anonymous—it's just $\lambda x.\, body$, with no way for the body to refer back to the function that contains it.

This seems like a fatal limitation. Without recursion, how can we express loops? How can we compute factorials, traverse lists, or do anything that requires repeating a computation?

10.7 The Solution: Fixed-Point Combinators

The trick is both ingenious and strange. Instead of a function calling itself by name, we use a helper called a **fixed-point combinator** that "feeds a function to itself" automatically.

The key insight: suppose you write your recursive function in a special way. Instead of calling yourself directly, you take an *extra parameter* called `self` and call *that* whenever you would have made a recursive call:

```
// Instead of:  function fact(n) { ... fact(n-1) ... }
// Write:       self => n => ... self(n-1) ...
const F = self => n => (n === 0) ? 1 : n * self(n - 1);
```

Now F is not recursive—it doesn't call itself. But if we could somehow pass F a version of itself that "works," we'd have recursion. A fixed-point combinator does exactly this: given F, it produces a value v such that $v = F(v)$. That v is the recursive function we wanted.

Fixed-Point Property

Y is a **fixed-point combinator** if, for every function f:

$$\mathbf{Y}\ f \;=\; f\ (\mathbf{Y}\ f)$$

In other words, $\mathbf{Y}\ f$ is a **fixed point** of f: a value that, when you apply f to it, gives you back the same value.

Why "fixed point"?

In mathematics, a fixed point of a function f is a value v where $f(v) = v$. For example, 0 is a fixed point of the function $f(x) = x^2$ because $f(0) = 0$. The **Y** combinator finds a fixed point of *any* function—even functions on lambda terms. When f is our "factorial template" F from above, the fixed point $\mathbf{Y}\ F$ is the actual factorial function: F applied to it gives back the same function.

10.8 The Y Combinator

The **Y** combinator was discovered by Haskell Curry in the 1940s, though Church's group at Princeton had studied fixed-point constructions earlier. Curry showed that *every* function in the λ-calculus has a fixed point—a consequence of the system's ability to express self-application. This was a double-edged sword: it made recursion possible, but it also meant Church's original

goal of using λ-calculus as a consistent foundation for *logic* was doomed (Kleene and Rosser showed in 1935 that the untyped λ-calculus is logically inconsistent). The λ-calculus survived by pivoting: it became a theory of *computation* rather than logic.

Here is the actual definition:

$$\mathbf{Y} = \lambda f.\,(\lambda x.\,f\ (x\ x))\ (\lambda x.\,f\ (x\ x)) \qquad \text{(normal order only)}$$

This looks intimidating, but it's built from a familiar pattern. Remember $\Omega = (\lambda x.\,x\ x)\ (\lambda x.\,x\ x)$, which loops forever because it applies itself to itself? The $\mathbf{Y}$ combinator uses the same self-application trick, but inserts f so the self-application does something useful (calling f each time) instead of just spinning in circles.

$\mathbf{Y}\ g = g\ (\mathbf{Y}\ g)$

Let's verify the fixed-point property by reducing $\mathbf{Y}\ g$ step by step:

$$\begin{aligned}
\mathbf{Y}\ g &= (\lambda f.\,(\lambda x.\,f\ (x\ x))\ (\lambda x.\,f\ (x\ x)))\ g \quad \text{(expand } \mathbf{Y}\text{)} \\
&\longrightarrow_\beta (\lambda x.\,g\ (x\ x))\ (\lambda x.\,g\ (x\ x)) \quad \text{(substitute } g \text{ for } f\text{)} \\
&\longrightarrow_\beta g\ \underbrace{\big((\lambda x.\,g\ (x\ x))\ (\lambda x.\,g\ (x\ x))\big)}_{\text{this is } \mathbf{Y}\ g \text{ again!}} \\
&= g\ (\mathbf{Y}\ g) \quad \checkmark
\end{aligned}$$

The key moment is the last step: after two β-reductions, we get g applied to an expression that is identical to $\mathbf{Y}\ g$. So $\mathbf{Y}\ g$ "unrolls" into $g\ (\mathbf{Y}\ g)$, which unrolls into $g\ (g\ (\mathbf{Y}\ g))$, and so on—as many times as needed.

10.9 The Z Combinator (Call-by-Value)

The $\mathbf{Y}$ combinator only works under normal-order (lazy) evaluation. Under applicative-order (eager) evaluation—which is what most languages use—$\mathbf{Y}\ f$ diverges because the self-application $x\ x$ is evaluated immediately, before f can "decide" to stop.

The fix is the **Z combinator**, which wraps the self-application in a λ to delay it:

$$\mathbf{Z} = \lambda f.\,(\lambda x.\,f\ (\lambda y.\,x\ x\ y))\ (\lambda x.\,f\ (\lambda y.\,x\ x\ y))$$

The extra $\lambda y.\ \ldots\ y$ acts as a "thunk"—it prevents $x\ x$ from being evaluated until the result is actually needed (i.e., until the function is called with an argument y). This is the combinator used in eager languages like JavaScript, Python, and our call-by-value Lean interpreter.

10.10 Factorial: Putting It All Together

Factorial via Y

Define the factorial "template" (a non-recursive function that takes a *self* parameter):

$$F = \lambda \mathit{self}.\,\lambda n.\,\textsc{If}\ (\textsc{IsZero}\ n)\ \overline{1}\ (\textsc{Mul}\ n\ (\mathit{self}\ (\textsc{Pred}\ n)))$$

Then $\textsc{Fact} = \mathbf{Y}\ F$ is the actual factorial function. Tracing $\textsc{Fact}\ \overline{3}$:

$$\begin{aligned}
\textsc{Fact}\ \overline{3} &= (\mathbf{Y}\ F)\ \overline{3} = F\ (\mathbf{Y}\ F)\ \overline{3} \quad \text{(fixed-point property: } \mathbf{Y}\ F = F\ (\mathbf{Y}\ F)\text{)} \\
&\twoheadrightarrow_\beta \textsc{Mul}\ \overline{3}\ (\textsc{Fact}\ \overline{2}) \quad (\overline{3} \text{ is not zero, so take the else branch}) \\
&\twoheadrightarrow_\beta \textsc{Mul}\ \overline{3}\ (\textsc{Mul}\ \overline{2}\ (\textsc{Fact}\ \overline{1})) \quad \text{(unroll one more time)} \\
&\twoheadrightarrow_\beta \textsc{Mul}\ \overline{3}\ (\textsc{Mul}\ \overline{2}\ (\textsc{Mul}\ \overline{1}\ \overline{1})) \quad (\overline{0} \text{ gives } \overline{1}) \\
&\twoheadrightarrow_\beta \overline{6} \quad \checkmark
\end{aligned}$$

Each recursive call "unrolls" the $\mathbf{Y}$ combinator one more time, feeding $\textsc{Fact}$ back into F as the *self* argument. The recursion terminates when n reaches $\overline{0}$ and $\textsc{IsZero}$ selects the base case ($\overline{1}$).

In typed calculi

$\mathbf{Y}$ cannot be typed in the simply typed λ-calculus. The problem is the self-application $x\ x$: for this to type-check, x would need to simultaneously have type $T \to T$ (because it's being used as a function) and type T (because it's the argument to itself). No finite type can satisfy both requirements. This is why typed languages like Lean, Haskell, and OCaml provide recursion through dedicated language features (`fix`, `partial`, or structural recursion) rather than through combinators.

Part II

Going Deeper

11 De Bruijn Indices

Named variables require constant α-conversion. In 1972, the Dutch mathematician Nicolaas Govert de Bruijn proposed an elegant solution while developing the **Automath** system—one of the earliest computerized proof checkers. De Bruijn's problem was practical: when a computer manipulates λ-terms, it must constantly rename variables to avoid capture, which is both error-prone and expensive. His insight was to eliminate names entirely: a variable becomes a natural number counting binders outward.

De Bruijn Syntax

$$e ::= n \mid \lambda.\, e \mid e_1\ e_2 \qquad (n \in \mathbb{N})$$

Variable n refers to the n-th enclosing λ (0-indexed).

Named	De Bruijn
$\lambda x.\, x$	$\lambda.\, 0$
$\lambda x.\, \lambda y.\, x$	$\lambda.\, \lambda.\, 1$
$\lambda x.\, \lambda y.\, y$	$\lambda.\, \lambda.\, 0$
$\lambda x.\, \lambda y.\, x\ y$	$\lambda.\, \lambda.\, 1\ 0$
$\lambda f\, g\, x.\, f\ x\ (g\ x)$	$\lambda.\, \lambda.\, \lambda.\, 2\ 0\ (1\ 0)$

Alpha equivalence becomes **structural equality**—no renaming needed.

The most common misconception. Your first instinct with the K combinator $\lambda x.\, \lambda y.\, x$ might be to assign $x = 0$ and $y = 1$ (treating indices like serial numbers). This gives $\lambda.\, \lambda.\, 0$—which is **wrong**. That's $\lambda x.\, \lambda y.\, y$, the function that returns its second argument!

De Bruijn indices are *not names*. They count distance. Each variable looks *upward* from where it sits and counts how many λ's it must skip to reach its binder:

Wrong: indices are names

λ *"x is the 0th parameter"*

λ *"y is the 1st parameter"*

points to *y*!

`0` *"x was assigned 0"*

$\lambda.\, \lambda.\, 0 = \lambda x.\, \lambda y.\, y$ ✗

Right: indices count distance

skip 1

λ *binder for x*

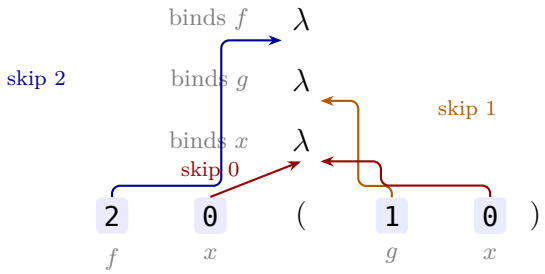
λ *binder for y* *(skip this: +1)*

`1` *"my binder is 1 λ up"*

$\lambda.\, \lambda.\, 1 = \lambda x.\, \lambda y.\, x$ ✓

Here is the rule: stand at the variable, look upward, count λ's starting from 0. The *nearest* λ is 0. The next one up is 1. The variable's index is the number of the λ that *binds* it.

S combinator: $\lambda f.\, \lambda g.\, \lambda x.\, f\ x\ (g\ x)$



$= \lambda.\, \lambda.\, \lambda.\, 2\ \ 0\ \ (1\ \ 0)$

The elevator rule

Imagine each λ is a floor in a building, numbered from the bottom up. A variable sits on the ground floor and needs to ride the elevator to its binder's floor. The de Bruijn index is *how many floors it rides*. The nearest λ (floor 0) costs 0. The next one up costs 1. And so on. The index tells you the distance, not the identity.

11.1 Shifting and Substitution

Shifting: $\uparrow_c^d (e)$

Add d to all variables $\geq c$:

$$\uparrow_c^d (n) = \begin{cases} n + d & \text{if } n \geq c \\ n & \text{if } n < c \end{cases}$$

$$\uparrow_c^d (\lambda.\, e) = \lambda.\ \uparrow_{c+1}^d (e)$$

$$\uparrow_c^d (e_1\ e_2) = \uparrow_c^d (e_1)\ \ \uparrow_c^d (e_2)$$

Reading the shift notation

$\uparrow_c^d (e)$ is a function of three things: the **shift amount** d (an integer—can be negative), the **cutoff** c (a natural number), and the **term** e. The curly-brace $\begin{cases}\cdots\end{cases}$ notation is a piecewise definition (an if/else): if the variable index n is $\geq c$, add d; otherwise leave it alone. The cutoff c starts at 0 and increases by 1 each time we go under a λ, which is how we track "how many binders are above us" and only adjust *free* variables.

Why shifting? When we substitute a term s into a body that has a λ removed, the indices of s's free variables may be off by one. Shifting corrects them. Think of it as updating line numbers after inserting or deleting a line of code.

De Bruijn Substitution

$$n[k := s] = \begin{cases} s & \text{if } n = k \\ n & \text{otherwise} \end{cases}$$
$$(\lambda.\, e)[k := s] = \lambda.\, \left(e[k{+}1 := \uparrow_0^1 (s)]\right)$$
$$(e_1\ e_2)[k := s] = (e_1[k := s])\ (e_2[k := s])$$

Reading the De Bruijn substitution

$e[k := s]$ means "in term e, replace variable index k with term s."

1. If e is a variable n: replace it with s when $n = k$, otherwise leave it. (The $\begin{cases}\cdots\end{cases}$ is an if/else.)
2. If e is $\lambda.\, body$: going under a binder means bumping k to $k{+}1$ (the target variable is one level deeper now) *and* shifting s up by 1 ($\uparrow_0^1 (s)$) so that s's free variables still point to the right binders from inside the extra λ.
3. Applications: substitute into both sides independently.

β-reduction with De Bruijn Indices

$$\boxed{(\lambda.\, e)\ s \ \longrightarrow_\beta\ \uparrow_0^{-1} \left(e[0 := \uparrow_0^1 (s)]\right)}$$

Shift s up by 1, substitute for index 0, shift result down by 1. **No α-conversion needed.**

Unpacking the formula step by step

Reading the formula right-to-left, inside-out:

1. $\uparrow_0^1 (s)$: Shift the argument s up by 1. This adjusts s's free variables because s is about to go *under* the λ binder in the body e.
2. $e[0 := \uparrow_0^1 (s)]$: In the body e, replace every occurrence of variable index 0 (the parameter bound by the λ we're eliminating) with the shifted s.
3. $\uparrow_0^{-1} (\ldots)$: Shift the whole result *down* by 1. The λ binder has been removed, so all remaining free variable indices need to decrease by 1 to compensate.

De Bruijn: $(\lambda.\,\lambda.\,1)\ (\lambda.\,0)$ (i.e., K I)

Body $e = \lambda.\,1$, arg $s = \lambda.\,0$.

$$\uparrow_0^1 (\lambda.\,0) = \lambda.\ \uparrow_1^1 (0) = \lambda.\,0 \quad (0 < 1, \text{ no change})$$
$$(\lambda.\,1)[0 := \lambda.\,0] = \lambda.\,\big(1[1 := \uparrow_0^1 (\lambda.\,0)]\big) = \lambda.\,\big(1[1 := \lambda.\,0]\big) = \lambda.\,(\lambda.\,0) \quad (1 = 1, \text{ replace})$$
$$\uparrow_0^{-1} (\lambda.\,\lambda.\,0) = \lambda.\,\lambda.\,0 \quad (\text{no free vars to shift})$$

Result: $\lambda.\,\lambda.\,0$, which in named form is $\lambda a.\,\lambda b.\,b$. But wait—**K I** should give $\lambda y.\,\mathbf{I} = \lambda y.\,\lambda z.\,z = \lambda.\,\lambda.\,0$. The inner 0 refers to the z binder. So this **is correct**! ✓

Moral: De Bruijn arithmetic is fiddly by hand—let the computer handle it.

11.2 Implementing It: De Bruijn in Lean

11.2.1 When Names Fight Back

Try reducing $(\lambda x.\,\lambda y.\,x)\ y$ in your head. Step 1: substitute y for x in $\lambda y.\,x$. But wait—y is free in the argument, and λy would capture it! Rule 5 kicks in: rename the bound y to z first, giving $\lambda z.\,y$.

This works, but it's *annoying*. Every substitution requires checking whether capture might happen, generating fresh names, and renaming. Worse, the "same" term can look different depending on which names were chosen during renaming. $\lambda x.\,x$ and $\lambda y.\,y$ and $\lambda z.\,z$ are all the identity function, but they're different strings of symbols. If you want to check whether two terms are equal, you need α-equivalence, which itself requires comparing structure modulo renaming.

Design challenge

11.1. Imagine you're designing a system where α-equivalent terms are *literally identical*—not just "the same up to renaming," but the same data structure with the same bits. What would you use instead of names to refer to variables?
Hint: What stays the same about $\lambda x.\,x$ and $\lambda y.\,y$? It's not the name. What is it?

11.2.2 The Insight: Count, Don't Name

Here's what $\lambda x.\,x$ and $\lambda y.\,y$ have in common: the variable in the body refers to the *nearest enclosing binder*. Not "the binder named x" or "the binder named y"—just "the binder that's one λ up."

If we replace names with numbers that count binders outward, both become the same thing: $\lambda.\,0$ ("the variable bound by the nearest λ, zero layers up"). This is exactly the de Bruijn representation from Section 11.

```
inductive Expr where
  | bvar : Nat → Expr        -- bound variable: de Bruijn index
  | lam  : Expr → Expr       -- lambda: no name needed!
  | app  : Expr → Expr → Expr
  deriving Repr, DecidableEq, Inhabited
```

Notice: `lam` has *no* `String` parameter. The binder doesn't need a name—variables find their binder by counting.

11.2.3 The New Challenge: Shifting

With names, substitution needed renaming to avoid capture. With de Bruijn indices, renaming is gone—but a new challenge appears. When you move a term under or out from a binder, the indices must be adjusted.

Shifting challenge

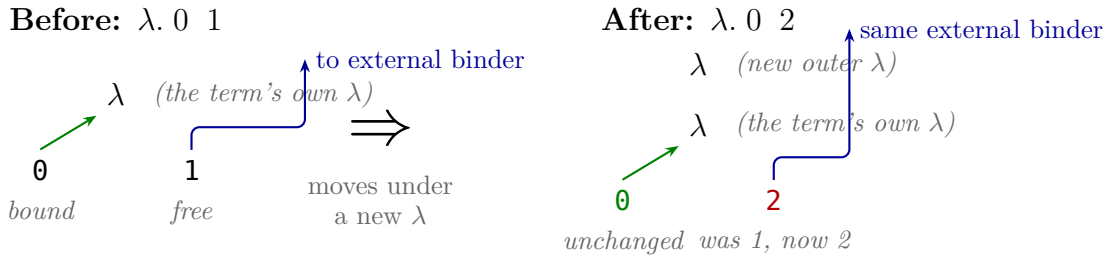
11.2. Consider the term $\lambda.\, 0\ 1$ (which in named form is $\lambda x.\, x\ y$ for some free y). If we substitute this into a context that has one more binder above it, the free variable 1 (referring to y) needs to increase by 1 to still point to the same thing. But 0 (the bound variable) must *not* change. How do you decide which indices to adjust?

Think about this carefully. The index 0 points to the λ right above it— it's a **bound** variable. The index 1 points past the λ to something external—it's **free**. When we move the term under another binder, all the free indices shift up by 1 (they now have one more binder to "skip over"), but the bound indices stay the same (they still point to the same λ they always did).

So the question becomes: how do you distinguish free indices from bound ones?

The answer: track how many binders you've descended through. Call this the **cutoff** c. Start at $c = 0$. Each time you enter a λ, increment c. An index n is free (and needs adjusting) if $n \geq c$, and bound (and stays fixed) if $n < c$.

Why does this work? If you're inside c nested binders, then indices $0, 1, \ldots, c{-}1$ all point to one of those binders (they're bound). Any index $\geq c$ points past all of them (it's free).



The bound variable 0 still points to its own λ—nothing changed. But the free variable had to increase from 1 to 2: it now needs to skip *two* λ's to reach the same external binder it always referred to.

```
def shift (d : Int) (c : Nat) : Expr → Expr
  | .bvar n    => if n ≥ c then .bvar (Int.toNat (n + d)) else .bvar n
  | .lam e     => .lam (shift d (c + 1) e)     -- going under a binder: c increases
  | .app e₁ e₂ => .app (shift d c e₁) (shift d c e₂)
```

With shifting in hand, we're ready for substitution. But this is the hardest part of the de Bruijn implementation—it took real researchers (de Bruijn himself, then many others refining the details) years to get this right. The difficulty is that *two* adjustments are needed, and it's hard to see why until you try the wrong thing and watch it break.

But before we write any code, there is a conceptual trap that catches almost everyone: confusing **substitution** with **beta reduction**. Getting this distinction clear first will save you from a lot of confusion later.

Substitution is not beta reduction. It's natural to think of substitution as "plugging an argument into a function." But that's beta reduction, which is a *computation step*. Substitution

is a lower-level operation that beta reduction *uses*.

Beta reduction A computation step: strip the outer λ, then substitute the argument into the body, adjusting indices for the removed binder. This is what happens when you "call a function."

Substitution A textual operation: find every free occurrence of a variable and replace it with a term. It knows nothing about function calls—it just does search-and-replace.

Beta reduction *calls* substitution as one of its internal steps, but they are different things. Consider the K combinator, $\lambda x.\, \lambda y.\, x$. If you try to substitute directly into the whole term:

```
-- WRONG: trying to use subst as if it were beta reduction
-- This does NOT compute "K a"!
def K    : Term := .lam "x" (.lam "y" (.var "x"))   -- named K = λx.λy. x
def K_db : Expr := .lam (.lam (.bvar 1))            -- de Bruijn K = λ.λ. 1
#eval subst "x" (.var "a") K    -- named version    (3-arg named subst)
#eval subst 0 (.bvar 99) K_db  -- de Bruijn version (2-arg de Bruijn subst)
```

Both produce the *original term, unchanged*. Why? Because x (or index 0) is not free in K—it's bound by K's own outermost λ. Substitution only replaces *free* occurrences, and there are none.

To actually compute "K applied to `a`," you need beta reduction, which first **strips the outer λ** and then substitutes into the **body**:

```
Beta reduction of K a:
   (λx. λy. x) a              -- the full application
   = (λy. x)[x := a]          -- strip outer λ, substitute into BODY
   = λy. a                     -- x was free in the body, so it got replaced

De Bruijn version:
   (λ. λ. 1) a
   = (λ. 1)[0 := a]           -- strip outer λ, substitute into BODY
                                -- 0 means "the variable the stripped λ bound"
                                -- from the body's top level, that's index 0
   = ...                        -- (we'll work out the details below)
```

The key insight: the `subst 0` in beta reduction targets index 0 *relative to the body*, not relative to the original term. The outer λ is already gone. From the body's perspective, the variable it used to bind is the nearest external thing—index 0.

Substitution is a tool, not a computation

Think of substitution as a screwdriver. Beta reduction is the act of assembling furniture. You don't hand someone a screwdriver and say "build me a bookshelf" —the screwdriver is one tool used during assembly. Similarly, you don't call `subst` on a whole λ-term and expect it to "run" the function. Beta reduction orchestrates the process: it strips the λ, prepares the argument, calls `subst` to do the replacement, and then cleans up the indices.

With that distinction clear, let's build `subst` itself. We'll do it in three stages: a naïve version that's almost right, then two fixes that repair it.

What substitution means, concretely. `subst` k s e means "in e, replace every free occurrence of index k with the term s." The caller decides what k and s are. When beta reduction calls substitution, it always starts with $k = 0$ (the variable the stripped λ used to bind). But substitution itself is general—it can target any index.

Stage 1: the naïve attempt. Let's try the simplest thing that could possibly work. For each term form:

- **Variable** n: if $n = k$, replace it with s. Otherwise leave it.
- **Application**: recurse into both sides.
- **Lambda**: recurse into the body.

```
-- WRONG! But a useful starting point.
def substNaive (k : Nat) (s : Expr) : Expr → Expr
  | .bvar n    => if n == k then s else .bvar n
  | .app e₁ e₂ => .app (substNaive k s e₁) (substNaive k s e₂)
  | .lam e     => .lam (substNaive k s e)  -- BUG: k and s unchanged
```

This works for terms with no nested lambdas. Let's test it on a simple case:

```
substNaive 0 (bvar 5) (bvar 0)       = bvar 5     ✓  replaced
substNaive 0 (bvar 5) (bvar 3)       = bvar 3     ✓  left alone
substNaive 0 (bvar 5) (app (bvar 0) (bvar 1))
                                      = app (bvar 5) (bvar 1)  ✓
```

But now try it on a lambda:

```
substNaive 0 (bvar 5) (λ. 0)         = λ. (bvar 5)     ✗ WRONG!
```

The term $\lambda.\,0$ is $\lambda x.\,x$—the identity function. Index 0 inside the body refers to the λ's *own* parameter, not to the external variable 0 we wanted to replace. But our naïve code doesn't know the difference: it sees index 0, matches $k = 0$, and replaces it.

Diagnosing the bug

11.3. The naïve version also fails on `substNaive` 0 (`bvar` 5) ($\lambda.\,\lambda.\,0\ \ 1\ \ 2$). The term $\lambda.\,\lambda.\,0\ \ 1\ \ 2$ is $\lambda x.\,\lambda y.\,y\ \ x\ \ z$ where z is free (index 2). Which variables get incorrectly replaced?
Hint: Inside two λ's, indices 0 and 1 are bound. The free variable 0 from outside is now at index 2.

Stage 2: fix the target index. The problem is clear: when we descend under a λ, every index gets pushed up by one (there's a new binder between us and whatever the index pointed to). The external variable 0 that we're looking for doesn't disappear—it becomes index 1 inside the first λ, index 2 inside two λ's, and so on.

So when we recurse under a λ, we must *increment the target*:

```
-- STILL WRONG! But closer.
def substV2 (k : Nat) (s : Expr) : Expr → Expr
  | .bvar n    => if n == k then s else .bvar n
  | .app e₁ e₂ => .app (substV2 k s e₁) (substV2 k s e₂)
```

```
  | .lam e     => .lam (substV2 (k + 1) s e)  -- FIX 1: target shifts
                --              ^^^^^
```

Let's re-test:

```
substV2 0 (bvar 5) (λ. 0)
  = λ. (substV2 1 (bvar 5) (bvar 0))
  = λ. (bvar 0)                              ✓  bound var untouched!

substV2 0 (bvar 5) (λ. 1)
  = λ. (substV2 1 (bvar 5) (bvar 1))
  = λ. (bvar 5)                              ✓  free var replaced!
```

Progress! But there's still a bug. Try this:

```
substV2 0 (bvar 1) (λ. 1)
  = λ. (substV2 1 (bvar 1) (bvar 1))
  = λ. (bvar 1)                              ✗ WRONG!
```

We're substituting $s =$ `bvar 1` (some free variable) into $\lambda.\,1$. Index 1 in the body refers to the external variable 0—the one we want to replace. So the result should contain s. And it does contain `bvar 1`… but now that `bvar 1` is *inside* a λ, so it points to a different thing! From inside the λ, index 1 refers to whatever is *one* binder outside, not to the original free variable s was pointing at.

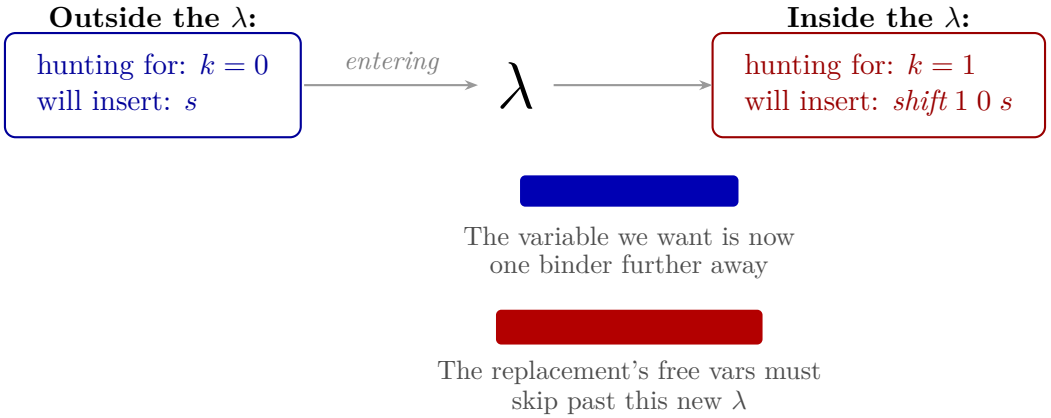
Seeing the capture

11.4. Work through `substV2` 0 (`bvar` 0) ($\lambda.\,1$) in named notation.
The named version is: substitute y for x in $\lambda z.\,x$. The result should be $\lambda z.\,y$. But index 0 inside the λ refers to z, not y!
What index should the result have inside the λ to correctly refer to y?
Answer: It should be `bvar` 1 inside the λ (skip past z's binder to reach y). The replacement term `bvar` 0 needed to become `bvar` 1 because it moved under a binder.

Stage 3: fix the replacement term. The replacement term s is being dropped into a context that has one more binder than where s came from. Its free variables need to increase by 1 to skip over that new binder. This is exactly what `shift` does:

```
-- CORRECT!
def subst (k : Nat) (s : Expr) : Expr → Expr
  | .bvar n    => if n == k then s else .bvar n
  | .lam e     => .lam (subst (k + 1) (shift 1 0 s) e)
  --                          ^^^^^^^  ^^^^^^^^^^^^
  --                          FIX 1    FIX 2
  --                          target   replacement moves under
  --                          moves    a binder: shift its
  --                          further  free variables up by 1
  --                          away
  | .app e₁ e₂ => .app (subst k s e₁) (subst k s e₂)
```

Here is what happens visually when substitution walks under a λ:



Let's verify the case that broke before:

```
subst 0 (bvar 1) (λ. 1)
  = λ. (subst 1 (shift 1 0 (bvar 1)) (bvar 1))
  = λ. (subst 1 (bvar 2) (bvar 1))       -- s shifted: 1 → 2
  = λ. (bvar 2)                           ✓  correctly points past the λ
```

But wait—why does 1 become 2? This is the most confusing part of de Bruijn substitution, and it's worth pausing on. You might think: "The target term $\lambda.\,1$ already uses indices from *inside* the λ. The 1 already accounts for the λ being there. So why are we adjusting anything?"

You're right that the target term is already in "inside coordinates." The problem is that k and s are not. They were decided from *outside* the λ, in "outside coordinates." When we step inside, we need to translate them.

The coordinate system analogy. Imagine naming each variable by its index from the outside. Suppose the context above us has x at index 0 and z at index 1:

```
Standing OUTSIDE the lambda:
    index 0 = x       ← k=0 means "replace x"
    index 1 = z       ← s=(bvar 1) means "insert z"

Standing INSIDE the lambda (which binds w):
    index 0 = w       ← the lambda's own parameter (NEW)
    index 1 = x       ← was index 0 from outside
    index 2 = z       ← was index 1 from outside
```

The lambda inserted a new floor between us and everything external. From outside, the instruction was "replace x (index 0) with z (index 1)." From inside, the *same* instruction is "replace x (index 1) with z (index 2)." In named notation, nothing changed—it's still "replace x with z." But in de Bruijn indices, the *addresses* of x and z both shifted up by 1 because we moved deeper into the building.

So the substitution transforms from:

$$(\lambda.\,1)[0 := \texttt{bvar}\ 1] \quad \xrightarrow{\text{enter the } \lambda} \quad (1)[1 := \texttt{bvar}\ 2]$$

And now $1 = 1$, so the substitution fires, producing `bvar 2`. Wrapped back in the λ: $\lambda.\,2$.

In named terms, this is just $(\lambda w.\, x)[x := z] = \lambda w.\, z$. The result still ignores its parameter and returns a free variable—but z instead of x.

The two fixes are a coordinate translation

The body of the λ is already expressed in "inside coordinates." But k and s arrived from outside, in "outside coordinates." The two fixes translate them into the same coordinate system as the body:
Fix 1 ($k+1$): Translates the target. "Replace index 0" becomes "replace index 1" because x moved from position 0 to position 1 when w took position 0.
Fix 2 (`shift 1 0 s`): Translates the replacement. "Insert bvar 1" becomes "insert bvar 2" because z moved from position 1 to position 2 for the same reason.
Both fixes do the same thing: add 1 to account for the new binder. Fix 1 does it to a single number (k). Fix 2 does it to every free variable inside a term (s). Same reason, same adjustment, applied to different things.

Why shifting is separate from substitution. You might wonder: why not bake shifting directly into the substitution function instead of having a separate `shift`? The reason is that `shift` is needed independently in beta reduction (step 1 and step 3 below), and having it as a separate, well-tested function makes each piece easier to reason about. The separation mirrors a general principle: build small, correct components and compose them.

Building intuition

11.5. Before reading further, think about what substitution should do for each of the three term forms:
(a) If the term is a **variable** n: when should it be replaced? What happens when $n \neq k$?
(b) If the term is an **application** $e_1\ e_2$: what do you do?
(c) If the term is a **lambda** $\lambda.\ body$: which of the two fixes applies here, and why?
(d) What would go wrong if you applied fix 1 but not fix 2? What about fix 2 but not fix 1?

Tracing the target

11.6. Suppose we're doing `subst` 0 s $(\lambda.\,\lambda.\,\lambda.\,3)$.
(a) At the outermost call, what is k?
(b) After descending through the first λ, what is k?
(c) After all three λs, what is k? Does it match the index 3?
Answer: k goes $0 \to 1 \to 2 \to 3$. Yes, index 3 inside three binders refers to the same variable as index 0 outside. The substitution fires correctly.

Tracing substitution

11.7. Work through `subst` 0 (`bvar` 1) $(\lambda.\,0\ \ 1)$ by hand. This represents substituting a free variable (index 1) for variable 0 in the body $\lambda x.\, x\ \ \square$ (where $\square$ is the variable being replaced).
Step 1: We hit the λ. We recurse with $k = 0 + 1 = 1$ and $s =$ `shift` 1 0 (`bvar` 1) = `bvar` 2.
Step 2: Now we substitute into the application 0 1:

- Left: `subst` 1 (`bvar` 2) (`bvar` 0). Is $0 = 1$? No. Result: `bvar` 0. (This is the λ's own variable—correctly untouched.)
- Right: `subst` 1 (`bvar` 2) (`bvar` 1). Is $1 = 1$? Yes! Result: `bvar` 2.

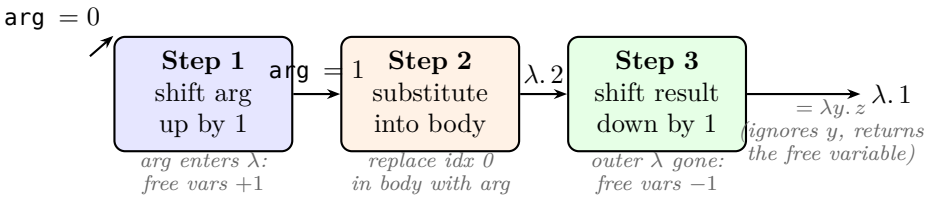
Final result: $\lambda.\,0\ \ 2$. The bound variable 0 stayed put; the free variable got replaced and correctly shifted.

Beta reduction: the full dance. This is where the distinction from earlier pays off. Substitution is the search-and-replace tool; beta reduction is the process that *uses* it. When we reduce $(\lambda.\, body)\ arg$, three things happen:

1. **Shift the argument up:** *arg* is about to enter the body, which is inside the λ. Its free variables need $+1$: `shift` 1 0 *arg*.
2. **Substitute:** Replace index 0 in the body with the shifted argument: `subst` 0 (`shift` 1 0 *arg*) *body*. Here is where substitution gets called—on the **body**, not the whole λ-term. The outer λ is already stripped. Index 0 means "the variable that λ used to bind," which is the nearest external variable from the body's perspective.
3. **Shift the result down:** The λ is gone—we've consumed it. All remaining free variables that pointed past it need -1: `shift` (-1) 0 $(\ldots)$.

Example: $(\lambda.\,\lambda.\,1)$ (**bvar** 0)

K combinator applied to a free variable




```
def betaReduce (body arg : Expr) : Expr :=
  shift (-1) 0 (subst 0 (shift 1 0 arg) body)
```

Why all three steps are needed. Each step exists for a specific reason, and skipping any one produces wrong results. The shift-up and shift-down are *not* redundant—they handle different lambdas. Substitution's internal shifting (which happens when `subst` walks under lambdas *inside* the body) handles lambdas that are still present. The shift-up and shift-down in `betaReduce` handle the lambda that is being *stripped*.

Consider a body that has **no inner lambdas**—just a bare variable:

```
(λ. 0) (bvar 1)            -- identity applied to a free variable
body = bvar 0, arg = bvar 1

Without shift-up:  subst 0 (bvar 1) (bvar 0) = bvar 1
                   shift (-1) 0 (bvar 1)      = bvar 0    ✗ wrong variable!

With shift-up:     shift 1 0 (bvar 1)         = bvar 2
                   subst 0 (bvar 2) (bvar 0)  = bvar 2
                   shift (-1) 0 (bvar 2)      = bvar 1    ✓ correct!
```

Without the shift-up, substitution drops the raw argument into the body. Then the shift-down corrupts it because it was never adjusted for the lambda's scope. The shift-up "pre-adjusts" the argument so that the shift-down brings it back to the right value.

Now consider a case where a variable **survives** substitution. Start with $(\lambda.\,1)$ (`bvar` 5), i.e., $(\lambda x.\, y)\ arg$ where y is free. The function ignores its argument and just returns y.

In the original term $\lambda.\,1$, the `bvar` 1 is **bound**— it points to a binder above the λ we're about

to strip. After stripping, the body is just `bvar 1`, and the binder it pointed to is gone. It's an orphan:

```
(λ. 1) (bvar 5)            -- named: (λx. y) arg; y is bound above

body = bvar 1               -- after stripping, y is FREE (binder gone!)
arg  = bvar 5
target index 0              -- x (the stripped λ's variable)

Step 1: shift arg up      → bvar 6
Step 2: subst 0 (bvar 6) (bvar 1)
        1 ≠ 0, leave it   → bvar 1     -- y survived, x wasn't even here

Without shift-down: bvar 1             ✗ points past a λ that's gone!
With shift-down:    bvar 0             ✓ y is now the nearest external
```

The `bvar 1` was counting the stripped lambda as one of the floors it needed to skip. That floor is gone now, so it must drop to `bvar 0`.

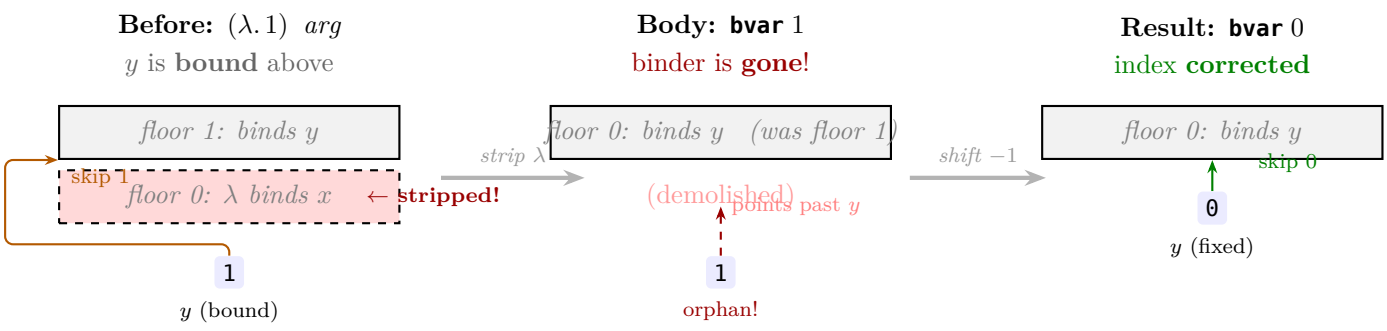
A richer example makes both roles visible. Take $(\lambda.\,\lambda.\,1\ \ 0)$ (`bvar 5`), i.e., $(\lambda y.\,\lambda x.\,y\ x)$ *arg*:

```
body = λ. 1 0,  arg = bvar 5

Step 1: shift arg up       → bvar 6
Step 2: subst 0 (bvar 6) (λ. 1 0)
  enter inner λ: subst 1 (bvar 7) (app (bvar 1) (bvar 0))
    bvar 1 (y): 1 == 1 → replaced with bvar 7       ← target
    bvar 0 (x): 0 ≠ 1 → stays                       ← inner λ's own param
  result: λ. 7 0
Step 3: shift down         → λ. 6 0

Named result: λx. arg x   -- takes x, applies arg to x
```

Here y gets replaced, so the shift-down only adjusts the argument's index ($7 \to 6$). The inner λ's own variable x (index 0) is bound and unaffected.

The building analogy. The clearest way to see shift-down is to think of lambdas as floors in a building. Each variable stores its binder's floor number, counted from its own position upward. Beta reduction *demolishes* a floor—and every variable that pointed above it must update its count.

Consider $(\lambda.\,1)$ (`bvar 5`), i.e., $(\lambda x.\,y)$ *arg*:



The demolished floor is gone, so y's floor (which was floor 1) is now floor 0. The variable was still saying "floor 1" after the demolition—the shift-down corrects it to "floor 0." In general, every free variable that pointed past the stripped λ gets decremented by 1.

The three steps, visualized. Here's what happens to each variable during the three steps of $(\lambda.\,\lambda.\,1\ \ 0)$ (`bvar 5`), i.e., $(\lambda y.\,\lambda x.\,y\ x)$ *arg*. Remember: stripping the outer λ gives body $= \lambda.\,1\ \ 0$, where `bvar 1` *was* bound to y but is now free:

Variable	Step 1 (shift arg up)	Step 2 (substitute)	Step 3 (shift down)	
y (target)	---	`bvar 1` → `bvar 7` (now part of arg)		replaced ✓
arg (free var)	`bvar 5` → `bvar 6`	replaces `bvar 1` (y)	`bvar 7` → `bvar 6`	adjusted ✓
x (inner λ's param)	---	`bvar 0` stays	`bvar 0` stays	untouched ✓

Result: $\lambda.\,6\ \ 0\ =\ \lambda x.\,arg\,x$ ✓

Each step handles a different variable's adjustment: step 1 pre-adjusts the argument so step 3 can bring it back to the right value; step 2 replaces y (the target) with the argument; step 3 would fix any surviving free variables whose floor was demolished—but in this example y was replaced, so only the argument's indices need adjusting.

Why the arg goes up and then back down

The shift-up (step 1) and shift-down (step 3) look like they cancel out for the argument—and for the arg's own indices, they do! $5 \to 6 \to 7 \to 6$ in this example. But the shift-down doesn't *know* which indices came from the argument and which were already in the body. It decrements *all* free variables uniformly. The shift-up pre-compensates the argument so that when the uniform shift-down happens, the argument ends up correct *and* any surviving body variables (like an unreplaced free variable) also end up correct. Both are handled by the same shift-down.

Full beta reduction trace

11.8. Work through the beta reduction of $(\lambda.\,0)$ (`bvar 0`) (applying the identity function to a free variable).
Step 1: `shift` 1 0 (`bvar 0`) = `bvar 1` (free variable shifts up).
Step 2: `subst` 0 (`bvar 1`) (`bvar 0`) = `bvar 1` (index 0 matches, replaced).
Step 3: `shift` (-1) 0 (`bvar 1`) = `bvar 0` (shift back down).
Result: `bvar 0`. The identity function returned its argument unchanged. ✓

A harder trace

11.9. Work through $(\lambda.\,\lambda.\,1)$ (`bvar 0`). This is the constant function $\lambda x.\,\lambda y.\,x$ applied to a free variable. The result should be $\lambda.$ `bvar 1` (a function that ignores its argument and returns the free variable).
Hint: The body is $\lambda.\,1$. Start by shifting the argument, then substitute into the body (which means going under the inner λ), then shift the result down.

And evaluation works the same way—search for a redex, fire the rule:

```
def step : Expr → Option Expr
```

```
  | .app (.lam body) arg => some (betaReduce body arg)
  | .app e₁ e₂ =>
    match step e₁ with
    | some e₁' => some (.app e₁' e₂)
    | none     => match step e₂ with
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | .lam e => match step e with
              | some e' => some (.lam e')
              | none    => none
  | _ => none

def eval (fuel : Nat) (e : Expr) : Expr :=
  match fuel with
  | 0         => e
  | fuel' + 1 => match step e with
                 | none    => e
                 | some e' => eval fuel' e'
```

11.2.4 Different Strategies

With the de Bruijn evaluator working, we can explore different evaluation strategies. Recall from Section 8: normal order evaluates the outermost redex first, while call-by-value evaluates arguments before applying functions.

Strategy challenge

11.10. How would you modify `step` to implement call-by-value? The key difference: before firing $(\lambda.\, body)\ arg \longrightarrow_\beta \ldots$, first check whether `arg` is already a value (a λ). If not, reduce it first.

```
def isValue : Expr → Bool
  | .lam _ => true
  | _      => false

def cbvStep : Expr → Option Expr
  | .app (.lam body) arg =>
    if isValue arg then some (betaReduce body arg)
    else match cbvStep arg with
         | some arg' => some (.app (.lam body) arg')
         | none      => none
  | .app e₁ e₂ =>
    match cbvStep e₁ with
    | some e₁' => some (.app e₁' e₂)
    | none     => match cbvStep e₂ with
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | _ => none
```

Test with **K I** Ω: normal order finds the answer in 2 steps (discards Ω without evaluating it), while call-by-value diverges (tries to evaluate Ω before passing it to **K**).

12 The Simply Typed Lambda Calculus

The untyped λ-calculus lets you write nonsense like Ω. The **simply typed lambda calculus** (STLC, $\lambda_{\to}$) adds types to rule out certain errors statically.

Church introduced the first typed lambda calculus in 1940, just a few years after the untyped version. His motivation was not programming (computers barely existed!) but logic. As Kleene and Rosser had shown in 1935, the untyped λ-calculus was inconsistent as a logic: you could derive any proposition, including false ones, using tricks like the **Y** combinator. By adding types, Church hoped to tame the system just enough to make it logically consistent while keeping its computational expressiveness.

The version we study here—the "simply typed" calculus—is the simplest typed λ-calculus. It was formalized in its modern presentation by Curry in the 1950s and independently by Howard in 1969 (who discovered its deep connection to logic, which we explore in Section 14). The cost: the STLC is not Turing-complete. But this is also its strength— every well-typed STLC term terminates, making it safe to use as a logic.

12.1 Types

Simple Types

$$\tau ::= \alpha \mid \tau_1 \to \tau_2$$

Base types α (like Nat, Bool) and function types. The arrow $\to$ is right-associative: $A \to B \to C = A \to (B \to C)$.

12.2 Turnstile Notation and Typing Judgments

The symbol $\vdash$ is the **turnstile**. It appears in **typing judgments**:

Reading the Turnstile $\vdash$

$$\Gamma \vdash e : \tau$$

reads: "under typing context Γ, expression e has type τ."

Γ	**Typing context**: a list of bindings like $x{:}\mathsf{Nat}$, $f{:}\mathsf{Nat} \to \mathsf{Bool}$
$\vdash$	The **turnstile**: separates assumptions (left) from conclusion (right)
$e : \tau$	The conclusion: e has type τ

Programmer's Analogy

Γ is the set of variables in scope with their types. $\Gamma \vdash e : \tau$ means "given these type bindings, the expression type-checks at type τ." The empty context $\varnothing \vdash e : \tau$ means e is a closed, self-contained, well-typed program.

Related symbols you may encounter:

Symbol	Name	Meaning
$\vdash$	turnstile	"we can derive" (syntactic)
$\Vdash$	double turnstile	"semantically entails"
$\nvdash$	negated turnstile	"we cannot derive"
$\models$	models	"satisfies" (model theory)

12.3 Typing Rules as Inference Rules

An inference rule has premises above the line and a conclusion below:

$$\frac{\text{premise}_1 \qquad \text{premise}_2}{\text{conclusion}}\ \textsc{RuleName}$$

Read it top-down: "if all the premises hold, then the conclusion follows." But there is a more useful way to read it—**bottom-up**: "to prove the conclusion, I need to establish these premises." The bottom-up reading turns type checking into a *detective story*: you start with the thing you want to prove and work backwards, asking "what do I need to know?" at each step.

The STLC has three rules, one for each shape of term:

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{Var}$$

Var says: "To type a variable, look it up in the context." This is the base case—no further premises need proving. It's like checking that a variable is declared before you use it.

$$\frac{\Gamma,\ x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash (\lambda x{:}\tau_1.\, e) : \tau_1 \to \tau_2}\ \textsc{Abs}$$

Abs says: "To type a lambda, *assume* the parameter has its declared type, add that assumption to the context, and check the body. The result is a function type." Notice how Γ grows: we *extend* it with $x : \tau_1$ when we go inside the lambda. This mirrors scoping—inside the function body, the parameter is in scope.

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \textsc{App}$$

App says: "To type an application, the left side must have a function type, and the right side must match the function's domain. The result type is the function's codomain." This has *two* premises—both must be established.

12.4 Building a Proof Tree: The Detective Method

Reading proof trees becomes easy once you learn the bottom-up method. Let's work through a non-trivial example in full detail.

The goal. Type the term $(\lambda f{:}A{\to}B.\, \lambda x{:}A.\, f\ x)$. In English: "a function that takes a function f and a value x, then applies f to x." We want to show $\varnothing \vdash (\lambda f{:}A{\to}B.\, \lambda x{:}A.\, f\ x) : (A \to B) \to A \to B$.

Step 1: What shape is the outermost term? It's a lambda $\lambda f{:}A{\to}B.\ \ldots$, so the Abs rule must apply. That rule says: assume $f : A \to B$, add it to the context, and check the body. Our remaining obligation is:

$$f : A \to B \vdash (\lambda x{:}A.\, f\ x) : A \to B$$

Step 2: The body is another lambda. Shape: $\lambda x{:}A.\, f\ x$. Apply Abs again: assume $x : A$, add it. Remaining obligation:

$$f : A \to B,\ x : A \vdash f\ x : B$$

Step 3: Now we have an application $f\ x$. The App rule applies. It requires two things: type the left side (f) and type the right side (x). This splits into two sub-obligations:

1. $f : A \to B,\ x : A \vdash f : A \to B$ (what type does f have?)
2. $f : A \to B,\ x : A \vdash x : A$ (what type does x have?)

Step 4: Both are variables—use Var. Look each one up in the context $\{f : A \to B,\ x : A\}$. Both are there. Done.

Assembling the tree. Now read the steps in reverse—bottom to top— and you have the proof tree:

$$\dfrac{\dfrac{\dfrac{(f : A{\to}B) \in \{f : A{\to}B,\ x : A\}}{f : A \to B,\ x : A \vdash f : A \to B}\ \textsc{Var} \qquad \dfrac{(x : A) \in \{f : A{\to}B,\ x : A\}}{f : A \to B,\ x : A \vdash x : A}\ \textsc{Var}}{f : A \to B,\ x : A \vdash f\ x : B}\ \textsc{App}}{f : A \to B \vdash (\lambda x{:}A.\, f\ x) : A \to B}\ \textsc{Abs}}{\varnothing \vdash (\lambda f{:}A{\to}B.\, \lambda x{:}A.\, f\ x) : (A \to B) \to A \to B}\ \textsc{Abs}$$

Read the tree upward from the bottom and watch the context grow:

- At the bottom, the context is $\varnothing$ (nothing assumed).
- The first Abs peels off $f{:}A{\to}B$ and adds it: the context becomes $f : A \to B$.
- The second Abs peels off $x{:}A$ and adds it: the context becomes $f : A \to B,\ x : A$.
- The App and Var rules all operate in this full context.

Every line in the tree shows *exactly* what's assumed at that point—nothing is hidden.

How to read any proof tree

The recipe:

1. **Start at the bottom.** That's the statement you want to prove.
2. **Look at the rule name** on the right side of each line. It tells you *why* this step is valid.
3. **Read upward.** Each level decomposes the problem into simpler pieces. Lambdas peel off a binder and extend the context. Applications split into two sub-problems. Variables bottom out by looking things up.
4. **The leaves (top) are axioms.** They're either variable lookups (Var) or assumptions. No premises above them—they're self-evident.

Bottom-up, a proof tree is a plan. Top-down, it's a certificate that the plan works. Type checkers work bottom-up ("what rule applies here?"). Humans verifying proofs work top-down ("do these premises justify this conclusion?").

12.5 More Worked Examples

Proof tree for the identity: $\varnothing \vdash \lambda x{:}A.\, x\ :\ A \to A$

This is the simplest possible proof tree. The term has one lambda (one Abs) wrapping one variable (one Var):

$$\dfrac{\dfrac{(x : A) \in \{x : A\}}{x : A \vdash x : A}\ \textsc{Var}}{\varnothing \vdash (\lambda x{:}A.\, x) : A \to A}\ \textsc{Abs}$$

Proof tree for K: $\varnothing \vdash \lambda x{:}A.\, \lambda y{:}B.\, x \;:\; A \to B \to A$

Two lambdas (two Abs steps), one variable lookup. Notice how the context grows at each lambda: $\varnothing \to \{x : A\} \to \{x : A,\, y : B\}$.

$$\dfrac{\dfrac{\dfrac{(x : A) \in \{x : A,\, y : B\}}{x : A,\; y : B \vdash x : A}\,\textsc{Var}}{x : A \vdash (\lambda y{:}B.\, x) : B \to A}\,\textsc{Abs}}{\varnothing \vdash (\lambda x{:}A.\, \lambda y{:}B.\, x) : A \to B \to A}\,\textsc{Abs}$$

Proof tree for function application

Derive $f : A \to B,\; x : A \vdash f\; x : B$:

$$\dfrac{\dfrac{(f : A{\to}B) \in \{f : A{\to}B,\; x : A\}}{f : A \to B,\; x : A \vdash f : A \to B}\,\textsc{Var} \qquad \dfrac{(x : A) \in \{f : A{\to}B,\; x : A\}}{f : A \to B,\; x : A \vdash x : A}\,\textsc{Var}}{f : A \to B,\; x : A \vdash f\; x : B}\,\textsc{App}$$

A type error: no derivation exists

Try typing $(\lambda x{:}\mathsf{Nat}.\, x)\; (\lambda y{:}\mathsf{Bool}.\, y)$:
The function has type $\mathsf{Nat} \to \mathsf{Nat}$, but the argument has type $\mathsf{Bool} \to \mathsf{Bool}$. App requires $\mathsf{Nat} = \mathsf{Bool} \to \mathsf{Bool}$, which fails. **No derivation exists**: $\nvdash$
This is the type system doing its job: it rejected a "nonsense" application where you'd be treating a boolean function as if it were a number.

12.6 Notational Shorthand in Proof Trees

The proof trees above are fully explicit: every line spells out the entire context. In textbooks and papers, you'll almost always see a compressed version that uses a single letter like Γ to stand for a context that's been defined elsewhere. Knowing how to read the compressed form—and more importantly, how to mentally expand it—is an essential skill.

The convention. Authors write something like "Let $\Gamma = f : A \to B,\; x : A$" and then use Γ throughout the tree. For example, the proof tree for $\varnothing \vdash (\lambda f{:}A{\to}B.\, \lambda x{:}A.\, f\; x) : (A \to B) \to A \to B$ is commonly written:

$$\dfrac{\dfrac{\dfrac{(f : A{\to}B) \in \Gamma}{\Gamma \vdash f : A \to B}\,\textsc{Var} \qquad \dfrac{(x : A) \in \Gamma}{\Gamma \vdash x : A}\,\textsc{Var}}{\Gamma \vdash f\; x : B}\,\textsc{App}}{f : A \to B \vdash (\lambda x{:}A.\, f\; x) : A \to B}\,\textsc{Abs}}{\varnothing \vdash (\lambda f{:}A{\to}B.\, \lambda x{:}A.\, f\; x) : (A \to B) \to A \to B}\,\textsc{Abs}$$

where $\Gamma = f : A \to B,\; x : A$.

This is exactly the same tree as the one in Section 12.4, just compressed. Every occurrence of Γ in the upper lines stands for $f : A \to B,\; x : A$.

Why authors do this. As terms get larger, contexts accumulate many bindings and the tree becomes unreadably wide. The Γ convention keeps it manageable. You'll also sometimes see $\Gamma, x : \tau$ to mean "the context Γ extended with $x : \tau$"—this makes Abs steps especially compact.

How to read the compressed form. When you encounter Γ in a proof tree:

1. Find the definition (usually a "where $\Gamma = \ldots$" line nearby).

2. Mentally substitute the full context back in.
3. Verify each rule still makes sense with the full context visible.

If you ever feel lost reading a proof tree, the first thing to try is expanding every Γ back to its definition.

12.7 Key Theorems of the STLC

The STLC enjoys three fundamental properties that together give it the property known as **type safety**: well-typed programs don't "go wrong."

Theorem 12.1 (Type Preservation / Subject Reduction)**.** If $\Gamma \vdash e : \tau$ and $e \longrightarrow_\beta e'$, then $\Gamma \vdash e' : \tau$.

In other words, beta reduction never changes a term's type. If you start with a well-typed expression of type Nat, every intermediate step of evaluation will also have type Nat. Types are *invariants* of computation.

Theorem 12.2 (Progress)**.** If $\varnothing \vdash e : \tau$ and e is not a value, then $\exists\, e'$ with $e \longrightarrow_\beta e'$.

A well-typed closed term is never "stuck." It's either already a value (a lambda), or there's a reduction step it can take. You'll never reach a state like $x\ y$ where nothing can happen and you don't have a result—that would mean you have a free variable, which type checking in the empty context prevents.

Theorem 12.3 (Strong Normalization)**.** Every well-typed STLC term reaches a normal form. No infinite loops.

This is the deepest result. It says that every well-typed term terminates. This is possible *because* the STLC rejects self-application: you can't type $\lambda x.\, x\ x$ (you'd need $x : A$ and $x : A \to B$ simultaneously), and self-application is the mechanism behind infinite loops (Ω) and recursion ($\mathbf{Y}$).

Together: preservation says types are stable, progress says computation doesn't get stuck, and strong normalization says it eventually finishes. The cost: the STLC is not Turing-complete—no infinite loops means some computable functions can't be expressed. Sections 16–19 explore how to get expressive power back while keeping (some of) these safety properties.

Exercises: Types

12.1. Write the proof tree for typing the **S** combinator: $\varnothing \vdash \lambda x{:}(A \to B \to C).\, \lambda y{:}(A \to B).\, \lambda z{:}A.\, x\ z\ (y\ z) : (A \to B \to C) \to (A \to B) \to A \to C$.
12.2. Why can't $\lambda x.\, x\ x$ be typed in the STLC? What type would x need?
12.3. The term $\lambda f{:}(A \to A).\, \lambda x{:}A.\, f\ (f\ x)$ is the Church numeral $\overline{2}$. What is its type? What does this tell you about the relationship between Church numerals and the STLC?

12.8 Implementing It: A Type Checker in Lean

12.8.1 The Motivation

Some λ-terms are "nonsense"—Ω loops forever, $\lambda x.\, x\ x$ applies a value to itself (what type would x need?). Can we reject these before running them?

Type system design

12.4. Think about what information you would need to decide whether a term "makes sense." If you see $e_1\ e_2$, what must be true about e_1 and e_2 for the application to be valid? What if you see $\lambda x.\, e$—what do you need to know about x?

Think through the application case first, since it's the one that goes wrong. In $e_1\ e_2$, the term e_1 is being *called as a function*. So e_1 must have a function type: it takes some input type A and produces some output type B. The term e_2 is the argument, so it must have type A—matching the function's input. The result has type B.

For a lambda $\lambda x.\, e$: you need to know what type x has (this is the annotation), and then you check whether the body e type-checks under the assumption that x has that type. If the body has type B, then the whole lambda has type $A \to B$.

For a variable: you need to look it up in a context—a list of "which variables have which types." With de Bruijn indices, the context is just a list of types, indexed by position.

This gives you three rules, one per constructor—exactly the inference rules from Section 12. Each rule becomes one branch of the type checker:

```
inductive Ty where
  | base  : String → Ty
  | arrow : Ty → Ty → Ty
  deriving Repr, DecidableEq

-- Typed terms: lambdas carry type annotations
inductive TExpr where
  | bvar : Nat → TExpr
  | lam  : Ty → TExpr → TExpr    -- annotation on the parameter
  | app  : TExpr → TExpr → TExpr
  deriving Repr
```

```
-- Γ ⊢ e : τ    (Γ is List Ty, index 0 = innermost binder)
def typeCheck (ctx : List Ty) : TExpr → Option Ty
  | .bvar n          => ctx[n]?                    -- Var: look up in Γ
  | .lam paramTy body =>                           -- Abs: extend Γ, check body
    match typeCheck (paramTy :: ctx) body with
    | some bodyTy => some (.arrow paramTy bodyTy)
    | none        => none
  | .app e₁ e₂ => do                               -- App: match domain type
    let funTy ← typeCheck ctx e₁
    let argTy ← typeCheck ctx e₂
    match funTy with
    | .arrow paramTy retTy =>
      if paramTy == argTy then some retTy else none
    | _ => none
```

Code ↔ Math

Each branch implements one inference rule from Section 12.3:
`.bvar n` ↔ Var (lookup in Γ), `.lam` ↔ Abs (extend Γ, check body), `.app` ↔ App (match domain type).

Returning `some` τ means $\Gamma \vdash e : \tau$; returning `none` means the term is ill-typed.

13 Scott Encodings

13.1 An Alternative to Church

Church encodings represent data as *folds*: a Church numeral iterates f a total of n times. This is elegant but has a practical problem: some operations are expensive. Predecessor, as we saw, requires $O(n)$ work to "rebuild" the count from scratch. Pattern matching on a Church-encoded sum type similarly requires deconstructing the entire fold.

Scott encodings, introduced by Dana Scott in the 1960s, take a different approach. Scott was one of the architects of denotational semantics— the project of giving mathematical meaning to programming languages. Where Church asked "how do I encode data as iterable functions?", Scott asked "how do I encode data as pattern-matchable functions?" The answer turned out to be simpler and more efficient for many operations, and it is closer to what real compilers actually do internally.

Each constructor becomes a function that takes one handler per constructor and calls the appropriate one. This is essentially what a `match` or `case` expression does at runtime in most compilers—so Scott encodings are closer to how data is actually implemented in practice.

13.2 Reading the Notation

Scott encodings introduce a few notational conventions that differ from the Church encoding sections:

Notation	Meaning
$\overline{n}_S$	The Scott encoding of the number n. The subscript S distinguishes it from the Church numeral $\overline{n}$. Without the subscript, $\overline{n}$ always means the Church version.
$\textsc{Pred}_S$	The Scott version of the predecessor function. Again, the subscript distinguishes it from the Church predecessor $\textsc{Pred}$.
$\lambda z\, s.\ \ldots$	The parameters z and s stand for "zero handler" and "successor handler." A Scott numeral takes these two arguments and calls whichever one matches its shape—z if it's zero, s if it's a successor.
$\lambda n\, c.\ \ldots$	For Scott lists, n stands for "nil handler" and c for "cons handler." Same idea: the list calls whichever handler matches.
$\overline{n+1}_S$	The Scott encoding of "the successor of n." This is a *pattern* in the definition, not an expression to evaluate. It says: "any Scott numeral that represents a non-zero value has this form."

The key mental model: a Scott-encoded value is a function that takes **one handler per constructor** and calls the matching one. For natural numbers, there are two constructors (zero and successor), so every Scott numeral takes two arguments. For lists, there are two constructors (nil and cons), so every Scott list takes two arguments. This is directly analogous to a `match` or `case` expression in a programming language:

```
-- A Scott numeral IS this match, packaged as a function
match n with
| zero   => z            -- call the zero handler
| succ p => s p          -- call the successor handler with predecessor p
```

13.3 Scott Booleans

Scott booleans look identical to Church booleans (both are choosing between two options):

$$\textsc{STrue} \triangleq \lambda t\, f.\, t \qquad\qquad \textsc{SFalse} \triangleq \lambda t\, f.\, f$$

No difference here—the distinction appears with more complex types.

13.4 Scott Numerals

A natural number is either zero or the successor of another number. In a pattern match, we provide one handler for each case:

Scott Numerals

$$\overline{0}_S \triangleq \lambda z\, s.\, z$$
$$\overline{n+1}_S \triangleq \lambda z\, s.\, s\, \overline{n}_S$$

Zero takes two handlers and calls the first (z). A successor $n+1$ takes two handlers and calls the second (s) with the **predecessor** $\overline{n}_S$ as an argument. This is the crucial difference from Church encoding: the predecessor is directly available, not reconstructed.

$$\begin{aligned}
\overline{0}_S &= \lambda z\, s.\, z \\
\overline{1}_S &= \lambda z\, s.\, s\, \overline{0}_S \\
\overline{2}_S &= \lambda z\, s.\, s\, \overline{1}_S \\
\overline{3}_S &= \lambda z\, s.\, s\, \overline{2}_S
\end{aligned}$$

Now $\textsc{Pred}$ is trivial—just call the number with handlers that extract the predecessor:

$$\textsc{Pred}_S \triangleq \lambda n.\, n\, \overline{0}_S\, (\lambda p.\, p)$$

If n is zero, return zero. If $n = s\ p$, the successor handler receives p directly. Compare this to the complex pair-walking trick needed for Church numerals!

$\textsc{Pred}_S\ \overline{3}_S = \overline{2}_S$

$$\begin{aligned}
\textsc{Pred}_S\ \overline{3}_S &= (\lambda n.\, n\, \overline{0}_S\, (\lambda p.\, p))\ (\lambda z\, s.\, s\, \overline{2}_S) && \text{(expand definitions)} \\
&\longrightarrow_\beta (\lambda z\, s.\, s\, \overline{2}_S)\ \overline{0}_S\ (\lambda p.\, p) && \text{(plug in } \overline{3}_S\text{)} \\
&\twoheadrightarrow_\beta (\lambda p.\, p)\ \overline{2}_S && \text{(it's a successor, so call } s \text{ with the predecessor)} \\
&\longrightarrow_\beta \overline{2}_S \quad \checkmark && \text{(identity returns it unchanged)}
\end{aligned}$$

Church vs. Scott

Church encoding: "I am the number 3 because I can apply things 3 times." You know *how to iterate* but not how to decompose.
Scott encoding: "I am the number 3 because I am the successor of 2." You know *how to pattern-match* and have direct access to subcomponents.
Church encodings give you `foldr` for free. Scott encodings give you `case` for free. Most real compilers (GHC, OCaml) use representations closer to Scott encoding internally, because pattern matching is the more fundamental operation—you can build folds from pattern matching, but building efficient pattern matching from folds is harder.

13.5 Scott-Encoded Lists

$$\textsc{SNil} \triangleq \lambda n\, c.\, n$$
$$\textsc{SCons} \triangleq \lambda h\, t\, n\, c.\, c\; h\; t$$

SNil selects the nil handler. SCons $h\ t$ selects the cons handler and passes it the head h and tail t directly. Extracting the head or tail is a single reduction step—no folding required.

14 The Curry–Howard Correspondence

14.1 Types Are Propositions; Programs Are Proofs

There is a deep and beautiful connection between typed lambda calculus and logic. It is arguably the most profound idea in this entire document.

The story unfolds over three decades. In 1934, Haskell Curry noticed that the types of certain combinators corresponded to axiom schemes in propositional logic—for instance, the type of **K**, namely $A \to B \to A$, is a well-known logical axiom. He noted this as a curiosity but did not develop it further.

In 1958, Curry and his collaborator Robert Feys wrote up the observation more carefully in their monumental textbook *Combinatory Logic, Volume I*. But the full picture remained unclear until 1969, when William Howard circulated an unpublished manuscript (finally published in 1980) making the correspondence explicit and systematic: types *are* logical propositions, and programs *are* proofs. Every rule of typed lambda calculus corresponds precisely to a rule of natural deduction in logic. Howard extended the correspondence beyond simple types to include universal quantification, existential types, and disjunction.

The correspondence says:

Type theory	**Logic**
Type τ	Proposition
Term $e : \tau$	Proof of the proposition
Function type $A \to B$	Implication "A implies B"
Inhabited type	Provable proposition
Empty type (no terms)	False / unprovable proposition
β-reduction	Proof simplification

This is not a loose analogy. It is a precise, formal correspondence. Every valid typing derivation in the STLC *is* a proof in propositional logic, and every proof *is* a well-typed program. This correspondence is the intellectual foundation of modern proof assistants—Lean, Coq, Agda, and Idris all exploit it directly, verifying mathematical proofs by type-checking programs.

14.2 Examples

The identity function proves $A \Rightarrow A$. The term $\lambda x{:}A.\, x$ has type $A \to A$. Under Curry–Howard, this is a proof that "if A, then A"—trivially true, since assuming A gives you A. The proof is the program: take the evidence and return it unchanged.

The K combinator proves $A \Rightarrow B \Rightarrow A$. $\lambda x{:}A.\, \lambda y{:}B.\, x$ has type $A \to B \to A$. This proves "if A, then (if B, then A)"—given evidence for A and B, you can produce evidence for A by ignoring B. The "weakening" or "thinning" rule of logic.

The S combinator proves $(A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$. This is the logical axiom scheme known as the *distribution* of implication. The program witnesses the proof: given a function from A and B to C, and a function from A to B, and an A, produce a C by applying them in the right combination.

Modus ponens is function application. The logical rule "from A and $A \Rightarrow B$, conclude B" corresponds exactly to the App typing rule: if you have $e_1 : A \to B$ and $e_2 : A$, then $e_1\ e_2 : B$.

Why this matters

The Curry–Howard correspondence is the foundation of proof assistants like Lean, Coq, and Agda. When you write a proof in Lean, you are literally writing a lambda calculus program. The type checker verifies the proof by type-checking the program. This is why our lambda calculus interpreter and Lean itself are built on the same ideas: they *are* the same ideas.

The correspondence also explains why the STLC is not Turing-complete: every well-typed STLC term normalizes (strong normalization theorem). Under Curry–Howard, this means every proof terminates—which is a property you want! A logic where you can prove anything (including false) by writing an infinite loop would be useless as a logic.

Reading a proof tree as a logical derivation

The typing derivation for $\lambda x{:}A.\, \lambda y{:}B.\, x : A \to B \to A$ is simultaneously a derivation in propositional logic:

$$\dfrac{\dfrac{\dfrac{(A) \in \text{assumptions}}{A}\ \textsc{Assumption}}{B \Rightarrow A}\ \Rightarrow\text{-Intro (discharge } B)}{A \Rightarrow B \Rightarrow A}\ \Rightarrow\text{-Intro (discharge } A)$$

This is exactly the same tree structure as the Var/Abs typing derivation. The Abs rule in type theory is the $\Rightarrow$-Introduction rule in logic. The Var rule is the assumption rule.

Exercises: Going Deeper

14.1. Define $\textsc{Succ}_S$ for Scott numerals and verify that $\textsc{Succ}_S\ \overline{2}_S = \overline{3}_S$.

14.2. Define $\textsc{IsZero}_S$ for Scott numerals. How much simpler is it compared to the Church version?

14.3. Under Curry–Howard, what logical proposition does the type $(A \to B) \to (B \to C) \to (A \to C)$ correspond to? Write a λ-term that has this type.
14.4. Why is there no closed term of type $A \to B$ (where A and B are distinct base types) in the STLC? What does this say about the corresponding logical proposition?
14.5. The type $\forall\alpha.\,\alpha$ (from System F, not the STLC) is uninhabited. Under Curry–Howard, what proposition does it represent?

15 Intuitionistic Logic

Before we can understand dependent types, we need to understand the philosophical revolution they grew out of. This detour into the foundations of mathematics may seem far from programming, but it leads directly to the ideas that make Lean, Coq, and Agda work.

15.1 The Classical View

In classical logic—the logic most people learn in school and most mathematicians use daily—every proposition is either true or false, regardless of whether anyone can prove it. The Goldbach conjecture ("every even number greater than 2 is the sum of two primes") is either true or false *right now*, even though nobody has proved it either way. Truth exists independently of our ability to verify it.

Classical logic gives you several powerful principles:

Law of Excluded Middle (LEM)	For any proposition P: either P or $\neg P$. There is no middle ground.
Double Negation Elim. (DNE)	If $\neg\neg P$, then P. If it's impossible for P to be false, then P is true.
Proof by Contradiction	To prove P, assume $\neg P$ and derive a contradiction. Since $\neg P$ led to absurdity, P must be true.

These feel obviously correct, and mathematicians use them constantly. But the constructivists noticed something unsettling about them.

The constructivist objection. Consider this classical proof: "There exist irrational numbers a and b such that a^b is rational."

Proof. We know $\sqrt{2}$ is irrational. Now consider the number $\sqrt{2}^{\sqrt{2}}$—we have no idea whether this is rational or irrational. But by the law of excluded middle, it must be one or the other.

Case 1: Suppose $\sqrt{2}^{\sqrt{2}}$ happens to be rational. Then take $a = \sqrt{2}$ and $b = \sqrt{2}$. Both are irrational, and $a^b = \sqrt{2}^{\sqrt{2}}$, which we assumed is rational. Done.

Case 2: Suppose $\sqrt{2}^{\sqrt{2}}$ is irrational. Then take $a = \sqrt{2}^{\sqrt{2}}$ (irrational by our assumption in this case) and $b = \sqrt{2}$ (irrational). Now compute a^b using the exponent law $(x^y)^z = x^{y \cdot z}$:

$$a^b \;=\; \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} \;=\; \sqrt{2}^{\,\sqrt{2}\cdot\sqrt{2}} \;=\; \sqrt{2}^{\,2} \;=\; 2$$

And 2 is rational. Done.

In both cases we found irrational a, b with a^b rational. So such a pair exists. □

The proof is valid in classical logic. But notice: *we don't know which case holds.* Is $\sqrt{2}^{\sqrt{2}}$ rational or irrational? The proof doesn't tell us—it just says "one of these two cases must be

true, and either way we win." We proved "there exist a and b" without being able to point to *which* a and b actually work. (It turns out $\sqrt{2}^{\sqrt{2}}$ is irrational, by the Gelfond–Schneider theorem—but proving that requires entirely different and much harder mathematics that this proof never invokes.)

We have an existence proof that doesn't produce the thing whose existence it claims.

For the constructivists—led by L. E. J. Brouwer in the 1910s and 1920s—this is not a real proof. Brouwer argued that mathematical objects don't exist in some Platonic realm waiting to be discovered. They exist only when a mathematician *constructs* them. To prove "there exists an x with property P," you must exhibit a specific x and demonstrate that it has property P. A proof by contradiction that merely shows non-existence is absurd—without pointing to a witness—is not a valid existence proof.

What intuitionistic logic rejects. Brouwer's student Arend Heyting (1930) formalized these ideas into **intuitionistic logic**: a logic that modifies the classical rules to enforce constructivity. The key differences:

Principle	**Classical**	**Intuitionistic**
$P \vee \neg P$ (excluded middle)	✓	not assumed
$\neg\neg P \Rightarrow P$ (double negation)	✓	not valid
Proof by contradiction for existence	✓	not valid
$\neg\neg\neg P \Rightarrow \neg P$	✓	✓
Everything else in propositional logic	✓	✓

Intuitionistic logic does not say excluded middle is *false*. It says excluded middle is *not assumed*—it might hold for specific propositions (e.g., "n is even or n is odd" is constructively valid because you can check by dividing), but you can't assert it for *every* proposition by default.

The BHK interpretation. Brouwer, Heyting, and Kolmogorov developed an interpretation of what proofs *mean* in intuitionistic logic, now called the **BHK interpretation**:

To prove...	**You must provide...**
$A \wedge B$ ("A and B")	A proof of A *and* a proof of B (a pair).
$A \vee B$ ("A or B")	A proof of A *or* a proof of B, together with a tag saying *which* (a tagged union).
$A \Rightarrow B$ ("A implies B")	A *method* that transforms any proof of A into a proof of B (a function).
$\exists x.\, P(x)$ ("there exists")	A specific witness a together with a proof that $P(a)$ holds (a dependent pair).
$\forall x.\, P(x)$ ("for all")	A method that, given any x, produces a proof of $P(x)$ (a dependent function).
$\neg A$ ("not A")	A method that transforms any proof of A into a proof of $\bot$ (absurdity)—i.e., a proof that A leads to contradiction.

Read that table again carefully. Do the right-hand descriptions remind you of anything?

A pair. A tagged union. A function. A dependent pair. A dependent function. These are **data**

types. The BHK interpretation, formulated decades before computers, is describing *programs*. A proof of "A and B" is a pair. A proof of "A implies B" is a function. A proof of "there exists x with $P(x)$" is a struct with a field for the witness and a field for the evidence.

Why AND is a product type (and multiplication). A proof of $A \wedge B$ consists of a proof of A *and* a proof of B, packaged together as a pair (a, b) where $a : A$ and $b : B$. The type of such pairs is the **product type** $A \times B$.

Why "product"? Because the number of values in $A \times B$ is the *product* of the number of values in A and B. If $A = \mathsf{Bool}$ (2 values) and $B = \mathsf{Bool}$ (2 values), then $A \times B$ has $2 \times 2 = 4$ values: $(\texttt{true}, \texttt{true})$, $(\texttt{true}, \texttt{false})$, $(\texttt{false}, \texttt{true})$, $(\texttt{false}, \texttt{false})$.

In Lean, the product type is `Prod A B` (written `A × B`), and the conjunction $A \wedge B$ is defined as a structure with two fields—which is exactly a product type. They are the same thing.

Why OR is a sum type (and addition). A proof of $A \vee B$ consists of *either* a proof of A *or* a proof of B, together with a tag saying which one you're providing. This is a **sum type** (also called a tagged union, disjoint union, or coproduct): $A + B$.

Why "sum"? Because the number of values in $A + B$ is the *sum* of the number of values in A and B. If $A = \mathsf{Bool}$ (2 values) and $B = \mathsf{Unit}$ (1 value), then $A + B$ has $2 + 1 = 3$ values: `inl(true)`, `inl(false)`, `inr(())`.

In Lean, disjunction $A \vee B$ is an inductive type with two constructors `Or.inl : A → A ∨ B` and `Or.inr : B → A ∨ B`—which is exactly a sum type.

Why implication is a function type (and exponentiation). A proof of $A \Rightarrow B$ is a method that transforms any proof of A into a proof of B—a function $A \to B$. The number of functions from A to B is $|B|^{|A|}$—exponentiation. If $A = \mathsf{Bool}$ (2 values) and $B = \mathsf{Bool}$ (2 values), there are $2^2 = 4$ functions from Bool to Bool: always-true, always-false, identity, and negation.

This gives us a complete arithmetic of types:

Logic	**Type**	**Arithmetic**	**Lean**
$A \wedge B$	$A \times B$ (product)	$\lvert A\rvert \cdot \lvert B\rvert$	`A × B`
$A \vee B$	$A + B$ (sum)	$\lvert A\rvert + \lvert B\rvert$	`A ⊕ B`, `Sum`
$A \Rightarrow B$	$A \to B$ (function)	$\lvert B\rvert^{\lvert A\rvert}$	`A → B`
$\top$ (true)	Unit	1	`Unit`
$\bot$ (false)	Empty	0	`Empty`
$\neg A$	$A \to$ Empty	$0^{\lvert A\rvert}$	`A → Empty`

The last row is illuminating: $0^{|A|} = 0$ when $|A| > 0$ (there are no functions from a nonempty type to an empty type) and $0^0 = 1$ (there is exactly one function from the empty type to itself—the identity on nothing). This matches the logic: $\neg A$ is unprovable when A is provable, and $\neg\bot$ has exactly one proof.

Even the laws of arithmetic correspond to logical laws: $A \times (B + C) = A \times B + A \times C$ is distributivity of AND over OR. $A \to (B \times C) \cong (A \to B) \times (A \to C)$ says a function returning a pair is the same as a pair of functions—which is the logical tautology $(A \Rightarrow B \wedge C) \Leftrightarrow (A \Rightarrow B) \wedge (A \Rightarrow C)$.

This is the Curry–Howard correspondence, seen from the logic side. The constructivists were describing computation before computation was formalized.

Why programmers should care. The connection to programming is not a metaphor. In a dependently typed proof assistant like Lean:

```
-- Fix A and B as propositions so `modus_ponens` is itself a proposition.
-- (Without this, Lean auto-binds A B : Sort _, and rejects modus_ponens
--  with "type of theorem modus_ponens is not a proposition". The ∧/∨/∃
--  lines happen to survive because those connectives force Prop.)
variable {A B : Prop}

-- "A and B" is a struct (pair)
theorem and_intro (ha : A) (hb : B) : A ∧ B := ⟨ha, hb⟩

-- "A or B" is a tagged union (sum type)
theorem or_left (ha : A) : A ∨ B := Or.inl ha

-- "A implies B" is a function
theorem modus_ponens (hab : A → B) (ha : A) : B := hab ha

-- "there exists n such that n > 5" needs a witness
theorem exists_gt_5 : ∃ n : Nat, n > 5 := ⟨6, by omega⟩
```

The proof of `exists_gt_5` must provide the number 6—an actual witness—and a proof that $6 > 5$. You cannot prove it by contradiction. You cannot say "suppose no such n exists" and derive absurdity. You must hand over the goods.

This is intuitionistic logic in action. Lean's kernel is an intuitionistic type theory. Every proof must construct its evidence. This is why proofs in Lean are programs: the BHK interpretation *is* the type system.

Classical logic in Lean

Lean does allow classical reasoning—but only when you explicitly opt in by importing `Classical`. When you write `Classical.em P` (excluded middle for P), or use the `by_contra` tactic, Lean adds the axiom of choice or excluded middle as an additional assumption. The proof still type-checks, but it is marked as "noncomputable"—Lean can verify it but cannot extract a runnable program from it, because the proof may use existence claims without witnesses. This is the type-theoretic manifestation of the constructivist objection: classical proofs don't always compute.

Why "intuitionistic"? The name comes from Brouwer's philosophy of **intuitionism**, which held that mathematics is a creation of the human mind, not a description of an external Platonic reality. Mathematical objects exist only when constructed by mental activity. A proof must be a mental construction that can, in principle, be carried out step by step. Brouwer himself was quite radical—he rejected formalization entirely and thought mathematics should be done purely through intuition, without symbols. The irony is that his followers formalized his ideas into one of the most precisely axiomatized logical systems in existence, and that formalization became the foundation of machine-checked proof.

16 System F: Polymorphism

16.1 The Problem of Repetition

The STLC has an embarrassing limitation. Consider the identity function. In the untyped calculus, $\lambda x.\, x$ works on anything—numbers, booleans, other functions. But in the STLC, types are rigid: you need $\lambda x{:}\mathsf{Nat}.\, x$ for natural numbers, $\lambda x{:}\mathsf{Bool}.\, x$ for booleans, $\lambda x{:}(\mathsf{Nat} \to \mathsf{Nat}).\, x$ for functions on naturals, and so on—infinitely many copies of the "same" function. This isn't a minor inconvenience. It means fundamental operations like composition, $\lambda f.\, \lambda g.\, \lambda x.\, f\ (g\ x)$, cannot be given a single type. You cannot even type Church numerals properly: $\overline{2}$ applies its first argument twice, but the STLC insists on knowing *which type* the argument has.

The question is: can we add "generics" to the type system without losing the safety properties (progress, preservation, strong normalization) that make the STLC useful?

16.2 Two Independent Discoveries

The answer came from two completely different intellectual traditions, arriving at the same system independently.

Girard (1972): from proof theory. Jean-Yves Girard, a French logician, was working on the proof theory of second-order arithmetic—a system where you can quantify not just over numbers ("for all n, …") but over *properties* of numbers ("for all predicates P, …"). He needed a typed lambda calculus that could express this second-order quantification, and he constructed what he called "System F" (the "F" was simply the next letter in his naming sequence—his earlier systems were called System D and System E).

Girard's breakthrough was proving that System F satisfies strong normalization. This was a major result in proof theory: it gave a constructive consistency proof for second-order arithmetic, extending the work of Gödel and Gentzen. The proof technique—now called the "Girard–Tait method" or "method of reducibility candidates"—involved defining a semantic notion of "well-behaved term" and showing by induction that all typed terms qualify.

Reynolds (1974): from programming. John Reynolds, an American computer scientist, was independently studying how to formalize the idea that some programs are "generic" in their type. He defined what he called the "polymorphic lambda calculus" with the same syntax and typing rules as Girard's System F, but motivated entirely by programming: a polymorphic function should work uniformly across all types, without inspecting which type it receives.

Reynolds also discovered a deep consequence of this uniformity, later refined by Philip Wadler (1989) into the celebrated **parametricity theorem** ("theorems for free"): the type of a polymorphic function severely constrains what it can do. A function of type $\forall\alpha.\, \alpha \to \alpha$ *must* be the identity—there is literally nothing else a polymorphic function of that type can do, because it cannot inspect its argument's type. The type alone proves the behavior.

16.3 The Key Idea

System F allows functions to take *types* as arguments, not just values. This lets you write a single identity function that works at any type:

$$\Lambda\alpha.\, \lambda x{:}\alpha.\, x \quad : \quad \forall\alpha.\, \alpha \to \alpha$$

The $\Lambda\alpha$ (capital lambda) abstracts over a type variable α. The $\forall\alpha$ in the type says "for any type α." When you *apply* this function, you first supply a type, then a value:

$$(\Lambda\alpha.\,\lambda x{:}\alpha.\,x)\ [\mathsf{Nat}]\ 5 \longrightarrow_\beta (\lambda x{:}\mathsf{Nat}.\,x)\ 5$$
$$\longrightarrow_\beta 5$$

The [Nat] is a **type application**: it instantiates α to Nat, giving back the Nat-specific identity function.

System F in programming languages

System F is the theoretical foundation for generics. When you write `fn identity<T>(x: T) -> T` in Rust, you are writing a System F term. The angle brackets `<T>` are ΛT. The `where` clauses of Rust traits, Haskell type classes, and Java's `<T extends Comparable>` are all refinements of System F's type abstraction.

Most practical languages use a restricted form called **Hindley–Milner** (HM) type inference, discovered independently by Hindley (1969) and Milner (1978). HM restricts where $\forall$ can appear (only at the outermost level of a let-binding) but in exchange gives you *complete type inference*: the compiler can figure out all the types without any annotations. This is why Haskell and ML rarely need type signatures—the HM algorithm infers them automatically. Full System F, by contrast, has undecidable type inference (proved by Joe Wells, 1999): you sometimes *must* provide annotations.

16.4 The Philosophical Significance

System F reveals something deep: **data can emerge from pure logic**. In System F, you can encode natural numbers, booleans, pairs, and lists *entirely within the type system*:

$$\mathsf{Nat}_F \triangleq \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha$$

This is the type of Church numerals! In the STLC, Church numerals have type $(A \to A) \to A \to A$ for a *fixed* type A. In System F, the $\forall\alpha$ lets a single numeral work at any type. Similarly:

$$\mathsf{Bool}_F \triangleq \forall\alpha.\,\alpha \to \alpha \to \alpha$$
$$\mathsf{Pair}_F\ A\ B \triangleq \forall\alpha.\,(A \to B \to \alpha) \to \alpha$$

These aren't tricks—they are the *natural types* that arise when you ask "what is the most general type a Church boolean can have?" The answer *is* the universally quantified type above. The data types we take for granted in programming (integers, booleans, lists) can all be understood as patterns of polymorphic function application. From the perspective of proof theory, Girard's encoding showed that second-order logic is rich enough to define all the data structures of mathematics internally.

16.5 Implementing System F

System F extends the STLC with two new constructs: **type abstraction** ($\Lambda\alpha.\,e$) and **type application** ($e\ [\tau]$). The types grow too: we add $\forall\alpha.\,\tau$ (universally quantified types) and type variables α.

```
-- Types: add type variables and universal quantification
inductive FTy where
  | tvar  : Nat → FTy                    -- type variable (de Bruijn)
  | base  : String → FTy                 -- base types like Nat, Bool
```

```
  | arrow : FTy → FTy → FTy             -- τ₁ → τ₂
  | forall_ : FTy → FTy                -- ∀α. τ  (α is index 0 in τ)
  deriving Repr, DecidableEq

-- Terms: add type abstraction and type application
inductive FExpr where
  | bvar  : Nat → FExpr                -- term variable (de Bruijn)
  | lam   : FTy → FExpr → FExpr        -- λ(x:τ). e
  | app   : FExpr → FExpr → FExpr      -- e₁ e₂
  | tlam  : FExpr → FExpr              -- Λα. e  (type abstraction)
  | tapp  : FExpr → FTy → FExpr        -- e [τ]  (type application)
  deriving Repr
```

The type checker needs two operations on types: **shifting** and **substitution**—the same operations we needed for terms with de Bruijn indices, but now operating on type variables inside types.

```
-- Shift type variables in a type
def FTy.shift (d : Int) (c : Nat) : FTy → FTy
  | .tvar n      => if n ≥ c then .tvar (Int.toNat (n + d)) else .tvar n
  | .base s      => .base s
  | .arrow t₁ t₂ => .arrow (t₁.shift d c) (t₂.shift d c)
  | .forall_ t   => .forall_ (t.shift d (c + 1))  -- under a ∀: bump cutoff

-- Substitute a type for type variable k
def FTy.subst (k : Nat) (s : FTy) : FTy → FTy
  | .tvar n      => if n == k then s
                    else if n > k then .tvar (n - 1) else .tvar n
  | .base s      => .base s
  | .arrow t₁ t₂ => .arrow (t₁.subst k s) (t₂.subst k s)
  | .forall_ t   => .forall_ (t.subst (k + 1) (s.shift 1 0))
```

Now the type checker. It has five cases—three from the STLC (Var, Abs, App) plus two new ones:

```
def fTypeCheck (ctx : List FTy) : FExpr → Option FTy
  -- Same as STLC:
  | .bvar n       => ctx[n]?
  | .lam ty body  =>
    match fTypeCheck (ty :: ctx) body with
    | some bodyTy => some (.arrow ty bodyTy)
    | none        => none
  | .app e₁ e₂    => do
    let funTy ← fTypeCheck ctx e₁
    let argTy ← fTypeCheck ctx e₂
    match funTy with
    | .arrow dom cod => if dom == argTy then some cod else none
    | _              => none
  -- NEW: type abstraction (Λα. e)
  | .tlam body    =>
    -- Shift all types in ctx up (they're now under one more ∀)
    let ctx' := ctx.map (·.shift 1 0)
```

```
    match fTypeCheck ctx' body with
    | some bodyTy => some (.forall_ bodyTy)
    | none        => none
  -- NEW: type application (e [τ])
  | .tapp e ty    => do
    let eTy ← fTypeCheck ctx e
    match eTy with
    | .forall_ bodyTy => some (bodyTy.subst 0 ty)  -- substitute τ for α
    | _               => none
```

The two new rules are:

$$\frac{\Gamma' \vdash e : \tau}{\Gamma \vdash (\Lambda\alpha.\, e) : \forall\alpha.\, \tau}\ \textsc{TAbs} \qquad \frac{\Gamma \vdash e : \forall\alpha.\, \tau}{\Gamma \vdash e\ [\sigma] : \tau[\alpha := \sigma]}\ \textsc{TApp}$$

TAbs says: to type $\Lambda\alpha.\, e$, check e in a context where all existing types are shifted (because we've introduced a new type variable). TApp says: if e has type $\forall\alpha.\, \tau$, then applying it to a type σ gives τ with α replaced by σ. That substitution in the code is `bodyTy.subst 0 ty`—exactly what we built.

```
-- Test: the polymorphic identity  Λα. λ(x:α). x : ∀α. α → α
#eval fTypeCheck [] (.tlam (.lam (.tvar 0) (.bvar 0)))
-- Result: some (FTy.forall_ (FTy.arrow (FTy.tvar 0) (FTy.tvar 0)))
--       = some (∀α. α → α)  ✓
```

17 System Fω: Type Operators

System F lets terms depend on types (polymorphism). But what about types that depend on *other types*? We've already seen the intuition in our discussion of higher-kinded types (Section 18.4): List is not a type—it's a *function on types*. Give it Nat and it returns List Nat. System Fω makes this kind of type-level computation a first-class concept.

17.1 From Types to Kind Systems

In the STLC, every type has the same status: Nat, Bool, Nat → Bool—these are all "types." But in System Fω, we need to distinguish between types that are "complete" (you can have a value of that type) and type constructors that are "waiting for an argument."

This distinction is formalized with **kinds**—the "types of types":

Kind	Meaning	Examples
$\star$	A proper type (has values)	Nat, Bool, List Nat
$\star \to \star$	Takes a type, returns a type	List, Option, IO
$\star \to \star \to \star$	Takes two types, returns a type	Either, Map, Pair
$(\star \to \star) \to \star$	Takes a type constructor, returns a type	Fix (fixed point of a functor)

Kinds are to types what types are to terms. Just as a type system prevents you from applying an integer where a function is expected (3 5 is a type error), a *kind system* prevents you from applying a type where a type constructor is expected (Nat Bool is a *kind* error—Nat has kind $\star$, so it doesn't accept arguments).

17.2 Type-Level Functions

In the STLC and System F, types are static—they can contain type variables, but they can't *compute*. In System Fω, you can define **type-level lambda abstractions**:

$$\lambda\alpha{:}\star\,.\,\mathsf{Pair}\;\alpha\;\alpha \quad : \quad \star \to \star$$

This is a type-level function: give it a type τ, and it returns $\mathsf{Pair}\;\tau\;\tau$ (pairs where both components have the same type). You can apply it: $(\lambda\alpha{:}\star\,.\,\mathsf{Pair}\;\alpha\;\alpha)\;\mathsf{Nat} \;=\; \mathsf{Pair\;Nat\;Nat}$.

This is exactly beta reduction, but at the type level. The computation rule is the same—substitution. What changes is that we allow this computation to happen *inside types*, not just inside terms.

17.3 What System Fω Enables

The practical payoff is the ability to abstract over type constructors, as we saw in Section 18.4. Here is a more formal view:

In System F, you can write:

$$\Lambda\alpha{:}\star\,.\,\lambda(x:\alpha).\,x \quad : \quad \forall(\alpha{:}\star).\,\alpha \to \alpha$$

The type variable α ranges over types of kind $\star$.

In System Fω, you can also write:

$$\Lambda(f{:}\star\to\star).\,\Lambda(\alpha{:}\star).\,\lambda(x:f\;\alpha).\,x \quad : \quad \forall(f{:}\star\to\star).\,\forall(\alpha{:}\star).\,f\;\alpha \to f\;\alpha$$

Now f ranges over type constructors of kind $\star \to \star$—things like List, Option, Tree. Inside the term, we apply f to α to get the concrete type $f\;\alpha$. This is exactly the "`Functor<F>`" pattern we discussed earlier, expressed in the formal calculus.

17.4 System Fω in Practice

Haskell is the mainstream language closest to System Fω. Its kind system, while implicit in early versions, was made explicit with the `KindSignatures` and `PolyKinds` extensions:

```haskell
-- Haskell infers: Functor :: (Type -> Type) -> Constraint
class Functor (f :: Type -> Type) where
  fmap :: (a -> b) -> f a -> f b

-- Higher-kinded type variable: t has kind (Type -> Type) -> Type -> Type
class MonadTrans (t :: (Type -> Type) -> Type -> Type) where
  lift :: Monad m => m a -> t m a
```

The `MonadTrans` example is striking: `t` is a type constructor that takes a type constructor as an argument and returns a type constructor. Its kind is $(\star \to \star) \to \star \to \star$—a second-order type-level function. This level of abstraction is impossible in System F and is the reason Haskell can define monad transformers generically.

Where System Fω sits

System Fω occupies a sweet spot: it's powerful enough for generic programming abstractions (functors, monads, monad transformers) but still has decidable type checking and

retains strong normalization. It doesn't have dependent types, so you can't express propositions as types—but you also don't need termination proofs or proof obligations. Most working programmers who use advanced generics (Haskell, Scala with higher-kinded types) are operating at roughly the System Fω level.

18 The Lambda Cube

We've now seen three extensions beyond the STLC: System F (terms depend on types), System Fω (types depend on types), and we're about to see dependent types (types depend on terms). Henk Barendregt organized these extensions into a unified framework.

18.1 The Question: What Can Depend on What?

Every type system answers a fundamental question: **what kinds of things can be parameterized by what other kinds of things?** In the STLC, the only abstraction is functions: terms parameterized by terms. System F adds a second kind: terms parameterized by types (polymorphism). But there are two other natural possibilities we haven't explored.

Types parameterized by types (type operators). Consider the type List. By itself, List is not a type—it's a *function on types*: give it Nat and it returns the type $\mathsf{List}\ \mathsf{Nat}$. In Haskell notation, List has "kind" $\star \to \star$—it takes a type and returns a type. This is **higher-kinded polymorphism**, and it's what lets Haskell define `Functor`, `Monad`, and other abstractions that work over *any container type*, not just lists.

Types parameterized by terms (dependent types). The type $\mathsf{Vec}\ n$ depends on a *value* n—a number, not a type. This is a function from terms to types, and it's the most powerful extension.

18.2 Barendregt's Classification

In 1991, Henk Barendregt (the Dutch mathematician who also proved key properties of the untyped λ-calculus and literally wrote the book on it—his 1984 monograph *The Lambda Calculus: Its Syntax and Semantics* is the definitive reference) organized these four kinds of dependency into a three-dimensional cube. Each axis represents one "new" kind of dependency beyond the STLC's terms-to-terms:

Dependency	What it means	System
Terms $\to$ Terms	Ordinary functions (already in STLC)	$\lambda_{\to}$
Types $\to$ Terms	Polymorphism: a term that works at many types	System F ($\lambda 2$)
Types $\to$ Types	Type operators: a type built from other types	$\lambda\underline{\omega}$
Terms $\to$ Types	Dependent types: a type computed from a value	λP

Each corner of the cube enables some subset of these three "extra" axes. The STLC ($\lambda_{\to}$) sits at the origin with none enabled. System F enables one axis. The corner with all three enabled is the **Calculus of Constructions** (CoC), introduced by Thierry Coquand and Gérard Huet in 1988.

18.3 Why a Cube?

The cube structure reveals a beautiful pattern. Each axis is independent: you can add polymorphism without dependent types, or dependent types without type operators. The systems at each corner satisfy a clean compositional property— the features compose without interfering. This is why the cube has $2^3 = 8$ corners, each a coherent type system.

Barendregt's insight was not that these systems existed (most had been studied individually), but that they formed a *unified family* with a common structure. Every system in the cube has the same syntax (variables, abstraction, application), the same computation rule (beta reduction), and the same style of typing rules (contexts, judgments, inference rules). What varies is only *which sorts of things can appear in which positions* in the typing rules.

This is philosophically significant: it suggests that the fundamental notion of "computation by substitution" (beta reduction) is stable, and what changes across type theories is only the question of what we *allow* to be substituted where.

18.4 Types → Types in Practice

The Types → Types axis is the subtlest to understand. Let's make it concrete by comparing what you *can* and *cannot* express in different languages.

What System F gives you: type application. In System F (and TypeScript, Java, Rust, etc.), you can *apply* a type constructor to an argument: `Array<string>`, `List Nat`, `Vec<i32>`. This is just instantiating a type variable. But the type constructor `Array` itself is not a value you can abstract over—it's baked into the definition.

What System F does *not* give you: abstracting over the constructor. In System F, type variables range over *types* like Nat, Bool, List Nat—complete types you could have a value of. But List by itself is not a complete type. It's a *function on types*: give it Nat and it returns List Nat. It has "kind" $\star \to \star$ (a type that takes a type and produces a type). System F has no way to write "for all type-level functions F of kind $\star \to \star$."

Why it matters: the Functor example. In Haskell, which has higher-kinded types, you can write `fmap` once and have it work for *any* container:

```haskell
class Functor f where                    -- f has kind Type -> Type
  fmap :: (a -> b) -> f a -> f b

-- Works for every container:
fmap (+1) [1, 2, 3]                      -- [2, 3, 4]       (f = List)
fmap (+1) (Just 5)                       -- Just 6           (f = Maybe)
fmap show (Node 1 Leaf Leaf)             -- Node "1" ...     (f = Tree)
```

The variable `f` ranges over *type constructors*: `List`, `Maybe`, `Tree`. The type `f a` means "apply whatever container `f` is to the element type `a`." The return type `f b` means "the same container, with transformed contents." The type system tracks which container you started with and guarantees you get the same shape back.

In TypeScript, you cannot express this. A first attempt might try:

```typescript
// NOT VALID TypeScript:
interface Functor<F<_>> {
  map<A, B>(fa: F<A>, f: (a: A) => B): F<B>;
```

```
}
```

The `F<_>` syntax does not exist in TypeScript. You might try a workaround with `extends`:

```
interface Functor<F extends Array<unknown> | Set<unknown>> {
  map<A, B>(fa: F, f: (a: A) => B): ???;
  //                                ^^^
  // Problem: F is already Array<unknown> — a concrete type.
  // We can't "re-open" it and change the inner type from A to B.
  // We can't express "same container, different contents."
}
```

The problem: `Array<unknown>` is a *complete type*. TypeScript has forgotten that it was built by applying the constructor `Array` to something. You can't say "take whatever constructor built `F` and re-apply it to `B`." The constructor is gone—consumed during type application.

What it would look like with higher-kinded types. If TypeScript had the Types → Types axis, you could write:

```
// FANTASY TypeScript with higher-kinded types:
interface Functor<F<_>> {                    // F takes one type param
  map<A, B>(fa: F<A>, f: (a: A) => B): F<B>;
}

const ArrayFunctor: Functor<Array> = {       // Array : Type -> Type
  map: (fa, f) => fa.map(f)
};
const PromiseFunctor: Functor<Promise> = {
  map: (fa, f) => fa.then(f)
};

// Generic over ANY container:
function double<F<_>>(func: Functor<F>, fa: F<number>): F<number> {
  return func.map(fa, x => x * 2);
}

double(ArrayFunctor, [1, 2, 3])       // type: Array<number>
double(PromiseFunctor, fetchCount())  // type: Promise<number>
```

The crucial line is `double<F<_>>`: a function generic over the *container itself*. The return type `F<number>` says "whatever container you gave me, you get the same shape back." Without higher-kinded types, you need a separate `doubleArray`, `doublePromise`, `doubleSet`, and so on—exactly the "code repetition" problem that polymorphism was supposed to solve, but at one level up.

The pattern

Each axis of the Lambda Cube removes one kind of code repetition:

1. **System F** (Types → Terms): Write one function for all *element types*. Instead of `identityNat`, `identityBool`, …, write `identity<T>`.
2. **System Fω** (Types → Types): Write one function for all *container types*. Instead of `mapArray`, `mapPromise`, …, write `map<F>`.

3. **Dependent types** (Terms → Types): Write one type for all *values*. Instead of `Vec3`, `Vec4`, …, write `Vec n`.

18.5 Simulation Is Not the Same as Native Support

A natural reaction to the Lambda Cube is: "But my language *can* express higher-kinded types!" For example, TypeScript's `fp-ts` library simulates Functor using a clever encoding: string literal types serve as names for type constructors, and TypeScript's module augmentation acts as a global registry that maps each name back to its concrete type. The result looks remarkably close to Haskell's `Functor`:

```
// HKT.ts — a "lookup table" from string names to type constructors
interface URI2HKT<A> {}
type Kind<F extends keyof URI2HKT<any>, A> = URI2HKT<A>[F];

// Functor.ts — F is a string key, Kind resolves it
interface Functor1<F extends keyof URI2HKT<any>> {
  map: <A, B>(f: (a: A) => B, fa: Kind<F, A>) => Kind<F, B>;
}

// Option.ts — register Option in the lookup table
declare module './HKT' {
  interface URI2HKT<A> { Option: Option<A>; }
}
```

It works. But it does not make TypeScript a System Fω language. The Lambda Cube boundaries are not about what can be *simulated*—they are about what the type system *natively understands and can reason about*.

The analogy with the untyped calculus. Consider: the untyped λ-calculus can compute anything—it's Turing complete. Church numerals simulate natural numbers, Church booleans simulate booleans, the **Y** combinator simulates recursion. Does that make the untyped calculus "a system with natural numbers and booleans and recursion"? In a sense yes—the encodings work. But in the important sense no: the system doesn't *know* those things are numbers. It can't check that you didn't accidentally add a boolean to a list. The STLC doesn't add any computational power—it adds *knowledge*. The type checker understands what is a number and what is a function, and it can verify that you use each correctly.

The same principle applies at every level of the cube:

Level	What "native support" means
System F	The compiler understands that a type variable ranges over types. It can check universality, infer instances, and optimize based on parametricity.
System Fω	The compiler understands that a type variable ranges over *type constructors*. It can check kind correctness ($\star \to \star$ vs. $\star$), report errors in terms of type constructors, and derive instances.
Dependent types	The compiler understands that types can contain *values*. It can evaluate expressions during type checking, compare types up to computation, and verify proofs.

In the `fp-ts` encoding, TypeScript thinks `F` is a *string*. It has no concept of kinds. It doesn't know the encoding represents a type constructor—it's just checking that a string literal is a valid key in an interface. The *programmer* knows the abstraction is there; the *compiler* doesn't. This means: error messages talk about string keys rather than type constructors; the compiler can't verify functor laws or kind correctness; and optimizations based on parametricity are impossible.

The OOP analogy

You can simulate object-oriented programming in C with structs containing function pointers. The Linux kernel does this extensively. Does that make C an object-oriented language? It can *express* OO patterns, but the language doesn't *understand* them—there's no inheritance checking, no virtual dispatch guarantees, no access control enforced by the compiler. The patterns work because the programmer maintains the discipline, not because the type system enforces it.

Similarly, you can simulate higher-kinded types in TypeScript, or dependent types in Java (with enough generics abuse). The simulation works—but the compiler is enforcing a *different, weaker* set of guarantees than a system that natively supports the feature. The Lambda Cube describes what the type system *understands as first-class concepts*: what it can reason about, check, and enforce—not what clever encodings can approximate.

19 Dependent Types: Types That Compute

19.1 Martin-Löf's Type Theory

Per Martin-Löf, a Swedish logician and philosopher, took the BHK interpretation literally and built a formal system around it. His **intuitionistic type theory** (1971, with major revisions in 1973 and 1984) made the connection between proofs and programs fully precise: each logical connective becomes a type constructor, and each proof rule becomes a typing rule.

In Martin-Löf's system, the type $\Sigma(x{:}A).\, B(x)$ (the dependent pair or "existential") represents "there exists an $x : A$ such that $B(x)$." A term of this type is a *pair* (a, b) where $a : A$ and $b : B(a)$—literally a witness a together with proof b that the witness satisfies the property. You cannot claim existence without producing the thing that exists.

The type $\Pi(x{:}A).\, B(x)$ (the dependent function or "universal") represents "for all $x : A$, $B(x)$." A term of this type is a function that, given any $a : A$, produces a proof of $B(a)$. A "for all" claim must come with a uniform method that works for every input.

This is the BHK interpretation, formalized as a type system:

Logic		**Martin-Löf type theory**
$A \wedge B$	$\longleftrightarrow$	$A \times B$ (product type)
$A \vee B$	$\longleftrightarrow$	$A + B$ (sum type)
$A \Rightarrow B$	$\longleftrightarrow$	$A \to B$ (function type)
$\exists x{:}A.\, B(x)$	$\longleftrightarrow$	$\Sigma(x{:}A).\, B(x)$ (dependent pair)
$\forall x{:}A.\, B(x)$	$\longleftrightarrow$	$\Pi(x{:}A).\, B(x)$ (dependent function)
$\bot$ (false)	$\longleftrightarrow$	Empty type (no constructors)
$\neg A$	$\longleftrightarrow$	$A \to \bot$ (function to empty type)

The last row is illuminating: "not A" is defined as a function from A to the empty type. If you

could produce a value of type A, this function would give you a value of the empty type—which is impossible, since the empty type has no values. So having such a function means A itself must be uninhabited (unprovable). Negation is not a primitive—it's *derived* from functions and the empty type.

19.2 From Philosophy to Software

Martin-Löf's type theory remained a topic in mathematical logic for over a decade. The transition to software came through two parallel efforts.

Coquand and Huet (1985–1988). Thierry Coquand, a French logician (and student of Girard), and Gérard Huet at INRIA developed the **Calculus of Constructions** (CoC)—a type theory that sat at the "top corner" of the Lambda Cube, combining polymorphism, type operators, and dependent types in a single unified system. Their insight was that Martin-Löf's ideas could be organized into a clean, minimalist calculus with just a few rules. The CoC became the theoretical foundation of the **Coq** proof assistant (1989, named as a pun on Coquand's name and the French word for "rooster").

The Automath and NuPRL traditions. Independently, de Bruijn's Automath project (1968, the same de Bruijn who invented de Bruijn indices) had already explored dependent types for formalizing mathematics. Robert Constable's NuPRL system at Cornell (1986) implemented Martin-Löf's type theory directly. These systems demonstrated that dependent types could be made practical, not just theoretical.

Lean 4 (2021, Leonardo de Moura at Microsoft Research) continues this lineage, implementing the **Calculus of Inductive Constructions** (CIC)—the CoC extended with inductive types and a universe hierarchy.

19.3 How Dependent Types Work

In the STLC, types and terms live in separate worlds—a type like $\mathsf{Nat} \to \mathsf{Bool}$ mentions other types but never mentions specific values. With dependent types, the wall comes down.

The classic example is a vector with its length in the type:

$$\mathsf{Vec} : \mathsf{Type} \to \mathsf{Nat} \to \mathsf{Type}$$

$\mathsf{Vec}\ \mathsf{Nat}\ 3$ is the type of lists of exactly three natural numbers. $\mathsf{Vec}\ \mathsf{Bool}\ 0$ is the type of empty boolean lists. The length 3 or 0 is a *value* appearing inside a *type*.

This lets you express invariants that simpler type systems can't:

```
-- Schematic only: `Vec` is not part of core Lean (you would define it
-- yourself, or import Mathlib). We show just the SIGNATURE to make the
-- point; there is no body here, so this is illustrative pseudocode.
def zipWith (f : α → β → γ) : Vec α n → Vec β n → Vec γ n
```

If you try to zip vectors of different lengths, the type checker rejects it at compile time—not at runtime. The n in the type signature is a value (a natural number), not just a type variable.

19.4 The Unification: Types Are Terms

Here is the conceptual leap. In the STLC and System F, there are two separate "worlds": the world of terms (which compute) and the world of types (which classify). These worlds interact through typing judgments ($e : \tau$), but they are fundamentally different kinds of objects.

Dependent types *erase this distinction*. A type is just a term that happens to classify other terms. Nat is a term (of type Type). Vec Nat 3 is a term (built by applying the function Vec to the arguments Nat and 3). Even the function type $A \to B$ is a term (a special case of $\Pi(x{:}A).\, B$ where B doesn't mention x).

This unification means the type checker must be able to *evaluate terms* during type checking. To decide if Vec Nat $(2+1)$ and Vec Nat 3 are the same type, the type checker must compute $2+1$ and see that it equals 3. The type checker is an evaluator. This is why our dependent type checker implementation includes `whnf` (evaluation) and `beq` (comparison after evaluation)—they are not optional features but essential to the core algorithm.

The Curry–Howard payoff

With dependent types, the Curry–Howard correspondence reaches its full potential. A type like $\forall(n : \mathsf{Nat}).\, n + 0 = n$ is simultaneously:

1. A **type** (you can declare variables of this type)
2. A **proposition** ("for all n, $n + 0 = n$")

A term inhabiting this type is simultaneously a **program** and a **proof**. This is exactly how Lean works: when you write `theorem add_zero (n : Nat) : n + 0 = n`, you are defining a term of a dependent type. The proof is a program; the type checker verifies it.

19.5 The Price of Power: Undecidability and Type : Type

Dependent types come with subtleties that simpler systems avoid.

Type checking requires evaluation. Since types can contain arbitrary computation, type checking in a dependently typed system is only decidable if all programs terminate. This is why Lean requires termination proofs for recursive functions (and why you fought with `termination_by` when implementing the interpreter!). If a non-terminating computation could appear in a type, the type checker might loop forever trying to normalize it.

The Type : Type paradox. If `Type` has type `Type`, you get a variant of Russell's paradox. Girard discovered this in 1972 ("Girard's paradox"): in a system with `Type : Type`, you can construct a term of any type, meaning every proposition is "provable"—the system is logically inconsistent. The solution is a **universe hierarchy**: $\mathsf{Type}_0 : \mathsf{Type}_1 : \mathsf{Type}_2 : \ldots$, where each level can only refer to levels below it. Lean implements this with its `Sort` hierarchy. Our toy implementation uses a single `univ` for simplicity, which is why it is not logically consistent—but the core algorithm is the same.

19.6 Implementing a Dependent Type Checker

This is where the magic happens. In a dependently typed system, **types and terms are the same thing**. There is no separate grammar for types—a type is just an expression that happens to classify other expressions. This simplifies the syntax dramatically:

```
-- In dependent types, terms and types are unified.
-- "Type" is itself a term (a universe).
inductive DTerm where
  | bvar   : Nat → DTerm                -- variable (de Bruijn)
  | app    : DTerm → DTerm → DTerm      -- application: e₁ e₂
  | lam    : DTerm → DTerm → DTerm      -- λ(x:A). e  (A is a term!)
  | pi     : DTerm → DTerm → DTerm      -- Π(x:A). B  (dependent function type)
  | univ   : DTerm                       -- Type (the universe)
  deriving Repr, DecidableEq, Inhabited
```

Notice: there is no separate `Ty` type. The `pi` constructor is the dependent function type $\Pi(x{:}A).\,B$, where B can mention x. When B does *not* mention x, this is just the ordinary arrow $A \to B$. And `univ` is the type of types: `Type` itself.

The type checker is more complex than before because it must **evaluate terms during type checking**. When checking whether two types are equal, we can't just compare syntax—we need to beta-reduce them first. For example, $(\lambda x.\,x)$ `Nat` and `Nat` are different expressions but the same type.

```
-- Shifting and substitution (same pattern as before)
def DTerm.shift (d : Int) (c : Nat) : DTerm → DTerm
  | .bvar n    => if n ≥ c then .bvar (Int.toNat (n + d)) else .bvar n
  | .app e₁ e₂ => .app (e₁.shift d c) (e₂.shift d c)
  | .lam ty e  => .lam (ty.shift d c) (e.shift d (c + 1))
  | .pi  ty e  => .pi  (ty.shift d c) (e.shift d (c + 1))
  | .univ      => .univ

-- NOTE: `subst` has signature `(k) (s) : DTerm → DTerm`, so the SUBJECT is
-- the last (pattern-matched) argument. We must call it in prefix form,
-- `DTerm.subst k s subject`: writing `subject.subst k s` (dot notation)
-- would insert `subject` as the *replacement* `s` — the wrong argument —
-- and the recursion would never shrink (Lean: "fail to show termination").
def DTerm.subst (k : Nat) (s : DTerm) : DTerm → DTerm
  | .bvar n    => if n == k then s
                  else if n > k then .bvar (n - 1) else .bvar n
  | .app e₁ e₂ => .app (DTerm.subst k s e₁) (DTerm.subst k s e₂)
  | .lam ty e  => .lam (DTerm.subst k s ty) (DTerm.subst (k+1) (s.shift 1 0) e)
  | .pi  ty e  => .pi  (DTerm.subst k s ty) (DTerm.subst (k+1) (s.shift 1 0) e)
  | .univ      => .univ

-- Beta reduction (needed during type checking!). We must reduce the FUNCTION
-- position first: in `((λ.λ.·) X) Y` the head `(λ.λ.·) X` is itself a redex,
-- so a checker that only fires when the function is *syntactically* a `lam`
-- would leave such applied type families stuck.
partial def DTerm.whnf (t : DTerm) : DTerm :=
  match t with
  | .app f a =>
    match f.whnf with
    | .lam _ body => (DTerm.subst 0 a body).whnf
    | f'          => .app f' a
  | e => e

-- Definitional equality: are two terms the same after reduction?
partial def DTerm.beq (a b : DTerm) : Bool :=
  match a.whnf, b.whnf with
  | .bvar n₁,    .bvar n₂    => n₁ == n₂
  | .app f₁ a₁,  .app f₂ a₂  => f₁.beq f₂ && a₁.beq a₂
  | .lam t₁ e₁,  .lam t₂ e₂  => t₁.beq t₂ && e₁.beq e₂
  | .pi  t₁ e₁,  .pi  t₂ e₂  => t₁.beq t₂ && e₁.beq e₂
  | .univ,        .univ       => true
  | _,            _           => false
```

The `whnf` function (weak head normal form) reduces applications until the top-level structure is

exposed. The `beq` function compares two terms after reduction—this is **definitional equality**, the core of dependent type checking. Two types are considered equal if they reduce to the same thing.

Now the type checker itself. The context maps variable indices to their types (which are themselves `DTerm`s):

```
partial def dTypeCheck (ctx : List DTerm) : DTerm → Option DTerm
  -- Variable: look up its type in the context, then SHIFT it into the
  -- current scope. The annotation was stored under fewer binders than are
  -- now in scope, so a variable at index n needs its looked-up type shifted
  -- up by n+1 (past all the binders crossed since it was recorded).
  | .bvar n    => (ctx[n]?).map (·.shift (Int.ofNat (n+1)) 0)
  -- Universe: Type has type Type (simplified; real systems use a hierarchy)
  | .univ      => some .univ
  -- Pi type: Π(x:A).B is valid if A : Type and B : Type (with x:A in scope)
  | .pi a b    => do
    let aTy ← dTypeCheck ctx a
    guard (aTy.whnf matches .univ)          -- A must be a type
    let bTy ← dTypeCheck (a :: ctx) b
    guard (bTy.whnf matches .univ)          -- B must be a type
    some .univ                               -- then Π(x:A).B : Type
  -- Lambda: λ(x:A).e has type Π(x:A).B if e:B with x:A in scope
  | .lam a body => do
    let aTy ← dTypeCheck ctx a
    guard (aTy.whnf matches .univ)
    let bodyTy ← dTypeCheck (a :: ctx) body
    some (.pi a bodyTy)
  -- Application: (e₁ e₂) — e₁ must have a Π type, e₂ must match the domain
  | .app e₁ e₂ => do
    let funTy ← dTypeCheck ctx e₁
    match funTy.whnf with
    | .pi dom cod =>
      let argTy ← dTypeCheck ctx e₂
      guard (dom.beq argTy)                  -- argument type must match domain
      some (DTerm.subst 0 e₂ cod)            -- CRUCIAL: substitute the actual
                                             -- argument into the codomain!
    | _ => none
```

The magic line

The most important line in the entire type checker is in the application case: `DTerm.subst 0 e₂ cod` (substitute the argument `e₂` for index 0 in the codomain `cod`). This is where dependent types earn their name. In the STLC, applying $f : A \to B$ to $x : A$ gives type B—the return type is fixed. But in a dependent system, applying $f : \Pi(x{:}A).\, B$ to $a : A$ gives type $B[x := a]$—the return type *depends on the argument*. This single substitution is what makes dependent types able to express "the output list has the same length as the input" or "this function returns a proof that its output is sorted."

Let's test it. We can encode the STLC's identity function and verify it:

```
-- λ(A : Type). λ(x : A). x  :  Π(A : Type). Π(_ : A). A
-- This is the dependently-typed polymorphic identity!
```

```
def polyId : DTerm :=
  .lam .univ (.lam (.bvar 0) (.bvar 0))

#eval dTypeCheck [] polyId
-- Result: some (DTerm.pi DTerm.univ (DTerm.pi (DTerm.bvar 0) (DTerm.bvar 1)))
-- Which reads: Π(A : Type). Π(x : A). A    ✓
```

Why λ and Π look so similar (but aren't). The term and its type look almost identical, which can be confusing:

Term (the function)	$\lambda(A : \mathsf{Type}).\, \lambda(x : A).\, x$	"given A and x, return x"
Type (the signature)	$\Pi(A : \mathsf{Type}).\, \Pi(\_ : A).\, A$	"takes a type A and an A, returns an A"

λ *builds* a function. Π *describes* what kind of function it is. Every λ has a Π as its type, the same way every function in a programming language has a type signature. In everyday code, the distinction is obvious:

```
// The FUNCTION (λ):    (x: number) => x + 1
// The TYPE (Π):        (x: number) => number
```

They look similar because they both mention the parameter name and type. But one is the *code* (what to compute) and the other is the *contract* (what goes in and what comes out).

Why the de Bruijn indices differ. In the *term* `.lam .univ (.lam (.bvar 0) (.bvar 0))`, both uses of `.bvar 0` mean "the nearest enclosing binder":

```
λ(A : Type). λ(x : A). x
↑ binder 1    ↑ binder 0  ↑ bvar 0 = nearest binder = x  ✓
              ↑ the annotation A here is (.bvar 0),
                because at this point only the outer λ is in scope,
                so A (the outer λ's variable) is index 0
```

But in the *type* `DTerm.pi .univ (DTerm.pi (.bvar 0) (.bvar 1))`, the final `.bvar 1` reaches past the inner binder:

```
Π(A : Type). Π(_ : A).  A
↑ binder 1    ↑ binder 0  ↑ bvar 1 = skip inner binder,
                             grab outer binder = A  ✓
              ↑ (.bvar 0) = A, because only the outer Π is in scope here
```

The term has `.bvar 0` at the end because its final variable is x (the *inner* λ's variable—the nearest binder). The type has `.bvar 1` at the end because its return type is A (the *outer* Π's variable), and from inside two nested Π binders, the outer one is at index 1.

20 Type Checking Is Proof Checking

Here is the punchline. In a dependently typed system, propositions are types and proofs are terms. So **checking a proof is just type checking a term**. There is no separate proof-verification engine—the type checker *is* the proof verifier. Let's see this in action.

We can encode simple logical connectives as types. The proposition "A implies B" is the function type $A \to B$ (a non-dependent Pi type). The proposition "for all $x : A$, $P(x)$" is $\Pi(x : A).\, P\ x$ (a dependent Pi type). A *proof* of these propositions is a term that inhabits the type.

```
-- Proposition: A → A  ("A implies A")
-- Proof: λ(A : Type). λ(a : A). a  (the identity function!)
-- Type checker confirms: this term has type Π(A:Type). A → A  ✓
-- Therefore: the proposition "A implies A" is proved.

-- Proposition: A → B → A  ("A implies (B implies A)")
-- Proof: λ(A:Type). λ(B:Type). λ(a:A). λ(b:B). a
def kProof : DTerm :=
  .lam .univ (.lam .univ (.lam (.bvar 1) (.lam (.bvar 1) (.bvar 1))))

-- The type checker returns: Π(A:Type). Π(B:Type). Π(_:A). Π(_:B). A
-- Which IS the proposition A → B → A.
-- The fact that type checking succeeded IS the proof verification.
#eval dTypeCheck [] kProof
-- some (DTerm.pi DTerm.univ (DTerm.pi DTerm.univ
--        (DTerm.pi (DTerm.bvar 1) (DTerm.pi (DTerm.bvar 1) (DTerm.bvar 3)))))
-- The final DTerm.bvar 3 reaches past the two inner Π binders to A.  ✓
```

The key insight: when the type checker accepts a term e at type τ, it has verified that e is a valid proof of the proposition τ. When it rejects a term, it means the "proof" is invalid. There is nothing else to check—**type safety is logical consistency**.

This is exactly what happens inside Lean when you write:

```
-- Lean's theorem keyword is just a "def" checked by this same machinery
theorem identity (A : Prop) (a : A) : A := a

-- Lean's type checker verifies:
--   a : A       (variable lookup)
--   λa. a : A → A  (Abs rule)
--   etc.
-- If it type-checks, the proof is valid. Period.
```

The entire 4539-line Lean kernel that verifies every proof in Lean's `mathlib` library works on essentially this principle: terms are proofs, types are propositions, and type checking is proof verification. What we've built above is a toy version of the same idea—simplified, but the core mechanism is identical.

21 Parsing Lambda Calculus

21.1 What Parsing Is

Every programming language exists in two forms. The **concrete syntax** is what humans write: `\x. x y`. The **abstract syntax** is what the machine manipulates: `lam "x" (app (var "x") (var "y"))`. A **parser** bridges these: it takes a string of characters and produces a tree.

Parsing is one of the oldest problems in computer science. The theory was developed in the 1950s–1970s alongside formal language theory, and it splits into two parts:

Lexing (tokenization): splitting a character stream into tokens. The string `\x␣y.␣x␣y` becomes tokens [`LAMBDA`, `ID`(x), `ID`(y), `DOT`, `ID`(x), `ID`(y)]. Whitespace is consumed; meaningful chunks are identified. For our simple grammar, we'll do this inline rather than as a separate pass.

Parsing (syntax analysis): organizing tokens into a tree according to a grammar. The token stream above becomes `lam`("x", `lam`("y", `app`(`var`("x"), `var`("y")))).

Context-free grammars. A grammar specifies which strings are valid programs. The standard formalism is a **context-free grammar** (CFG), defined by Noam Chomsky in 1956 (originally for natural languages, but quickly adopted by computer scientists). A CFG consists of:

1. **Terminals**: literal characters or tokens (`\`, `.`, identifiers)
2. **Nonterminals**: abstract categories (*expr*, *app*, *atom*)
3. **Production rules**: how nonterminals expand into sequences of terminals and nonterminals
4. A **start symbol**: the nonterminal that represents a complete program

Two major approaches exist for building a parser from a grammar: **parser generators** (tools like yacc, ANTLR, or Menhir that read a grammar file and output parser code) and **hand-written parsers** (recursive descent or parser combinators). We'll use the hand-written approach because it's more instructive—you see every decision the parser makes.

Grammar design

21.1. What syntax would feel natural for writing λ-terms? Think about: how do you write a lambda? How do you group things? What does application look like? Write out a grammar (like the BNF from Section 4) for your syntax.

21.2 The Grammar

Here is one clean design:

```
expr   ::= '\' ident+ '.' expr    -- lambda: \x y z. body
         | app
app    ::= atom+                  -- one or more atoms, left-assoc
atom   ::= '(' expr ')'           -- grouped expression
         | ident                   -- variable name
ident  ::= [a-zA-Z_][a-zA-Z0-9_']*
```

Multi-argument lambdas (`\x y. e` desugars to nested λ's), left-associative application, and parentheses for grouping. This grammar is simple enough to parse without any special tools—no need for yacc, ANTLR, or lex. Two approaches work cleanly: recursive descent and parser combinators.

21.3 Approach 1: Recursive Descent

The simplest technique is **recursive descent**: each grammar rule becomes a function, and the functions call each other recursively. The structure of the code mirrors the structure of the grammar.

Parser design

21.2. The `parseExpr` function must decide: is this a lambda or an application? What character tells you which one? The `parseApp` function must handle left-associative application: `f x y` means `(f x) y`. How would you accumulate applications left-to-right?

Think about `parseExpr` first. You're looking at the input and need to decide what you're about to parse. If the next character is `\`, you know it's a lambda. Otherwise, it must be an application (a sequence of atoms). That's it—just one character of lookahead tells you which branch to take.

Now think about `parseApp`. Application syntax is just *juxtaposition*: `f x y` means "apply f to x, then apply the result to y." This is left-associative: $f\ x\ y = (f\ x)\ y$. So you parse the first atom, then keep parsing more atoms and wrapping them in left-associative `app` nodes. If you start with f, then read x, you have $\texttt{app}(f, x)$. Then read y: $\texttt{app}(\texttt{app}(f, x), y)$. Stop when there are no more atoms.

Finally, `parseAtom` handles the two base cases: either a parenthesized expression (start with `(`, parse recursively, expect `)`) or a bare identifier.

```
-- Dispatch: lambda or application?
partial def parseExpr (s : PState) : ParseResult Term :=
  match peek s with
  | some '\\' => parseLambda s     -- starts with \: must be a lambda
  | _         => parseApp s        -- otherwise: try application

-- Lambda: consume \, collect parameter names, consume ., parse body
-- Fold right: [x, y, z] → lam x (lam y (lam z body))
partial def parseLambda (s : PState) : ParseResult Term := ...

-- Application: parse atoms left-to-right, accumulate with app
partial def parseApp (s : PState) : ParseResult Term :=
  match parseAtom s with
  | some (first, s) => go s first   -- got one atom, try for more
  | none => none
  where go s acc := match parseAtom s with
    | some (next, s') => go s' (.app acc next)  -- left-associative!
    | none => some (acc, s)

-- Atom: parenthesized expression or identifier
partial def parseAtom (s : PState) : ParseResult Term := ...
```

This works well for our simple grammar. But notice the manual threading of `PState` through every function—you pass it in, get an updated state back, pass that to the next function. If parsing fails, you return `none` and the caller must handle it. This "plumbing" obscures the structure. As grammars grow more complex, the plumbing dominates the code.

21.4 Approach 2: Parser Combinators (The Parsec Idea)

Parser combinators solve the plumbing problem by treating parsers as *values*—first-class objects that can be combined with operators, just like numbers combine with $+$ and $\times$. The idea was pioneered by Philip Wadler (1985), developed by Graham Hutton and Erik Meijer (1996), and brought to practical maturity by Daan Leijen's **Parsec** library for Haskell (2001).

21.4.1 The Core Abstraction

A parser is a function: it takes an input string and a position, and either succeeds (returning a result and the new position) or fails:

```
-- A parser that produces values of type α
def Parser (α : Type) := String → Nat → Option (α × Nat)
-- Input string → position → maybe (result, new position)
```

This is a simple type, but it encodes a powerful idea: a parser is a *computation with two effects*—it consumes input (state) and it can fail (partiality). The entire Parsec approach comes from combining these computations with a small set of operators.

21.4.2 Primitive Combinators

We start with combinators that do almost nothing by themselves:

```
-- `pure a`: always succeeds, produces `a`, consumes nothing
def pure (a : α) : Parser α := fun s pos => some (a, pos)

-- `fail`: always fails
def fail : Parser α := fun _ _ => none

-- `char c`: succeeds if the next character is `c`
def char (c : Char) : Parser Char := fun s pos =>
  if pos < s.length && s.get ⟨pos⟩ == c then some (c, pos + 1) else none

-- `satisfy p`: succeeds if the next character satisfies predicate `p`
def satisfy (p : Char → Bool) : Parser Char := fun s pos =>
  if pos < s.length then
    let c := s.get ⟨pos⟩
    if p c then some (c, pos + 1) else none
  else none
```

Always bounds-check the position

Both combinators test `pos < s.length` *before* reading. This is not optional. In current Lean, `s.get ⟨pos⟩` past the end of the string returns the *default* character `'A'` rather than failing—and `'A'.isAlphanum` is `true`, so an unguarded `many (satisfy Char.isAlphanum)` would "succeed" forever and loop until the stack blows ("deep recursion"). The recursive-descent parser guards its reads the same way; combinators must too.

Each one is tiny. The power comes from *composition*.

21.4.3 Sequential Composition: Bind

The most important combinator is `bind`: run one parser, feed its result to a function that returns another parser, then run that:

```
-- `p >>= f`: run p, feed its result to f, run the resulting parser
def bind (p : Parser α) (f : α → Parser β) : Parser β := fun s pos =>
  match p s pos with
  | some (a, pos') => f a s pos'    -- p succeeded: continue with f
```

```
  | none           => none          -- p failed: propagate failure
```

This is *monadic sequencing*. All the state-threading plumbing we did manually in recursive descent is now hidden inside `bind`. Each parser receives the updated position automatically.

21.4.4 Choice: Or-Else

```
-- `p <|> q`: try p; if it fails, try q (backtracking)
def orElse (p q : Parser α) : Parser α := fun s pos =>
  match p s pos with
  | some r => some r        -- p succeeded
  | none   => q s pos       -- p failed: try q at the same position
```

This is **backtracking**: if p fails, we rewind to the original position and try q. The parser explores alternatives without the programmer manually managing the position.

Reading order vs. compilation order

The four functions above (`pure`, `fail`, `bind`, `orElse`) are shown as *standalone* definitions to demystify what the operators do. In the finished parser we *drop* these standalone `def`s and instead register the `Functor`/`Applicative`/`Monad`/`Alternative` instances (a few subsections down)—those instances are what make `>>=`, `do`-notation, and `<|>` available. Lean resolves those operators only *after* the instances are in scope, so in an actual source file the instance block must come *before* the combinators that use them (`many1`, `spaces`, `token`, `ident`, `lamP`, …). We present them in conceptual order here for readability; the compile-ready ordering is: instances first, then the repetition/whitespace helpers, then the grammar (in a `mutual` block).

21.4.5 Repetition

```
-- `many p`: apply p zero or more times, collect results
partial def many (p : Parser α) : Parser (List α) := fun s pos =>
  match p s pos with
  | none         => some ([], pos)
  | some (a, pos') =>
    match many p s pos' with
    | some (rest, pos'') => some (a :: rest, pos'')
    | none               => some ([a], pos')

-- `many1 p`: one or more (fails if p doesn't match at least once)
def many1 (p : Parser α) : Parser (List α) :=
  p >>= fun first =>
  many p >>= fun rest =>
  pure (first :: rest)
```

21.4.6 Whitespace and Tokens

Real parsers need to skip whitespace between tokens:

```
-- Skip whitespace
def spaces : Parser Unit :=
  many (satisfy Char.isWhitespace) >>= fun _ => pure ()
```

```
-- Parse p, then skip trailing whitespace
def token (p : Parser α) : Parser α :=
  p >>= fun a =>
  spaces >>= fun _ =>
  pure a

-- Parse a specific character, skip trailing whitespace
def symbol (c : Char) : Parser Char := token (char c)
```

The `token` combinator is a small but important pattern: it wraps any parser to consume trailing whitespace, so the rest of your grammar never mentions whitespace explicitly.

21.4.7 Parsers as Functor, Applicative, and Monad

In Haskell (and real Parsec implementations), parsers aren't just used with bare functions like `bind` and `pure`. They implement the **typeclass hierarchy**: Functor, Applicative, and Monad. This unlocks powerful syntax and generic combinators.

```
-- Functor: transform the result of a parser
instance : Functor Parser where
  map f p := fun s pos =>
    match p s pos with
    | some (a, pos') => some (f a, pos')
    | none           => none

-- Applicative: run two parsers, combine their results
instance : Applicative Parser where
  pure a := fun s pos => some (a, pos)
  seq pf pa := fun s pos =>
    match pf s pos with
    | some (f, pos') =>
      match pa () s pos' with
      | some (a, pos'') => some (f a, pos'')
      | none            => none
    | none => none

-- Monad: sequence parsers where the second depends on the first's result
instance : Monad Parser where
  bind p f := fun s pos =>
    match p s pos with
    | some (a, pos') => f a s pos'
    | none           => none

-- Alternative: choice between parsers
instance : Alternative Parser where
  failure := fun _ _ => none
  orElse p q := fun s pos =>
    match p s pos with
    | some r => some r
    | none   => q () s pos
```

Why does the typeclass hierarchy matter? Because each level gives you *different combinators for free*:

Functor gives you `<$>` (map): transform a parser's result without changing what it consumes. If `digit` parses a character, `Char.toNat <$> digit` parses a character and converts it to a number.

Applicative gives you `<*>` (apply): run two parsers in sequence and combine their results. This is enough for parsers that don't need to inspect intermediate results:

```
-- Parse a pair: (expr, expr)
def pairP : Parser (Term × Term) :=
  (fun _ a _ b _ => (a, b))          -- how to combine results
    <$> symbol '(' <*> exprP         -- ( expr
    <*> symbol ',' <*> exprP         -- , expr
    <*> symbol ')'                   -- )
```

Monad gives you `>>=` (bind) and, crucially, `do` notation. This is the most common style in practice because it reads like imperative code:

```
-- The lambda parser in do-notation:
def lamP : Parser Term := do
  let _ ← symbol '\\'
  let params ← many1 ident
  let _ ← symbol '.'
  let body ← exprP
  pure (params.foldr Term.lam body)

-- Compare to the >>= version:
def lamP' : Parser Term :=
  symbol '\\' >>= fun _ =>
  many1 ident >>= fun params =>
  symbol '.'  >>= fun _ =>
  exprP       >>= fun body =>
  pure (params.foldr Term.lam body)
```

The two are identical—`do` notation is syntactic sugar for chains of `>>=`. But the `do` version is more readable, especially for longer parsers. This is what real Parsec code looks like.

Applicative vs. Monad. Why have both? Applicative parsing is *less powerful*—the structure of the parse is fixed before any input is consumed. Monadic parsing lets the second parser *depend on the result* of the first (e.g., "if we parsed a keyword `let`, now parse a binding; if we parsed `if`, now parse a condition"). But applicative parsing is easier to analyze statically, which some parser libraries exploit for optimization. Most practical parsers use monadic combinators because the expressiveness is essential.

Parsers are monads

Look at the types: `Parser α` wraps a computation that might fail. `pure` lifts a value into the parser. `bind` sequences two parsers. This is exactly the monad pattern! Parsec's insight was that monadic composition gives you the right abstraction for parsing: sequencing (`>>=`), choice (`<|>`), and failure propagation—all handled automatically.

If you've used JavaScript Promises, you've used the same pattern: `fetch(url).then(parse).then(validate)` sequences computations that might fail,

just like `parseExpr >>= parseDot >>= parseBody`.

21.4.8 Building Our Lambda Parser

With these building blocks, the parser mirrors the grammar almost directly. Now that we have `do` notation, let's use it:

```
-- Parse an identifier: one or more alphanumeric/underscore characters
def ident : Parser String := token do
  let chars ← many1 (satisfy fun c => c.isAlphanum || c == '_' || c == '\'')
  pure (String.mk chars)

-- Parse a keyword (an identifier matching a specific string)
def keyword (s : String) : Parser Unit := do
  let name ← ident
  if name == s then pure () else failure

-- These four parsers are MUTUALLY recursive (lamP calls exprP, exprP calls
-- appP, appP calls atomP, atomP calls exprP), so they must be wrapped in a
-- `mutual ... end` block; otherwise Lean reports "unknown identifier exprP".
mutual
-- Parse a lambda: '\' ident+ '.' expr
partial def lamP : Parser Term := do
  let _ ← symbol '\\'
  let params ← many1 ident       -- collect parameter names
  let _ ← symbol '.'
  let body ← exprP               -- parse the body
  pure (params.foldr Term.lam body) -- desugar: [x,y] → lam x (lam y body)

-- Parse an application: atom+, left-associative
partial def appP : Parser Term := do
  let atoms ← many1 atomP
  -- `many1` guarantees at least one atom, but Lean can't see that, so
  -- `atoms.head` would demand a nonemptiness proof. `head!` uses the
  -- `Inhabited Term` instance for the (unreachable) empty case.
  pure (atoms.tail.foldl Term.app atoms.head!)

-- Parse an atom: parenthesized expression or identifier
partial def atomP : Parser Term :=
  (do let _ ← symbol '('; let e ← exprP; let _ ← symbol ')'; pure e)
  <|> (do let name ← ident; pure (Term.var name))

-- Parse an expression: lambda or application
partial def exprP : Parser Term := lamP <|> appP
end
```

Compare this to the recursive descent version. The manual state threading is gone—`bind` handles it. Error propagation is gone—`Option` handles it. What remains is just the grammar, expressed as composition of small parsers. The code *reads like* the grammar.

21.4.9 Backtracking and Committed Choice

Our simple `<|>` always backtracks on failure. Parsec introduced a crucial refinement: **committed choice**. Once a parser has consumed input successfully, it *commits* to that branch—it won't backtrack. This gives two benefits:

Better error messages. If you write `\x. y` and the parser successfully consumes the `\` and `x` but fails on what comes next, you want "expected `.` at position 3," not "expected identifier" (which is what you'd get if it backtracked and tried the `app` branch instead).

Better performance. Without committed choice, the parser might try exponentially many alternatives. With it, once any input is consumed, the decision is final—no more alternatives are explored.

Parsec provides a `try` combinator that explicitly enables backtracking for cases where you need it:

```
-- `try p`: run p, but if it fails AFTER consuming input, backtrack
-- (our simple version always backtracks; Parsec only backtracks with `try`)
def try_ (p : Parser α) : Parser α := fun s pos =>
  match p s pos with
  | some r => some r
  | none   => none   -- reset to original position (in Parsec, this matters)
```

The rule of thumb: if two alternatives start with the same token, wrap the first in `try`. Otherwise, let committed choice give you speed and error quality for free.

21.4.10 From Wadler to Parsec: The History

Parser combinators have a rich intellectual history:

Wadler (1985) showed that recursive descent parsers can be expressed as higher-order functions in a lazy functional language, and that monadic sequencing captures the essence of sequential parsing.

Hutton and Meijer (1996) wrote the influential paper "Monadic Parser Combinators," which crystallized the approach: a parser is a monad, and the monad interface (`return`, `>>=`) gives you sequencing for free.

Leijen (2001) created Parsec, which solved the two practical problems that had kept combinators out of production use: error messages (by tracking source positions and "expected" labels) and performance (by introducing committed choice). Parsec demonstrated that combinator parsers could handle real programming languages like Haskell itself.

The idea spread everywhere: Scala's `scala-parser-combinators`, Rust's `nom` and `combine` crates, Python's `parsy`, and JavaScript's `parsimmon` all follow the Parsec pattern. Lean itself has a sophisticated parser combinator library built into its front end. The idea of "small composable units combined with operators" is one of functional programming's most successful exports to mainstream software engineering.

21.5 Named-to-De-Bruijn Conversion

Once we have a named `Term`, we convert to de Bruijn. The key: maintain a list of names where each name's *position* is its de Bruijn index.

Conversion challenge

21.3. If the context is `["x", "y"]` (meaning x was bound most recently, y before that), what de Bruijn index should x get? What about y?
Answer: x is at position 0, y at position 1.

```
def toDeBruijn (ctx : List String := []) : Term → Option Expr
  | Term.var x =>
    match ctx.idxOf? x with
    | some i => some (Expr.bvar i)    -- position IS the index
    | none   => none                  -- unbound variable
  | Term.lam x body =>
    match toDeBruijn (x :: ctx) body with   -- push x at index 0
    | some e => some (Expr.lam e)
    | none   => none
  | Term.app e₁ e₂ => do
    let e₁' ← toDeBruijn ctx e₁
    let e₂' ← toDeBruijn ctx e₂
    some (Expr.app e₁' e₂')
```

21.6 A Definitions Environment

Typing Church encodings by hand is painful. We add a definitions map: when the parser sees a known name like `succ` or `true`, it expands it to the corresponding λ-term. This is a simple tree-walk replacement (respecting shadowing). See Appendix E for the full `expandDefs` and `churchDefs` implementations.

22 Putting It All Together: The REPL

22.1 What a REPL Is

REPL stands for **Read–Eval–Print Loop**. The idea is ancient—Lisp had one in 1958—and simple: read user input, evaluate it, print the result, repeat. Every interactive language environment uses this pattern: Python's `>>>` prompt, Node.js's console, GHCi for Haskell, and Lean's `#eval`.

The name describes the four phases precisely:

1. **Read**: Accept a line of text from the user.
2. **Eval**: Parse it, transform it, and evaluate it.
3. **Print**: Display the result (or an error message).
4. **Loop**: Go back to step 1.

What makes a REPL more than just "run code and show output" is the *loop*: state accumulates across iterations. Definitions made in one line are available in the next. This turns a REPL from a calculator into an interactive environment for exploration.

22.2 The Pipeline

Our REPL ties every component we've built into a single pipeline. Each arrow represents a function we implemented in an earlier section:

$$\texttt{string} \xrightarrow{\text{parse}} \texttt{Term} \xrightarrow{\text{expand}} \texttt{Term} \xrightarrow{\text{toDeBruijn}} \texttt{Expr} \xrightarrow{\text{eval}} \texttt{Expr} \xrightarrow{\text{display}} \texttt{string}$$

Stage	What it does	Section
Parse	String → named AST	Section 21
Expand	Substitute known definitions	Section 21
To de Bruijn	Named → de Bruijn indices	Section 11
Eval	Beta-reduce to normal form	Section 7
Display	De Bruijn AST → readable string	—

The pipeline function itself is a composition of these stages, where each stage can fail:

```
def pipeline (defs : Defs) (input : String) : Option String := do
  let named    ← parse input           -- stage 1: parse
  let expanded := expandDefs defs named -- stage 2: expand definitions
  let expr     ← toDeBruijn [] expanded -- stage 3: named → de Bruijn
  let result   := eval 1000 expr       -- stage 4: evaluate (with fuel)
  pure (display result)                -- stage 5: display
```

Notice the `do` notation—this is the same monadic sequencing we saw in the parser combinators. The `←` operator extracts a value from an `Option`, and if any step returns `none`, the whole pipeline short-circuits and returns `none`. Monads everywhere.

If any stage fails (parse error, unbound variable), the REPL reports the error and waits for the next input—it never crashes.

22.3 State and Commands

Beyond evaluating expressions, our REPL supports commands. The key design decision: `defs` is threaded through the loop as state. When you `:let two = succ (succ zero)`, the name `two` is added to `defs` and available in all subsequent inputs.

```
-- The REPL loop
partial def repl (defs : Defs) : IO Unit := do
  IO.print "λ> "
  let input ← (← IO.getStdin).getLine
  let input := input.trim
  match input with
  | ":quit"  => pure ()
  | ":defs"  => printDefs defs; repl defs
  | _        =>
    if input.startsWith ":let " then
      -- :let name = expr — add a new definition
      match parseLetBinding input with
      | some (name, term) =>
        IO.println s!"{name} defined"
        repl (defs.insert name term)
      | none => IO.println "Parse error"; repl defs
    else
      -- Evaluate an expression
      match pipeline defs input with
      | some result => IO.println s!"Result:    {result}"
      | none        => IO.println "Error: parse or conversion failed"
```

```
      repl defs
```

This is a **functional loop**: there is no mutable state. Instead, the updated `defs` map is passed as an argument to the next recursive call. Each iteration of the REPL is a fresh function call with potentially different state. This pattern—looping via recursion, state via arguments—is the functional alternative to `while` loops with mutable variables.

These two listings are illustrative sketches

The `pipeline` and `repl` listings above are written for readability, using suggestive names—`Defs`, `display`, `printDefs`, `parseLetBinding`, `defs.insert`—that we have *not* defined. They stand in for concrete pieces that the runnable implementation spells out differently. In the complete, compile-checked code of Appendix E: `Defs` is simply `List (String × Term)`; "display" is the `ToString` instance (so `s!"{result}"`); "printDefs" is a `for` loop over the list; "parseLetBinding" is `rest.splitOn " = "`; and "`defs.insert name term`" is just consing `(name, term) :: defs`. Read this section for the *shape* of the pipeline; read the appendix for code you can paste and run.

22.4 Example Session

```
λ> \x. x
Parsed:    (λx. x)
De Bruijn: (λ. 0)
Result:    (λ. 0)

λ> succ (succ (succ zero))
Result:    (λ. (λ. (1 (1 (1 0)))))

λ> add (succ (succ zero)) (succ (succ (succ zero)))
Result:    (λ. (λ. (1 (1 (1 (1 (1 0)))))))

λ> :let two = succ (succ zero)
two defined

λ> mul two (succ two)
Result:    (λ. (λ. (1 (1 (1 (1 (1 (1 0))))))))

λ> and true false
Result:    (λ. (λ. 0))

λ> :let apply_twice = \f x. f (f x)
apply_twice defined

λ> apply_twice succ zero
Result:    (λ. (λ. (1 (1 0))))
```

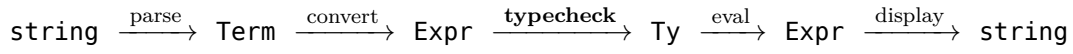
That last result—$(\lambda.\,\lambda.\,1\ (1\ 0))$—is the Church numeral $\overline{2}$: applying successor twice to zero gives 2. (Recall that the numeral $\overline{n}$ is $\lambda f.\,\lambda x.\,f^n\,x$; in de Bruijn form the bound f is index 1 and x is index 0, so the applied head is 1 and the innermost argument is 0.)

22.5 Extending to a Typed REPL

Our REPL evaluates untyped lambda terms. But the components are modular enough to extend. Let's sketch how to add each feature from the type system sections.

22.5.1 Adding Type Checking

Insert a type-checking stage between conversion and evaluation. The pipeline becomes:




```
def typedPipeline (defs : Defs) (input : String) : Option String := do
  let named    ← parse input
  let expanded := expandDefs defs named
  let expr     ← toDeBruijn [] expanded
  let ty       ← typeCheck [] expr      -- NEW: reject ill-typed terms
  let result   := eval 1000 expr
  pure s!"{display result} : {displayTy ty}"
```

Now `(\x. x x)` is rejected before evaluation—exactly what the STLC was designed to do.

Schematic sketch — not runnable as written

This `typedPipeline` is deliberately schematic. As written it would *not* type-check: `toDeBruijn` produces the *untyped* `Expr`, but `typeCheck` expects the *annotated* `TExpr` (a lambda carries its parameter type). A real typed pipeline needs an annotated front end all the way through—a parser that reads $\lambda(x{:}T).\, e$, a named-to-index conversion for the annotated AST, and only then `typeCheck`. The next subsections build exactly that annotated grammar; the point here is the *shape* of the extended pipeline, not a drop-in definition.

22.5.2 Adding Type Annotations

To type-check lambda terms, the type checker needs to know the types of parameters. We extend both the AST and the parser.

Extending the AST. The named term type gains an optional type annotation on lambdas:

```
inductive Ty where
  | nat  : Ty
  | bool : Ty
  | arr  : Ty → Ty → Ty     -- function type: Ty → Ty
  deriving Repr

inductive Term where
  | var : String → Term
  | lam : String → Ty → Term → Term     -- now: λ(x : T). body
  | app : Term → Term → Term
```

Parsing types. The type grammar is its own small parser. Function arrows are *right*-associative ($A \to B \to C$ means $A \to (B \to C)$):

```
-- Parse a base type: Nat, Bool, or (type)
partial def atomTyP : Parser Ty :=
```

```
  (do keyword "Nat"; pure .nat)
  <|> (do keyword "Bool"; pure .bool)
  <|> (do let _ ← symbol '('; let t ← typeP; let _ ← symbol ')'; pure t)

-- Parse a function type: base -> base -> ... (right-associative)
partial def typeP : Parser Ty := do
  let lhs ← atomTyP
  (do let _ ← token (char '-' >>= fun _ => char '>') -- parse "->"
      let rhs ← typeP                                 -- right-recursive!
      pure (.arr lhs rhs))
  <|> pure lhs
```

Parsing typed lambdas. The lambda parser now expects a colon and type annotation after each parameter:

```
-- Parse one typed parameter: ident : type
partial def typedParam : Parser (String × Ty) := do
  let name ← ident
  let _ ← symbol ':'
  let ty ← typeP
  pure (name, ty)

-- Parse a typed lambda: \(x : Nat)(y : Bool). body
-- Each parameter is parenthesized with its type.
-- (`typedLamP` is mutually recursive with the typed analogues of appP/atomP/
--  exprP — call them tappP/tatomP/texprP — and would sit in a `mutual` block
--  with them, exactly like the untyped parser. We show only typedLamP here.)
partial def typedLamP : Parser Term := do
  let _ ← symbol '\\'
  let params ← many1 (do
    let _ ← symbol '('
    let p ← typedParam
    let _ ← symbol ')'
    pure p)
  let _ ← symbol '.'
  let body ← texprP          -- the typed expression parser (typed analogue of exprP)
  pure (params.foldr (fun (name, ty) acc => .lam name ty acc) body)
```

Example. With this grammar, the identity function on naturals is:

```
λ> \(x : Nat). x
Parsed:    λ(x : Nat). x
Type:      Nat → Nat
Result:    (λ. 0)

λ> \(f : Nat -> Nat)(x : Nat). f x
Parsed:    λ(f : Nat → Nat). λ(x : Nat). f x
Type:      (Nat → Nat) → Nat → Nat
Result:    (λ. (λ. (1 0)))
```

The pipeline now produces typed output. If you try to apply a boolean to a function expecting

a natural, the type checker rejects it before evaluation even begins.

22.5.3 Adding Commands

A production REPL supports multiple commands:

Command	What it does
`:type expr`	Show the type without evaluating (like GHCi's `:t`)
`:let name = expr`	Bind a name to a term
`:step expr`	Show each reduction step, not just the result
`:debruijn expr`	Show the de Bruijn representation
`:defs`	List all current definitions
`:quit`	Exit

Each command is just a different path through the pipeline. `:type` runs parse → convert → typecheck and stops. `:step` runs parse → convert → eval-with-tracing, printing each intermediate form.

What you've built

Starting from three constructs (variable, lambda, application) and one rule (beta reduction), you've built a system that can do arithmetic, logic, recursion, and more—all from functions. The interpreter follows the exact same structure as industrial implementations of functional languages, just without the optimizations. GHC (Haskell), OCaml's compiler, and Lean's own kernel all follow this pattern: parse source text into an AST, elaborate it into a core calculus, type-check, then evaluate by repeated reduction.
The pipeline you've built is a miniature version of what every programming language implementation does. The components are the same—lexer, parser, AST, type checker, evaluator, REPL loop—just at a smaller scale. If you understand why each piece exists and how they connect, you understand the architecture of every compiler and interpreter.

23 The Landscape: From STLC to Lean

23.1 The Historical Arc

The progression from STLC to modern proof assistants spans half a century of interplay between logic, mathematics, and computer science. Here is the story in compressed form:

System	Year	Key addition	People
STLC	1940	Simple types	Church
System F	1972	Polymorphism ($\forall\alpha$)	Girard, Reynolds
ML type inference	1978	Hindley–Milner	Milner, Hindley
Martin-Löf TT	1971–84	Dependent types	Martin-Löf
CoC	1985	All axes of the cube	Coquand, Huet
CIC	1989	+ inductive types	Paulin-Mohring
Coq	1989	First mature proof assistant	Huet, Coquand et al.
Agda	1999–2007	Dependent types with patterns	Norell, Bove
Lean 4	2021	CIC + practical PL features	de Moura

Each row builds on the ones above it. Church invented simple types to fix the logical inconsistency Kleene and Rosser found. Girard added polymorphism to capture second-order logic. Martin-Löf added dependent types to capture constructive mathematics. Coquand unified them all. And de Moura made the result into a practical programming language.

23.2 What Each Extension Costs

Every extension pays for its expressiveness. The trade-offs reveal a deep tension between power and decidability:

STLC: Type checking is decidable and simple (our 15-line type checker). Type *inference* is also decidable—you don't need any annotations. But the system is too weak: you can't write generic code, you can't express recursion, and Church numerals can't be given a single type.

System F: Type checking is still decidable. But type *inference* is undecidable (Wells, 1999)—sometimes you must provide type annotations. The Hindley–Milner restriction trades full System F for complete inference, which is why Haskell and ML can often omit annotations while Java and Rust cannot.

Dependent types: Type *checking* requires evaluation, so it's only decidable if all programs terminate. This is the origin of Lean's termination checker and its insistence on structural recursion. Without termination, the type checker could loop forever. The reward: types can express arbitrary mathematical propositions, and type checking is proof verification.

23.3 The Philosophical Thread

The deepest lesson of this progression is that **types are the language of invariants**. At every level, the question is the same: what properties can you express and verify statically?

In the STLC, you can say "this function takes a number and returns a boolean." In System F, you can say "this function works at every type uniformly." With dependent types, you can say "this sorting function returns a list that is a permutation of the input and is in ascending order"—and the type checker will verify it.

This perspective connects back to Hilbert's original question that started this entire story (Section 1). Hilbert wanted a mechanical procedure that could verify mathematical proofs. The Entscheidungsproblem asked: is there an algorithm that, given a statement in formal logic, decides whether the statement is provable?

Church and Turing showed the answer is "no" in general. But dependent type theory gives a remarkable partial answer: *if you express your propositions as types, then type checking is your proof verifier*. You can't automatically *find* proofs (that's still undecidable), but you can automatically *check* them. The type checker is the mechanical proof verifier Hilbert was looking for—with the caveat that someone still has to write the proof (the program) by hand.

This is exactly what proof assistants do. When a mathematician formalizes a theorem in Lean, they are writing a program. When Lean's type checker accepts it, Lean has mechanically verified the proof. The 4539-line Lean kernel—a dependent type checker at its heart—is the closest thing we have to Hilbert's dream, built on the lambda calculus that Church invented to show the dream was impossible in its original form.

The unifying theme

Every system in this progression is a lambda calculus. The syntax is the same three constructs: variables, abstraction, application. The computation rule is the same: beta reduction. What changes is the *type system*—the rules that determine which terms are well-formed. The STLC is the starting point. Each extension adds a new kind of abstraction (over types, over type operators, or over values inside types), which makes the system more expressive while (carefully) preserving the properties that make it useful.
This is why understanding the STLC matters: it's not just a toy system. It's the foundation that every modern typed language builds on.

24 Common Mistakes and FAQ

This section collects the most common points of confusion for newcomers. If something earlier didn't quite click, you may find the answer here.

24.1 "Can I write $\lambda(x, y).\, e$ for a function of two arguments?"

No. Lambda calculus functions take exactly one argument—this is fundamental to the system, not just a syntactic limitation. The grammar allows only $\lambda x.\, e$ with a single variable after the λ. Multi-argument functions are handled via currying (Section 4.3): $\lambda x.\, \lambda y.\, e$ is a function that takes x and returns *another function* that takes y.

The shorthand $\lambda x\, y.\, e$ is just notation for $\lambda x.\, \lambda y.\, e$. It does not change the underlying system.

24.2 "What's the difference between $=$ and $\equiv_\alpha$?"

$=$ is used informally to mean "is the same term" or "we define this to be." $\equiv_\alpha$ (α-equivalence) is a specific technical relation: two terms are α-equivalent if they differ only in the names of their bound variables.

For example, $\lambda x.\, x \;\equiv_\alpha\; \lambda y.\, y$ because they are the same function with different parameter names. But we would not write $\lambda x.\, x = \lambda y.\, y$ in a formal context, because they are literally different strings of symbols.

In practice, most treatments (and our de Bruijn implementation) identify α-equivalent terms—they are considered "the same" for all purposes.

24.3 "Why does my reduction seem to go in circles?"

Some terms genuinely don't terminate. The classic example is $\Omega = (\lambda x.\, x\ x)\ (\lambda x.\, x\ x)$, which reduces to itself forever (Section 7).

If you're reducing by hand and keep getting the same term back, you probably have a non-terminating term. Try applying a different evaluation strategy—but be aware that some terms have *no* normal form under any strategy.

If you're using the interpreter and it returns the same thing you put in, the "fuel" (step limit) may have run out. Try increasing it.

24.4 "Is $\lambda x.\, x\ x$ typeable?"

No, not in the simply typed lambda calculus (STLC). For $x\ x$ to be well-typed, x must simultaneously be a function (type $A \to B$) and its own argument (type A). This requires $A = A \to B$, an infinite type. The STLC has no infinite types and no recursive types, so this term is untypeable.

This is why the **Y** combinator (which contains $x\ x$) also cannot be typed in the STLC. If you want recursion in a typed system, you need either an explicit `fix` operator, recursive types, or a more expressive type system like System F.

24.5 "What's the difference between $\longrightarrow_\beta$ and $\twoheadrightarrow_\beta$?"

$\longrightarrow_\beta$ (single-headed arrow) means **one step** of β-reduction: exactly one redex was reduced. $\twoheadrightarrow_\beta$ (double-headed arrow) means **zero or more steps**: the term reduces to the target after some (possibly zero) number of single steps. So $e \twoheadrightarrow_\beta e'$ means "there is a chain $e \longrightarrow_\beta e_1 \longrightarrow_\beta e_2 \longrightarrow_\beta \cdots \longrightarrow_\beta e'$." If the chain has zero steps, it means $e = e'$.

24.6 "$\overline{0}$ and False are the same term. Isn't that a problem?"

Yes, they are both $\lambda t\, f.\, f$ (select the second argument). Nil is also the same term. This is a genuine limitation of Church encoding: different "types" of data can collide because there are no types in untyped λ-calculus to distinguish them.

In practice, this means Church-encoded programs must be disciplined about which "type" of value they expect. A function expecting a boolean could be given a number and would not "know the difference." This is one of the motivations for adding a type system (Section 12), which ensures such confusions are caught at compile time.

24.7 "Why do I need de Bruijn indices? Named variables seem fine."

Named variables are fine for pen-and-paper work, but they have a nasty implementation problem: α-conversion. Every time you perform substitution, you must check for variable capture and potentially rename binders. This makes the code more complex and error-prone.

De Bruijn indices eliminate the problem entirely. Two α-equivalent terms are *literally identical* when written with de Bruijn indices: $\lambda x.\, x$ and $\lambda y.\, y$ both become $\lambda.\, 0$. No renaming is ever needed, and structural equality is α-equality. The trade-off is that de Bruijn terms are harder for humans to read, but easier for machines to manipulate.

24.8 "Can I use lambda calculus as an actual programming language?"

Technically yes—it is Turing-complete, so it can compute anything. In practice, raw λ-calculus is far too slow and unreadable for real work. Every functional programming language is essentially λ-calculus with ergonomic and performance improvements bolted on: named functions, pattern matching, type checking, native data types, efficient compilation, and so on.

The value of studying λ-calculus is not to use it directly, but to understand the *foundations* that all these languages are built on.

24.9 "Why is it called 'lambda'? What does the λ symbol mean?"

The origin is surprisingly mundane. Church originally used a circumflex ($\hat{x}$) to mark bound variables, then switched to Λx for typographical reasons, which was eventually simplified to λx. The symbol λ has no deep mathematical meaning—it is just a marker that says "a function is being defined here."

Part III

Building a Lambda Calculus Interpreter

You've learned the theory. Now imagine you want to *build* it—a program that takes a λ-expression as input, reduces it, and shows you the result. This part guides you through the process of designing and implementing such a program.

The approach here is deliberately *not* "here's the code, memorize it." Instead, each section poses a design challenge, gives you the tools and intuition to solve it yourself, and then reveals the answer. **Try each challenge before reading on.** Even a few minutes of honest struggle—sketching on paper, writing pseudo-code, getting stuck and thinking about *why* you're stuck—will teach you far more than passively reading the solution.

The reason this works is that every piece of implementation code is a direct translation of the theory from the earlier sections. If you understand why substitution has six cases, you can derive the substitution function. If you understand why de Bruijn indices exist, you can figure out what shifting must do. The code doesn't come from nowhere—it comes from the math, and the math comes from the problems the math was invented to solve. Our goal is for you to see each piece of code as *inevitable*: the only thing the implementation *could* look like, given the theory.

The complete, copy-pasteable implementation is collected in Appendix E. But **don't skip ahead**—the appendix is a reference, not a tutorial. The learning happens here.

We use Lean 4 as our implementation language because its own type system is built on a λ-calculus, giving a nice meta-circular elegance. But the ideas translate to any language. If you prefer, read the Lean code as pseudocode—the patterns (algebraic data types, pattern matching, recursion) exist in Python, Haskell, Rust, OCaml, TypeScript, and more.

25 Where to Go from Here

Prove correctness. Lean is a proof assistant—prove that evaluation preserves types ($\Gamma \vdash e : \tau \ \wedge \ e \longrightarrow_\beta e' \implies \Gamma \vdash e' : \tau$).

System F. Add polymorphism: $\Lambda\alpha.\, \lambda x{:}\alpha.\, x \ : \ \forall\alpha.\, \alpha \to \alpha$.

Dependent types. Implement a language where types depend on values, exploring the Curry–Howard correspondence between proofs and programs.

Compilation. Replace tree-walking with a stack machine, G-machine, or CPS transform for real performance.

A A Lean 4 Primer

This book uses a handful of Lean 4 conveniences that a reader coming from mainstream programming may not have met, and a couple of proof tactics. None of them are essential to the *ideas*—they are ergonomic sugar over plain recursion and pattern matching. This appendix introduces each one and, where it matters, shows the "desugared" code it expands to. Everything here compiles on `leanprover/lean4:v4.31.0` (core, no imports).

`deriving`: generated instances

When you write `deriving Repr, DecidableEq, Inhabited` after an inductive type, Lean *synthesizes* the corresponding instances for you, so you don't hand-write them:

```
inductive Color where
  | red | green | blue
  deriving Repr, DecidableEq, Inhabited

#eval (Color.red == Color.red)      -- true   (DecidableEq → BEq gives ==)
#eval (default : Color)             -- Color.red  (Inhabited: first constructor)
```

`Repr` gives a printable form (what `#eval` shows); `DecidableEq` gives decidable equality (and hence `==`); `Inhabited` names a default element (used by "partial" fallbacks like `List.head!`). You could write all three by hand; `deriving` just generates the obvious code.

`structure` and `instance`: type classes

A `structure` is a record; an `instance` registers an implementation of a type class (an interface). Once registered, operators like `+`, `==`, `>>=`, `<|>` resolve to it automatically:

```
structure Point where
  x : Nat
  y : Nat

instance : Add Point where
  add p q := ⟨p.x + q.x, p.y + q.y⟩

#eval ((⟨1, 2⟩ + ⟨3, 4⟩ : Point)     -- ⟨4, 6⟩ — `+` found the Add instance
```

This is exactly how the parser-combinator section makes `Parser` a `Monad` and `Alternative`: register the instances, then use `do` and `<|>`.

`do` / `←`: the `Option` monad

The single most-used piece of sugar in this book. Inside a `do` block for `Option`, `let a ← e` means "if `e` is `some a`, continue; if it is `none`, abandon the whole block and return `none`." It removes a staircase of nested `match`es. All three of these are the *same* function:

```
-- (1) do-notation
def firstTwo (xs : List Nat) : Option (Nat × Nat) := do
  let a ← xs[0]?
  let b ← xs[1]?
  pure (a, b)
```

```
-- (2) what it desugars to: Option.bind (written `.bind` here)
def firstTwoBind (xs : List Nat) : Option (Nat × Nat) :=
  xs[0]?.bind fun a =>
  xs[1]?.bind fun b =>
  pure (a, b)

-- (3) what `bind` itself does: match and short-circuit on none
def firstTwoMatch (xs : List Nat) : Option (Nat × Nat) :=
  match xs[0]? with
  | none   => none
  | some a =>
    match xs[1]? with
    | none   => none
    | some b => some (a, b)
```

← is `bind` plus a name; `pure x` is `some x`; `bind` is "match, propagate `none`." Whenever you see `do` in `toDeBruijn`, `typeCheck`, or the parser, mentally expand it to form (3).

`guard`: a mid-block failure test

`guard p` succeeds (with `()`) when `p` holds and fails (`none`) otherwise. In a `do` block it is a guard clause:

```
def evenHalf (n : Nat) : Option Nat := do
  guard (n % 2 == 0)      -- bail out with `none` unless n is even
  pure (n / 2)

-- desugared:
def evenHalfManual (n : Nat) : Option Nat :=
  if n % 2 == 0 then pure (n / 2) else none
```

The dependent type checker uses `guard (dom.beq argTy)` the same way: "unless the argument type matches, reject."

`<|>`: choice (`Alternative`)

`p <> q|` means "try `p`; if it fails, try `q`." For `Option` it is just:

```
def firstSome (a b : Option Nat) : Option Nat := a <|> b

-- desugared:
def orElseManual (a b : Option Nat) : Option Nat :=
  match a with | some x => some x | none => b
```

The parser combinators use `lamP <> appP|` to mean "a lambda, or else an application."

`_ matches _`: a pattern as a Bool

`e matches pat` is `true` exactly when `e` fits the pattern. It desugars to a `match` that returns a `Bool`:

```
#eval (some 3 matches Option.some _)   -- true
#eval (([] : List Nat) matches _ :: _) -- false
```

```
-- `x matches some _` is literally:
--   match x with | Option.some _ => true | _ => false
```

The checker writes `aTy.whnf matches .univ` to ask "did this reduce to `Type`?"

`Id.run do` + `let mut`: imperative-looking folds

Lean has no mutable state, but `Id.run do` lets you *write* code that looks imperative—`let mut` plus reassignment—and compiles it down to a pure fold. These two are equal:

```
def sumTo (n : Nat) : Nat := Id.run do
  let mut acc := 0
  for i in [0:n] do
    acc := acc + i
  return acc

-- what it is under the hood: a left fold, no mutation
def sumToFold (n : Nat) : Nat := (List.range n).foldl (· + ·) 0
```

The Church-encoding definition lists (`Id.run do … let mut ds := …`) use this to build a list step by step; it is a fold in disguise.

`partial` and `termination_by`

Lean accepts a recursive `def` only once it is convinced the recursion *stops*. It does this automatically when each recursive call is on a structural piece of an argument. When it cannot see why a function halts, you have two choices.

Opt out with `partial`. This tells Lean "trust me, this terminates" and skips the check. It is what `freshVar`, `subst` (named version), `whnf`, and the parser use:

```
-- Collatz: nobody has proved this halts for all n, so we must use `partial`.
partial def collatzLen (n : Nat) : Nat :=
  if n ≤ 1 then 0
  else 1 + collatzLen (if n % 2 == 0 then n / 2 else 3 * n + 1)
```

Prove it with `termination_by`. When there *is* a measure that shrinks, name it, and discharge the "it decreases" goal (often `omega` suffices):

```
def log2 : Nat → Nat
  | 0     => 0
  | 1     => 0
  | n + 2 => 1 + log2 ((n + 2) / 2)
  termination_by n => n        -- the measure that decreases
  decreasing_by omega          -- prove (n + 2) / 2 < n + 2

#eval log2 8   -- 3
```

The `fuel` parameter used for `eval` is a third option: make the recursion structural on a `Nat` that always counts down, so no proof is needed at all.

`induction` / `cases` ↔ the recursor

`induction` and `cases` are tactics, but they are not primitive: each compiles to a call of the type's automatically generated *recursor* (`T.rec`). Here is a proof both ways—first with the tactic, then as the raw term it elaborates to:

```
-- with the `induction` tactic
theorem zero_add_tac (n : Nat) : 0 + n = n := by
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih]

-- the same proof as a term: Nat.rec with an explicit motive
theorem zero_add_rec (n : Nat) : 0 + n = n :=
  Nat.rec (motive := fun n => 0 + n = n)
    rfl                                    -- base case: 0 + 0 = 0
    (fun k ih => by rw [Nat.add_succ, ih]) -- step: uses the hypothesis `ih`
    n
```

`cases` is the same idea with no induction hypothesis—it just splits on the constructor:

```
theorem someOfNe {x : Option Nat} (h : x ≠ none) : ∃ y, x = some y := by
  cases x with
  | none   => exact absurd rfl h
  | some y => exact ⟨y, rfl⟩
```

`obtain ⟨…⟩` ↔ `Exists.elim` / `match`

`obtain` destructures a hypothesis. For an existential it pulls out the witness and its property; underneath, that is `Exists.elim` (equivalently a `match`):

```
-- with obtain
theorem obtainDemo (h : ∃ n : Nat, n > 5) : ∃ m : Nat, m > 4 := by
  obtain ⟨n, hn⟩ := h
  exact ⟨n, by omega⟩

-- desugared: Exists.elim hands you (n, hn)
theorem obtainDemoElim (h : ∃ n : Nat, n > 5) : ∃ m : Nat, m > 4 :=
  Exists.elim h (fun n hn => ⟨n, by omega⟩)
```

The anonymous-constructor brackets `⟨n, hn⟩` build *and* take apart structures (pairs, `And`, `Exists`, `Point`, …).

`by omega`: linear arithmetic, decided

`omega` is a decision procedure for linear arithmetic over `Nat` and `Int` (equalities, `<`, `≤`, `+`, constant multiples, and their Boolean combinations). It is *not* magic and *not* a general prover—it just happens to settle the small arithmetic goals we hit:

```
example (a b : Nat) (h : a + 1 = b) : a < b := by omega
example : ∃ n : Nat, n > 5 := ⟨6, by omega⟩    -- proves 6 > 5
```

When you see `by omega` in the book (e.g. the BHK `exists_gt_5` example), read it as "the arithmetic here is routine—`omega` checks it."

B Quick Reference Card

Core Syntax and Rules

$$
\begin{aligned}
e &::= x \mid \lambda x.\, e \mid e_1\ e_2 \\
\alpha: &\quad \lambda x.\, e \ \equiv_\alpha\ \lambda y.\, e[x := y] \quad (y \text{ fresh}) \\
\beta: &\quad (\lambda x.\, e)\ a \ \longrightarrow_\beta\ e[x := a] \\
\eta: &\quad \lambda x.\, f\ x \ \longrightarrow_\eta\ f \quad (x \notin \mathrm{FV}(f))
\end{aligned}
$$

Famous Combinators

$$
\begin{aligned}
\mathbf{I} &= \lambda x.\, x & \mathbf{K} &= \lambda x\, y.\, x & \mathbf{S} &= \lambda f\, g\, x.\, f\ x\ (g\ x) \\
\Omega &= (\lambda x.\, x\ x)^2 & \mathbf{Y} &= \lambda f.\, (\lambda x.\, f(x\ x))^2
\end{aligned}
$$

Church Encodings

$$
\begin{aligned}
\textsc{True} &= \lambda t\, f.\, t & \textsc{False} &= \lambda t\, f.\, f \\
\overline{n} &= \lambda f\, x.\, f^n\ x & \textsc{Succ} &= \lambda n\, f\, x.\, f\ (n\ f\ x) \\
\textsc{Add} &= \lambda m\, n\, f\, x.\, m\ f\ (n\ f\ x) & \textsc{Mul} &= \lambda m\, n\, f.\, m\ (n\ f) \\
\textsc{Pair} &= \lambda a\, b\, f.\, f\ a\ b & \textsc{Fst} &= \lambda p.\, p\ \textsc{True}
\end{aligned}
$$

De Bruijn

$$
(\lambda.\, e)\ s \ \longrightarrow_\beta \uparrow_0^{-1} \left(e[0 := \uparrow_0^1 (s)]\right)
$$

STLC Typing Rules

$$
\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x{:}\tau_1.\, e : \tau_1 \to \tau_2}\ \textsc{Abs} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \textsc{App}
$$

C Glossary

Term	**Definition**
Abstraction	$\lambda x.\, e$: a function with parameter x and body e.
α-conversion	Renaming bound variables; $\lambda x.\, x \equiv_\alpha \lambda y.\, y$.
β-reduction	$(\lambda x.\, e)\ a \longrightarrow_\beta e[x := a]$. The computation rule.
η-conversion	$\lambda x.\, f\ x = f$ when $x \notin \mathrm{FV}(f)$. Extensionality.
Church encoding	Representing data as pure λ-terms.
Church–Rosser	Normal forms are unique up to $\equiv_\alpha$.
Combinator	A closed term ($\mathrm{FV}(e) = \varnothing$).
Context (Γ)	A list of variable-type bindings for judgments.
De Bruijn index	Nameless variable: a number counting binders outward.
Fixed-point combinator	$\mathbf{Y}$ with $\mathbf{Y}\ f = f\ (\mathbf{Y}\ f)$.
Inference rule	Premises above a line, conclusion below.
Normal form	A term with no redexes; fully reduced.
Redex	A reducible expression $(\lambda x.\, e)\ a$.
Substitution	$e[x := a]$: replace free x in e with a, avoiding capture.
Turnstile ($\vdash$)	Separates context from conclusion in $\Gamma \vdash e : \tau$.
WHNF	A term that is a λ or has no top-level redex.

D Exercise Solutions

Solutions are provided in condensed form. Working through the exercises yourself before reading the solutions is strongly recommended.

Variables and Renaming (Section 6)

5.1. $\mathrm{FV}(\lambda x.\, \lambda y.\, x\ z\ y) = \{z\}$. The variables x and y are bound by their respective λ's. Only z is free.

5.2. Yes, $\lambda f.\, \lambda x.\, f\ (f\ x)$ is closed. FV $= \varnothing$: both f and x are bound. This is the Church numeral $\overline{2}$.

5.3. The inner λx shadows the outer one. The inner x refers to the inner binder. Alpha-convert the outer: $\lambda x.\, \lambda x.\, x \equiv_\alpha \lambda y.\, \lambda x.\, x$. This is $\mathbf{K}^* = \lambda y.\, \lambda x.\, x$: a function that takes two arguments and returns the *second* (the opposite of $\mathbf{K}$). Equivalently, it is $\lambda y.\, \mathbf{I}$.

5.4. No. $\lambda x.\, x\ y$ has FV $= \{y\}$, but $\lambda y.\, y\ y$ has FV $= \varnothing$. Alpha conversion can only rename bound variables—it cannot change the set of free variables. These terms have different free variable sets, so they cannot be α-equivalent.

Beta Reduction (Section 7)

6.1. $(\lambda x.\, x\ x)\ (\lambda y.\, y) \longrightarrow_\beta (\lambda y.\, y)\ (\lambda y.\, y) \longrightarrow_\beta \lambda y.\, y$. The normal form is the identity $\mathbf{I}$.

6.2. Let $\omega = \lambda y.\, y\ y$. Then $(\lambda f.\, \lambda x.\, f\ (f\ x))\ \omega \longrightarrow_\beta \lambda x.\, \omega\ (\omega\ x)$. Normalizing the body (normal order) actually *terminates* in three more steps:

$$\lambda x.\, \omega\,(\omega\, x) \longrightarrow_\beta \lambda x.\, (\omega\, x)\,(\omega\, x) \longrightarrow_\beta \lambda x.\, (x\, x)\,(\omega\, x) \longrightarrow_\beta \lambda x.\, (x\, x)\,(x\, x).$$

The result $\lambda x.\, (x\, x)\,(x\, x)$ **is a normal form**: its only applications are $x\, x$, and x is a variable, not a λ, so there is no redex to fire. The term does *not* diverge on its own. Divergence appears only if you later *apply* this normal form to an argument—feeding it, say, ω or itself re-creates Ω-style self-application that loops. Having a normal form and looping-when-applied are different properties; see the box "Normal form vs. diverges-when-applied" in Section 8.

6.3. Two redexes: (1) the outer application $(\lambda x.\, x)\ ((\lambda y.\, y)\ z)$ is a redex (the whole expression), and (2) the inner application $(\lambda y.\, y)\ z$ inside the argument is also a redex.

6.4. No, $\lambda x.\, (\lambda y.\, y)\ x$ is not in normal form—the body contains the redex $(\lambda y.\, y)\ x$. Reducing: $(\lambda y.\, y)\ x \longrightarrow_\beta x$, giving $\lambda x.\, x$. Then yes, we can η-reduce… but $\lambda x.\, x$ is already the identity; there is no f to η-reduce to (the body is x, not $f\ x$ for some f with $x \notin \mathrm{FV}(f)$). So the final normal form is $\lambda x.\, x = \mathbf{I}$.

6.5. We need $(\lambda y.\, x\ y)[x := \lambda y.\, y]$. The binder is λy, so we check whether the replacement term $\lambda y.\, y$ has y free. It does not: $\mathrm{FV}(\lambda y.\, y) = \varnothing$ (the y inside $\lambda y.\, y$ is bound by its own λ). Since $y \notin \mathrm{FV}(\lambda y.\, y)$, no capture is possible, so **rule 4 (the safe case) applies**—push the substitution under the binder unchanged: $\lambda y.\, (x\ y)[x := \lambda y.\, y] = \lambda y.\, (\lambda y.\, y)\ y$. The result $\lambda y.\, (\lambda y.\, y)\ y$ reduces to $\lambda y.\, y$, which is correct (η-equivalent to the original).

Church Encodings (Section 10)

9.1. IsZero (Succ $\overline{0}$): First, Succ $\overline{0} \twoheadrightarrow_\beta \overline{1} = \lambda f\, x.\, f\ x$. Then IsZero $\overline{1} = \overline{1}$ ($\lambda\_.$ False) True $\twoheadrightarrow_\beta$ ($\lambda\_.$ False) True $\longrightarrow_\beta$ False.

9.2. Add $\overline{2}\ \overline{3} = (\lambda m\, n\, f\, x.\, m\ f\ (n\ f\ x))\ \overline{2}\ \overline{3}$. Substituting: $\lambda f\, x.\, \overline{2}\ f\ (\overline{3}\ f\ x) \twoheadrightarrow_\beta \lambda f\, x.\, f\ (f\ (f\ (f\ x)))) = \overline{5}$.

9.3. Sub $\triangleq \lambda m\, n.\, n$ Pred m. Apply Pred a total of n times to m. Since $\overline{n}$ applies its first argument n times to its second, n Pred m does exactly this.

9.4. Length $\triangleq \lambda l.\, l\ (\lambda\_\ acc.\, \text{Succ}\ acc)\ \overline{0}$. Fold over the list: for each element (ignored), increment the accumulator. Start from $\overline{0}$.

9.5. Yes, Nil $= \lambda c\, n.\, n$ and $\overline{0} = \lambda f\, x.\, x$ are the same term (up to renaming: $c \leftrightarrow f$, $n \leftrightarrow x$). They are α-equivalent. This illustrates a fundamental limitation of Church encoding in untyped λ-calculus: different "types" of data can be identical terms. Only a type system can distinguish them.

Types (Section 12)

11.1. The proof tree for the **S** combinator has type $(A \to B \to C) \to (A \to B) \to A \to C$. It requires three Abs rules (one for each λ), three App rules (one for $x\ z$, one for $y\ z$, and one combining them into $x\ z\ (y\ z)$), and three Var axioms (x, y, z):

$$\dfrac{\dfrac{\dfrac{\dfrac{x : A{\to}B{\to}C}{\Gamma \vdash x : A{\to}B{\to}C}\,\text{V} \quad \dfrac{z : A}{\Gamma \vdash z : A}\,\text{V}}{\Gamma \vdash x\ z : B{\to}C}\,\text{App} \quad \dfrac{\dfrac{y : A{\to}B}{\Gamma \vdash y : A{\to}B}\,\text{V} \quad \dfrac{z : A}{\Gamma \vdash z : A}\,\text{V}}{\Gamma \vdash y\ z : B}\,\text{App}}{\Gamma \vdash x\ z\ (y\ z) : C}\,\text{App}}{\vdash \lambda x\, y\, z.\, x\ z\ (y\ z) : (A{\to}B{\to}C) \to (A{\to}B) \to A \to C}\,\text{Abs}{\times}3$$

where $\Gamma = x : A{\to}B{\to}C,\ y : A{\to}B,\ z : A$.

11.2. For $x\ x$ to type-check, x must have a function type $A \to B$ (left of the application) and also be a valid argument (type A). So we need $A \to B = A$, which means $A = A \to B = (A \to B) \to B = \ldots$—an infinite type. The STLC has only finite types, so $\lambda x.\, x\ x$ is untypeable.

11.3. $\lambda f{:}(A \to A).\, \lambda x{:}A.\, f\ (f\ x)$ has type $(A \to A) \to A \to A$. This is the same as $A \to A$ where we've substituted $A \to A$ for A. Every Church numeral $\overline{n}$ has this same type. Under Curry–Howard, it's a proof of "if A implies A, then A implies A"—which is trivially true, and there are infinitely many distinct proofs (one per numeral).

De Bruijn Substitution (Section 11.2)

10.1. The naïve version fails on $\lambda.\, \lambda.\, 0\ 1\ 2$: indices 0 and 1 are bound (by the inner and outer λs respectively), and index 2 is the free variable that corresponds to external variable 0. `substNaive` with $k = 0$ incorrectly replaces `bvar 0` (which is the innermost λ's own parameter) with s, while leaving the actual target (`bvar 2`) untouched.

10.2. `substV2` 0 (`bvar` 0) ($\lambda.\, 1$): Named version is "substitute y for x in $\lambda z.\, x$." Result should be $\lambda z.\, y$, i.e., $\lambda.\, 1$ (index 1 skips past z's binder to reach y). But `substV2` produces $\lambda.\, 0$—the replacement `bvar` 0 was dropped in unchanged, and inside the λ it points to z instead of y. The fix: shift s up when entering the λ.

10.3. (a) Replace with s when $n = k$; leave as `bvar` n when $n \neq k$. (b) Recurse: `subst` k s into both sides independently. (c) Both fixes apply: the target k increments to $k+1$ (fix 1) because the variable we're hunting for is one more binder away, and the replacement s gets shifted up by 1 via `shift` 1 0 s (fix 2) because its free variables need to skip the new binder. (d) Fix 1 without fix 2: correctly finds the target variable but inserts s with wrong indices (capture bug from stage 2). Fix 2 without fix 1: shifts s correctly but looks for the wrong variable inside the λ (would replace bound variables or miss the target, like stage 1).

10.4. k goes $0 \to 1 \to 2 \to 3$. Yes, index 3 inside three binders refers to the same variable as index 0 outside. The substitution fires correctly.

10.5. See the worked trace in the exercise box.

10.6. Body is $\lambda.\, 1$, argument is `bvar` 0.

Step 1: `shift` 1 0 (`bvar` 0) = `bvar` 1.

Step 2: `subst` 0 (`bvar` 1) ($\lambda.\, 1$). We hit the λ: recurse with $k = 1$, s = `shift` 1 0 (`bvar` 1) = `bvar` 2.

Now: `subst` 1 (`bvar` 2) (`bvar` 1). Is $1 = 1$? Yes! Result: `bvar` 2.

After wrapping: $\lambda.\, 2$.

Step 3: `shift` (-1) 0 ($\lambda.\, 2$). Inside the λ, cutoff is 1. Is $2 \geq 1$? Yes: $2 - 1 = 1$. Result: $\lambda.\, 1$.

Final: $\lambda.$ `bvar` 1—a function that ignores its argument and returns the free variable. ✓

Going Deeper (Section 14)

13.1. $\textsc{Succ}_S \triangleq \lambda n.\, \lambda z\, s.\, s\ n$. This creates a new Scott numeral that, when pattern-matched, calls the successor handler s with n as the predecessor. Verification: $\textsc{Succ}_S\ \overline{2}_S = (\lambda n.\, \lambda z\, s.\, s\ n)\ \overline{2}_S \longrightarrow_\beta \lambda z\, s.\, s\ \overline{2}_S = \overline{3}_S$.

13.2. $\textsc{IsZero}_S \triangleq \lambda n.\, n\ \textsc{True}\ (\lambda\_.\, \textsc{False})$. If n is zero, it selects the first handler (True). If n is a successor, it calls the second handler (which ignores the predecessor and returns False). This is much simpler than the Church version—a single reduction step vs. the Church version's $O(n)$ iterations.

13.3. $(A \to B) \to (B \to C) \to (A \to C)$ corresponds to the transitivity of implication: "if A implies B and B implies C, then A implies C." A witnessing term: $\lambda f{:}(A \to B).\, \lambda g{:}(B \to C).\, \lambda x{:}A.\, g\ (f\ x)$. This is function composition.

13.4. There is no closed term of type $A \to B$ when $A \neq B$ because to produce a B, you would need either a variable of type B or a function that returns B. But the only variable available is $x : A$, and there is no way to convert an A into a B. Under Curry–Howard, this means the proposition "A implies B" (for unrelated A and B) is not provable—which is correct.

13.5. $\forall \alpha.\, \alpha$ says "for every type α, I can produce a value of type α." Under Curry–Howard, this is the proposition "everything is true"—i.e., falsity ($\bot$). It is uninhabited because no single term can work at every type. (In System F, the term would need to simultaneously be a natural number, a boolean, a function, etc.)

E Complete Implementation

This appendix collects the complete Lean 4 implementation from the implementation sections in one place, suitable for copy-pasting into a project. The code is organized by module.

Project Layout

```
LambdaCalc/
├── lakefile.lean
├── LambdaCalc.lean
└── LambdaCalc/
    ├── Named.lean       -- Named terms, FV, substitution, eval
    ├── DeBruijn.lean    -- De Bruijn terms, shift, subst, eval
    ├── Parser.lean      -- Recursive descent parser
    ├── Defs.lean        -- Church encoding definitions + expandDefs
    ├── Typed.lean       -- STLC type checker
    └── Main.lean        -- REPL
```

Named.lean — Named Representation

```
inductive Term where
  | var : String → Term
  | lam : String → Term → Term
  | app : Term → Term → Term
  deriving Repr, DecidableEq, Inhabited

open Term

def Term.toString : Term → String
  | .var x     => x
  | .lam x e   => s!"(λ{x}. {e.toString})"
  | .app e₁ e₂ => s!"({e₁.toString} {e₂.toString})"

instance : ToString Term := ⟨Term.toString⟩

def freeVars : Term → List String
  | .var x     => [x]
  | .lam x e   => (freeVars e).filter (· != x)
  | .app e₁ e₂ => (freeVars e₁ ++ freeVars e₂).eraseDups

partial def freshVar (avoid : List String) (base : String := "x") : String :=
  let rec go (n : Nat) : String :=
    let candidate := s!"{base}{n}"
    if candidate ∈ avoid then go (n + 1) else candidate
  if base ∉ avoid then base else go 0

partial def subst (x : String) (s : Term) : Term → Term
  | .var y     => if y == x then s else .var y
  | .app e₁ e₂ => .app (subst x s e₁) (subst x s e₂)
  | .lam y e   =>
    if y == x then .lam y e
    else if y ∉ freeVars s then
```

```
      .lam y (subst x s e)
    else
      let z := freshVar (freeVars s ++ freeVars e ++ [x])
      .lam z (subst x s (subst y (.var z) e))

def betaStep : Term → Option Term
  | .app (.lam x body) arg => some (subst x arg body)
  | .app e₁ e₂ =>
    match betaStep e₁ with
    | some e₁' => some (.app e₁' e₂)
    | none     => match betaStep e₂ with
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | .lam x e => match betaStep e with
                | some e' => some (.lam x e')
                | none    => none
  | _ => none

def eval (fuel : Nat) (t : Term) : Term :=
  match fuel with
  | 0         => t
  | fuel' + 1 => match betaStep t with
                 | none    => t
                 | some t' => eval fuel' t'
```

DeBruijn.lean — De Bruijn Representation

```
inductive Expr where
  | bvar : Nat → Expr
  | lam  : Expr → Expr
  | app  : Expr → Expr → Expr
  deriving Repr, DecidableEq, Inhabited

open Expr

def Expr.toString : Expr → String
  | .bvar n    => s!"{n}"
  | .lam e     => s!"(λ. {e.toString})"
  | .app e₁ e₂ => s!"({e₁.toString} {e₂.toString})"

instance : ToString Expr := ⟨Expr.toString⟩

def shift (d : Int) (c : Nat) : Expr → Expr
  | .bvar n    => if n ≥ c then .bvar (Int.toNat (n + d)) else .bvar n
  | .lam e     => .lam (shift d (c + 1) e)
  | .app e₁ e₂ => .app (shift d c e₁) (shift d c e₂)

def subst (k : Nat) (s : Expr) : Expr → Expr
  | .bvar n    => if n == k then s else .bvar n
  | .lam e     => .lam (subst (k + 1) (shift 1 0 s) e)
  | .app e₁ e₂ => .app (subst k s e₁) (subst k s e₂)
```

```
def betaReduce (body arg : Expr) : Expr :=
  shift (-1) 0 (subst 0 (shift 1 0 arg) body)

-- Normal order (leftmost outermost)
def step : Expr → Option Expr
  | .app (.lam body) arg => some (betaReduce body arg)
  | .app e₁ e₂ =>
    match step e₁ with
    | some e₁' => some (.app e₁' e₂)
    | none     => match step e₂ with
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | .lam e => match step e with
              | some e' => some (.lam e')
              | none    => none
  | _ => none

-- Call-by-value
def isValue : Expr → Bool
  | .lam _ => true
  | _      => false

def cbvStep : Expr → Option Expr
  | .app (.lam body) arg =>
    if isValue arg then some (betaReduce body arg)
    else match cbvStep arg with
         | some arg' => some (.app (.lam body) arg')
         | none      => none
  | .app e₁ e₂ =>
    match cbvStep e₁ with
    | some e₁' => some (.app e₁' e₂)
    | none     => match cbvStep e₂ with
                  | some e₂' => some (.app e₁ e₂')
                  | none     => none
  | _ => none

def eval (fuel : Nat) (e : Expr) : Expr :=
  match fuel with
  | 0         => e
  | fuel' + 1 => match step e with
                 | none    => e
                 | some e' => eval fuel' e'
```

Parser.lean — Recursive Descent Parser

```
-- We track the position as a plain Nat index into the input's
-- character list. (Earlier editions indexed with `String.Pos`; that
-- API was redesigned in recent Lean, so a Nat over `input.toList` is
-- both simpler and stable. See the "String positions" note below.)
structure PState where
  input : List Char
  pos   : Nat
```

```
  deriving Repr

abbrev ParseResult (α : Type) := Option (α × PState)

def PState.peekC (s : PState) : Option Char := s.input[s.pos]?

partial def skipWS (s : PState) : PState :=
  match s.peekC with
  | some c => if c == ' ' || c == '\t'
              then skipWS { s with pos := s.pos + 1 } else s
  | none => s

def peek (s : PState) : Option Char := s.peekC

def expect (s : PState) (c : Char) : ParseResult Unit :=
  match s.peekC with
  | some c' => if c' == c
               then some ((), skipWS { s with pos := s.pos + 1 })
               else none
  | none => none

def isIdentStart (c : Char) : Bool := c.isAlpha || c == '_'
def isIdentCont (c : Char) : Bool := c.isAlphanum || c == '_' || c == '\''

partial def parseIdent (s : PState) : ParseResult String :=
  let s := skipWS s
  match s.peekC with
  | some c =>
    if isIdentStart c then
      let rec go (p : Nat) : Nat :=
        match s.input[p]? with
        | some c' => if isIdentCont c' then go (p + 1) else p
        | none    => p
      let endP := go (s.pos + 1)
      let name := String.ofList (s.input.drop s.pos |>.take (endP - s.pos))
      some (name, skipWS { s with pos := endP })
    else none
  | none => none

mutual
partial def parseExpr (s : PState) : ParseResult Term :=
  let s := skipWS s
  match peek s with
  | some '\\' => parseLambda s
  | _         => parseApp s

partial def parseLambda (s : PState) : ParseResult Term :=
  match expect s '\\' with
  | none => none
  | some (_, s) =>
    let rec parseParams (s : PState) (acc : List String)
        : ParseResult (List String) :=
      match parseIdent s with
```

```
      | some (name, s') => parseParams s' (acc ++ [name])
      | none => if acc.isEmpty then none else some (acc, s)
    match parseParams s [] with
    | none => none
    | some (params, s) =>
      match expect s '.' with
      | none => none
      | some (_, s) =>
        match parseExpr s with
        | none => none
        | some (body, s) =>
          let term := params.foldr (fun p acc => Term.lam p acc) body
          some (term, s)

partial def parseApp (s : PState) : ParseResult Term :=
  match parseAtom s with
  | none => none
  | some (first, s) =>
    let rec go (s : PState) (acc : Term) : Term × PState :=
      match parseAtom s with
      | some (next, s') => go s' (Term.app acc next)
      | none => (acc, s)
    let (result, s) := go s first
    some (result, s)

partial def parseAtom (s : PState) : ParseResult Term :=
  let s := skipWS s
  match peek s with
  | some '(' =>
    match expect s '(' with
    | none => none
    | some (_, s) =>
      match parseExpr s with
      | none => none
      | some (e, s) =>
        match expect s ')' with
        | none => none
        | some (_, s) => some (e, s)
  | _ =>
    match parseIdent s with
    | some (name, s) => some (Term.var name, s)
    | none => none
end

def parse (input : String) : Option Term :=
  let s : PState := skipWS { input := input.toList, pos := 0 }
  match parseExpr s with
  | some (term, _) => some term
  | none           => none
```

Defs.lean — Definitions Environment

```
def expandDefs (defs : List (String × Term)) : Term → Term
  | Term.var x =>
    match defs.lookup x with
    | some t => t
    | none   => Term.var x
  | Term.lam x e =>
    let defs' := defs.filter (fun (n, _) => n != x)
    Term.lam x (expandDefs defs' e)
  | Term.app e₁ e₂ =>
    Term.app (expandDefs defs e₁) (expandDefs defs e₂)

def churchDefs : List (String × Term) :=
  let t name := Term.var name
  let l := Term.lam
  let a := Term.app
  [ ("true",  l "t" (l "f" (t "t"))),
    ("false", l "t" (l "f" (t "f"))),
    ("and",   l "p" (l "q" (a (a (t "p") (t "q"))
                              (l "t" (l "f" (t "f")))))),
    ("or",    l "p" (l "q" (a (a (t "p")
                              (l "t" (l "f" (t "t"))))
                              (t "q")))),
    ("not",   l "p" (a (a (t "p")
                       (l "t" (l "f" (t "f"))))
                       (l "t" (l "f" (t "t"))))),
    ("zero",  l "f" (l "x" (t "x"))),
    ("succ",  l "n" (l "f" (l "x"
               (a (t "f") (a (a (t "n") (t "f")) (t "x")))))),
    ("add",   l "m" (l "n" (l "f" (l "x"
               (a (a (t "m") (t "f"))
                  (a (a (t "n") (t "f")) (t "x"))))))),
    ("mul",   l "m" (l "n" (l "f"
               (a (t "m") (a (t "n") (t "f")))))),
    ("id",    l "x" (t "x")),
    ("const", l "x" (l "y" (t "x")))
  ]

def toDeBruijn (ctx : List String := []) : Term → Option Expr
  | Term.var x =>
    match ctx.idxOf? x with
    | some i => some (Expr.bvar i)
    | none   => none
  | Term.lam x body =>
    match toDeBruijn (x :: ctx) body with
    | some e => some (Expr.lam e)
    | none   => none
  | Term.app e₁ e₂ => do
    let e₁' ← toDeBruijn ctx e₁
    let e₂' ← toDeBruijn ctx e₂
    some (Expr.app e₁' e₂')
```

Typed.lean — STLC Type Checker

```
inductive Ty where
  | base  : String → Ty
  | arrow : Ty → Ty → Ty
  deriving Repr, DecidableEq

inductive TExpr where
  | bvar : Nat → TExpr
  | lam  : Ty → TExpr → TExpr
  | app  : TExpr → TExpr → TExpr
  deriving Repr

def typeCheck (ctx : List Ty) : TExpr → Option Ty
  | .bvar n          => ctx[n]?
  | .lam paramTy body =>
    match typeCheck (paramTy :: ctx) body with
    | some bodyTy => some (.arrow paramTy bodyTy)
    | none        => none
  | .app e₁ e₂ => do
    let funTy ← typeCheck ctx e₁
    let argTy ← typeCheck ctx e₂
    match funTy with
    | .arrow paramTy retTy =>
      if paramTy == argTy then some retTy else none
    | _ => none
```

Main.lean — REPL

```
def evalString (defs : List (String × Term)) (input : String)
    : String := Id.run do
  let some parsed := parse input | return "Parse error"
  let expanded := expandDefs defs parsed
  let some db := toDeBruijn [] expanded | return "Unbound variable"
  let result := eval 10000 db
  s!"{result}"

partial def repl (defs : List (String × Term)
    := churchDefs) : IO Unit := do
  IO.print "λ> "
  let input ← (← IO.getStdin).getLine
  let input := input.trim
  if input == ":quit" then return
  if input == ":defs" then
    for (name, _) in defs do IO.print s!"{name} "
    IO.println ""; repl defs; return
  if input.startsWith ":let " then
    let rest := input.drop 5 |>.trim
    match rest.splitOn " = " with
    | [name, expr] =>
      match parse expr with
      | some term =>
```

```
        let term := expandDefs defs term
        IO.println s!"{name} defined"
        repl ((name, term) :: defs)
      | none => IO.println "Parse error in definition"; repl defs
    | _ => IO.println "Usage: :let name = expr"; repl defs
    return
  match parse input with
  | none => IO.println "Parse error"; repl defs
  | some namedTerm =>
    let expanded := expandDefs defs namedTerm
    IO.println s!"Parsed:    {expanded}"
    match toDeBruijn [] expanded with
    | none => IO.println "Error: unbound variable"; repl defs
    | some dbTerm =>
      IO.println s!"De Bruijn: {dbTerm}"
      IO.println s!"Result:    {eval 10000 dbTerm}"
      IO.println ""; repl defs

def main : IO Unit := repl
```